

Δέντρα - Γράφοι

1 Γενικά-Περιγραφή δέντρων

Ένα (πριωμένο) δένδρο T είναι ένα σύνολο κόμβων που αποθηκεύουν στοιχεία σε σχέση γονέα-παιδιού με τις εξής ιδιότητες:

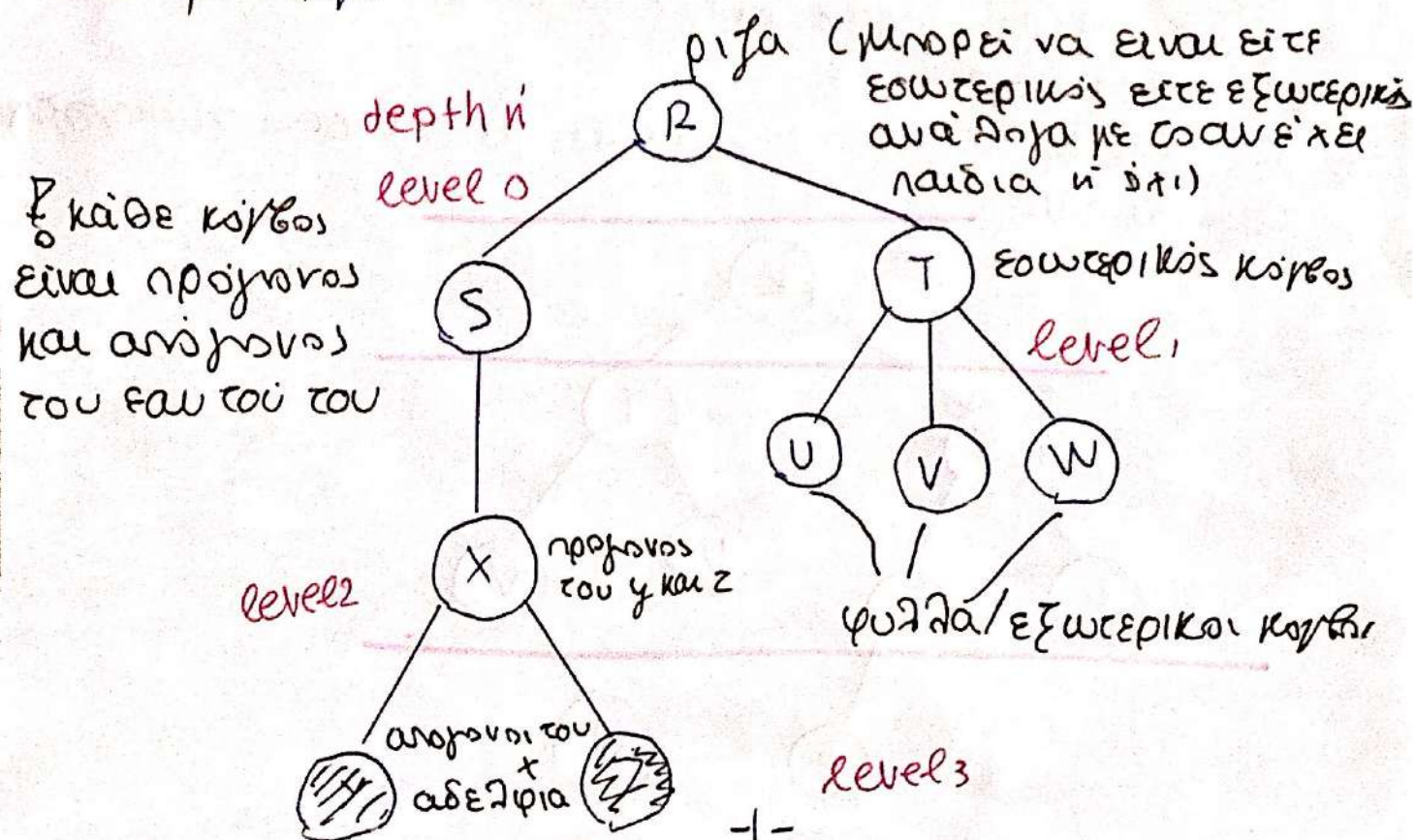
♥ Αν T δεν είναι κενό, τότε έχει έναν ειδικό κόμβο ο οποίος ονομάζεται ρίζα και δεν έχει γονέα

♥ Κάθε κόμβος v του T διαφορετικός από την ρίζα έχει έναν μόνο γονέα. Κάθε κόμβος με γονέα w είναι παιδί του w

=> **κενό δένδρο**: Το δένδρο που δεν έχει κόμβους υψος $\rightarrow -1$

=> **Ακμή (edge)**: του δένδρου, είναι ένα ζεύγος κόμβων (u, v) τέτοιο ώστε u είναι γονέας του v ή το αντίστροφο

=> **Μονοπάτι**: Μια ακολουθία κόμβων που συνδέονται με ακμές



! Αντι για επίπεδο μπορούμε να χρησιμοποιήσουμε το βάθος

? Το μήκος του μονοπατιού από τη ρίζα μέχρι είναι κόμβος είναι ίσο με το βάθος του κόμβου

→ Το **ύψος** του δένδρου είναι το μεγαλύτερο βάθος

⇒ **Υποδένδρο**: ενός δένδρου που έχει ρίζα είναι κόμβος v είναι το δένδρο που αποτελείται από όλους τους απογόνους του v συμπεριλαμβανομένου του v .

⇒ **Ταξινομημένο**: δένδρο που υπάρχει γραμμική διάταξη των παιδιών κάθε κόμβου (1ο παιδί, 2ο...). Συνήθως με τα αδέλφια αριστερά προς τα δεξιά.

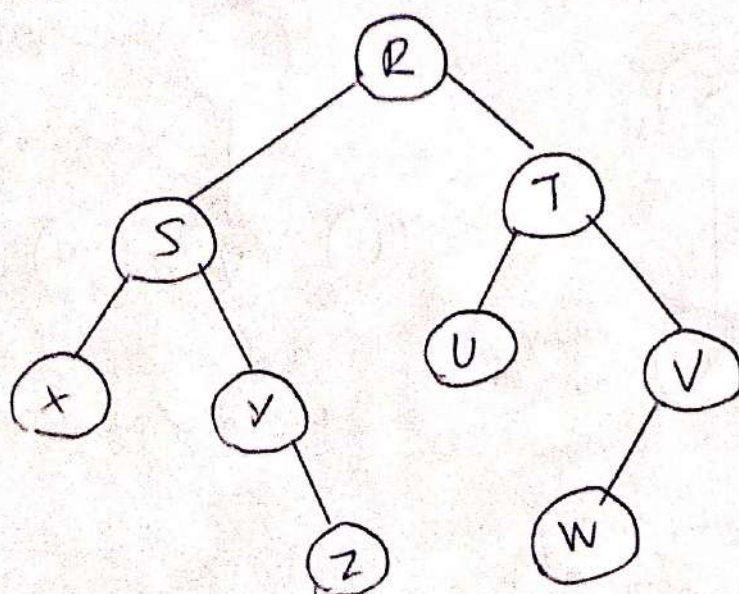
2 Δυαδικά Δένδρα - Binary trees

Ένα δυαδικό δένδρο είναι ένα δένδρο σε σχήμα.

♥ κάθε κόμβος έχει το πολύ 2 παιδιά

♥ κάθε παιδί χαρακτηρίζεται ως είτε αριστερό παιδί είτε δεξί παιδί

♥ Το αριστερό παιδί προηγείται του δεξιού στη διάταξη των παιδιών

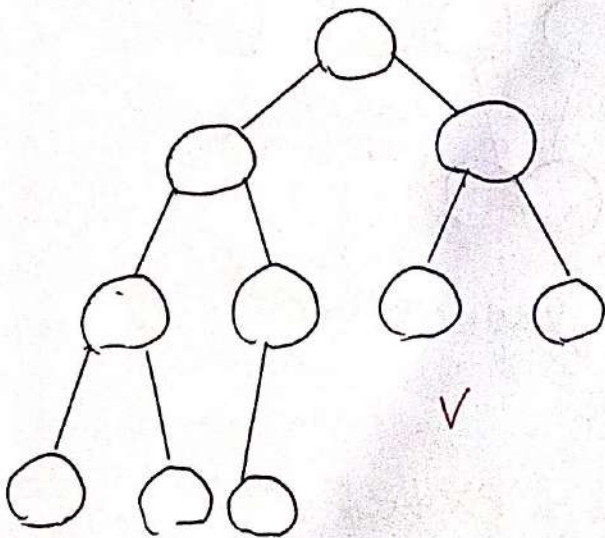


- 8 -

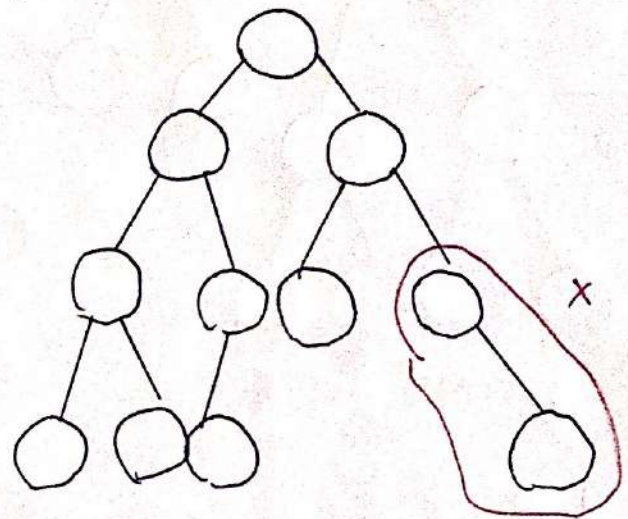
2 1. Πλήρη δυαδικά δένδρα: Complete Binary Trees

Σε ένα πλήρες δυαδικό δένδρο:

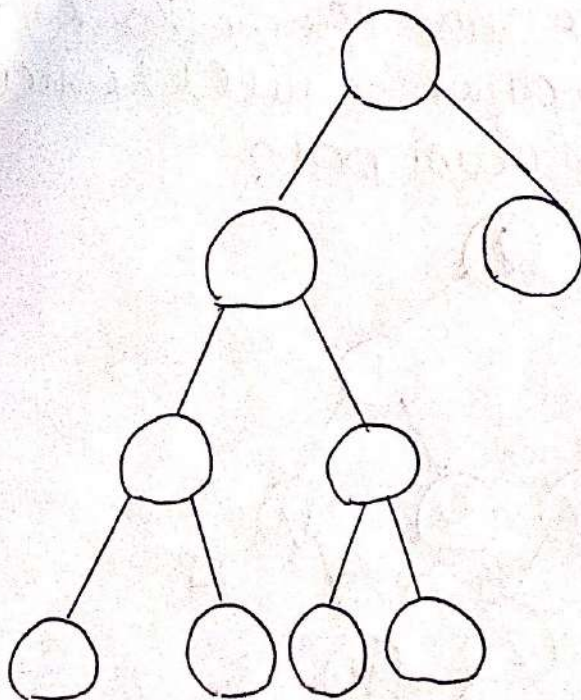
- ♥ Τα φύλλα βρίσκονται είτε σε ένα μόνο επίπεδο είτε σε δύο διαδοχικά επίπεδα
- ♥ Τα φύλλα στο χαμηλότερο επίπεδο τοποθετούνται όσο πιο αριστερά γίνεται.
- ♥ Όλα τα επίπεδα εκτός του τελευταίου είναι συμπληρωμένα με κόμβους



complete Binary tree



NOT complete binary tree



$$Υψος = \lfloor \log n \rfloor$$

$$h = \text{κόμβοι}$$

ελάχιστος αριθμός

$$\text{κόμβων} = 2^h$$

μέγιστος αριθμός κόμβων

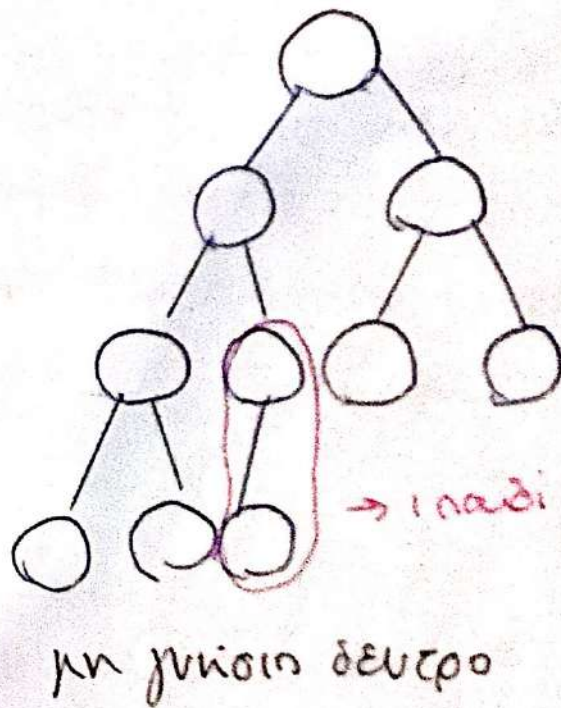
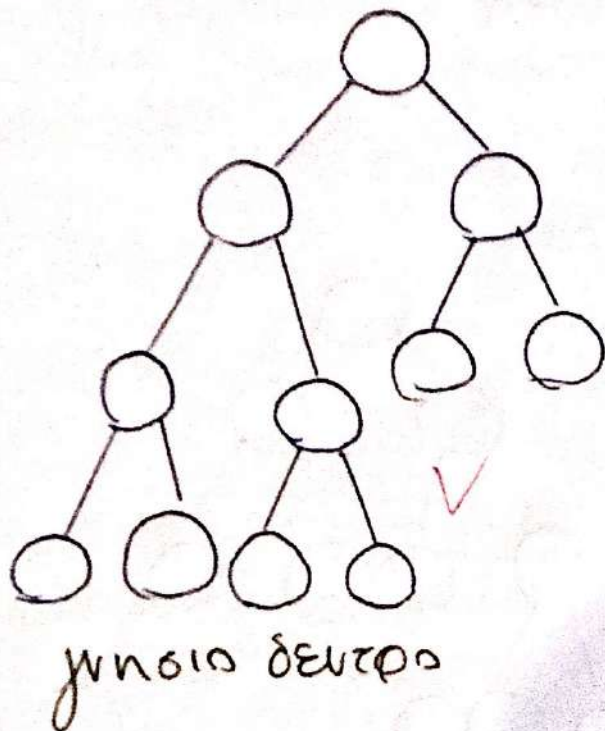
$$2^{h+1} - 1$$

NOT complete
Binary trees

2.2 Γνήσια Δυαδικά δένδρα - proper/full

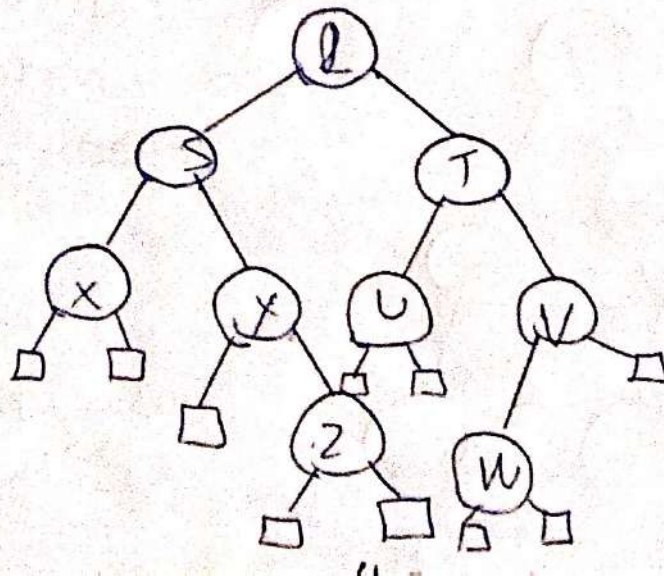
Αν κάθε κόμβος έχει είτε 0 είτε 2 παιδιά

♥ κάθε εσωτερικός κόμβος έχει ακριβώς 2 παιδιά



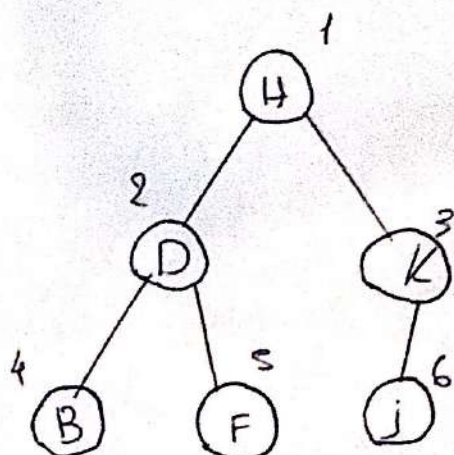
2.3 Στεκταμένα δένδρα - extended

Αν θέλουμε ένα μη κενό γνήσιο δυαδικό δένδρο τότε αναπαριστούμε κάθε εσωτερικό κόμβο με παιδιά σε σχήμα τετραγώνου (null) άρα και αν έχει αριστερό ή δεξί παιδί μόνο

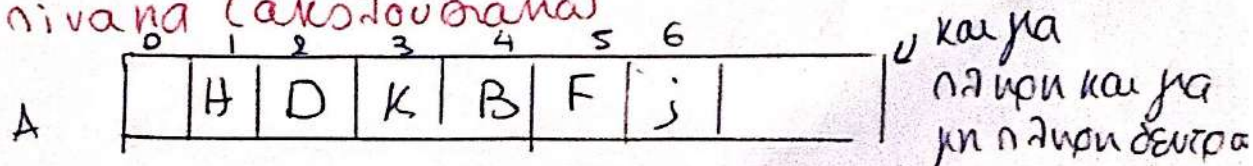


2.4 Πώς μπορώ να αναπαραστήσω ένα δυαδικό δένδρο

α. Με κόμβους



β. Με πίνακα (ακολουθία)



Αριστερό παιδί $A[i] \rightarrow A[2i]$

Δεξί παιδί $A[i] \rightarrow A[2i+1]$

Γονέας $A[i] \rightarrow A[i/2]$

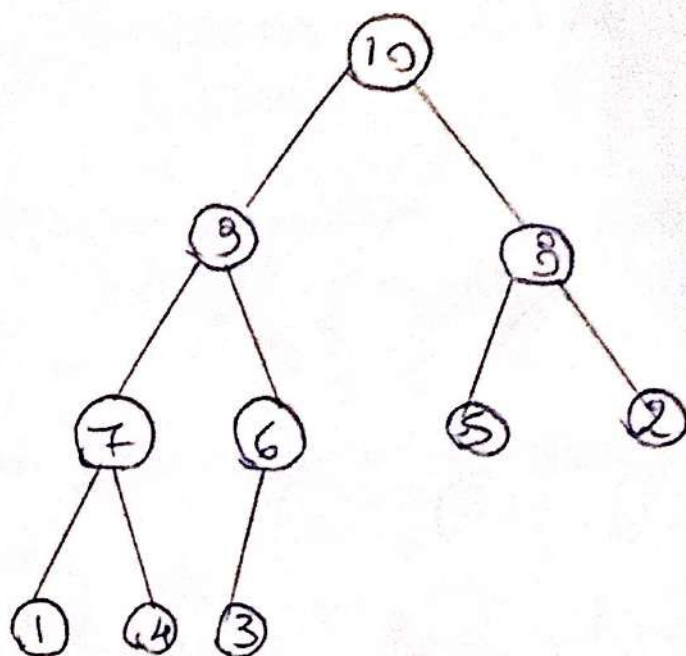
Ρίζα $\rightarrow A[1]$

το $A[i]$ είναι φύλλο $\rightarrow$ ισχύει αλ

3 Σώρος (Heaps)

Ένα πλήρες δυαδικό δένδρο με τιμές αποθηκευμένες στους κόμβους του έτσι ώστε κανένα παιδί να μην έχει τιμή μεγαλύτερη ή ίση του γονέα του (Max heap)

Αν τώρα η τιμή του γονέα είναι μικρότερη ή ίση από το παιδί του και αυτό ισχύει για όλους τους κόμβους, τότε έχουμε σωρό ελαττωτών (min-heap)



max heap

3.1 Σωροί - Ουρά προτεραιότητας

Ένας σωρός παρέχει μια αναδιάταξη για μια ουρά προτεραιότητας.

Το στοιχείο με την μεγαλύτερη προτεραιότητα βρίσκεται στην ρίζα του δέντρου.

Αν αφαιρέσουμε την τιμή από την ρίζα, πρέπει να αναδιαρθρώσουμε το δέντρο ώστε να είναι και πάλι Σωρός.

=> 1. Ανακατάσταση ιδιότητας σωρού σε δέντρο

=> 2. Σειράβαξη πάνω στο δυαδικό δέντρο (Heapify)

♥ Ξεκινάμε από τους εσωτερικούς κόμβους. (Αν ένας κόμβος γενικά δεν έχει παιδιά τότε δεν κάνουμε τίποτα).

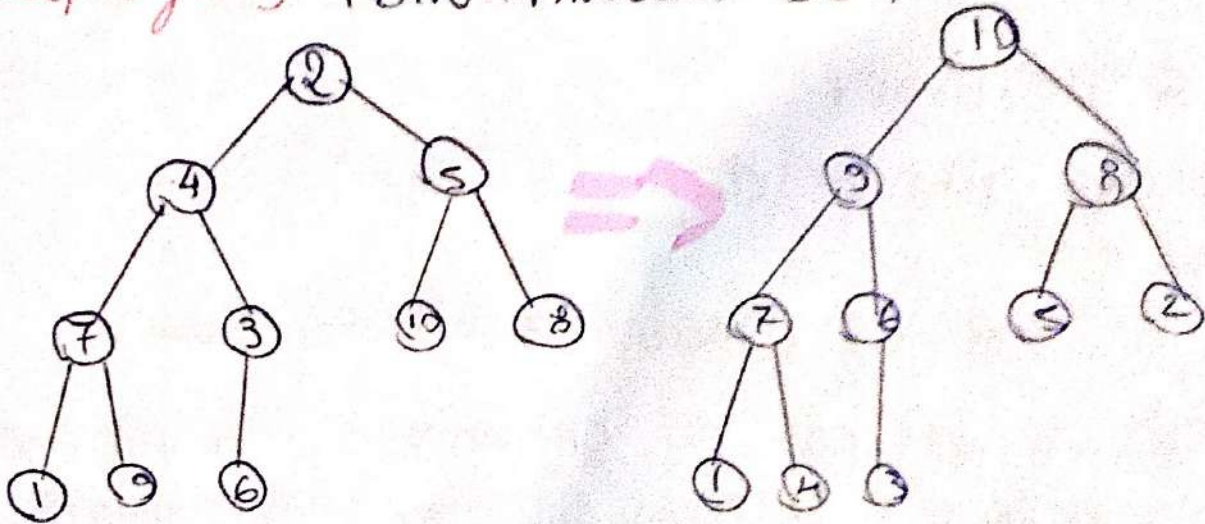
♥ Αν ο κόμβος έχει παιδιά επιλέγουμε το παιδί με την μεγαλύτερη τιμή και τους αλλάζουμε θέσεις, αν η τιμή του παιδιού είναι μεγαλύτερη από την τιμή του κόμβου διαλλάβανουμε την διαδικασία πάνω στο παιδί.

=> 3. Εισαγωγή αριθμού σε σωρό: $O(\log n)$

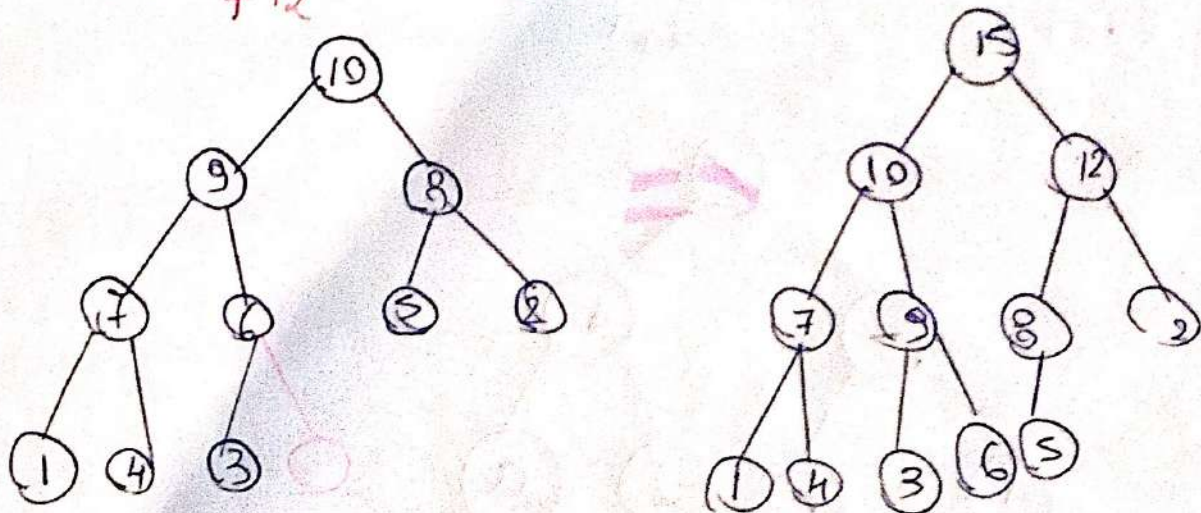
♥ Βάζουμε έναν κενό κόμβο ως φύλλο στην πρώτη διαθέσιμη θέση από τα αριστερά κάτω κάτω

♥ Εφαρμόζουμε τον αλγόριθμο Heapifying για να διατηρηθεί η ιδιότητα του σωρού

Heapifying πολυπλοκότητα $O(n)$



Εισαγωγή 15 (Εισαγωγή φύλλου + Heapifying)
+ 12



πολυπλοκότητα εισαγωγής στοιχείου σε σωρό: $O(\log n)$

=> 4. Διαγραφή από σωρό (highest priority)
χρόνιο πολυπλοκότητα: $O(\log n)$

=> 5. ΠΟΛΥΠΛΟΚΟΤΗΤΕΣ

Ανάλυση / Οργάνωση	Σωρό	Ταξινότση	Αλγόριθμος / Οργάνωση
push / pop vwvw PQ	$O(n)$	$O(n^2)$	$O(1)$
remove highest priority item	$O(\log n)$	$O(1)$	$O(n)$
insert	$O(\log n)$	$O(n)$	$O(1)$

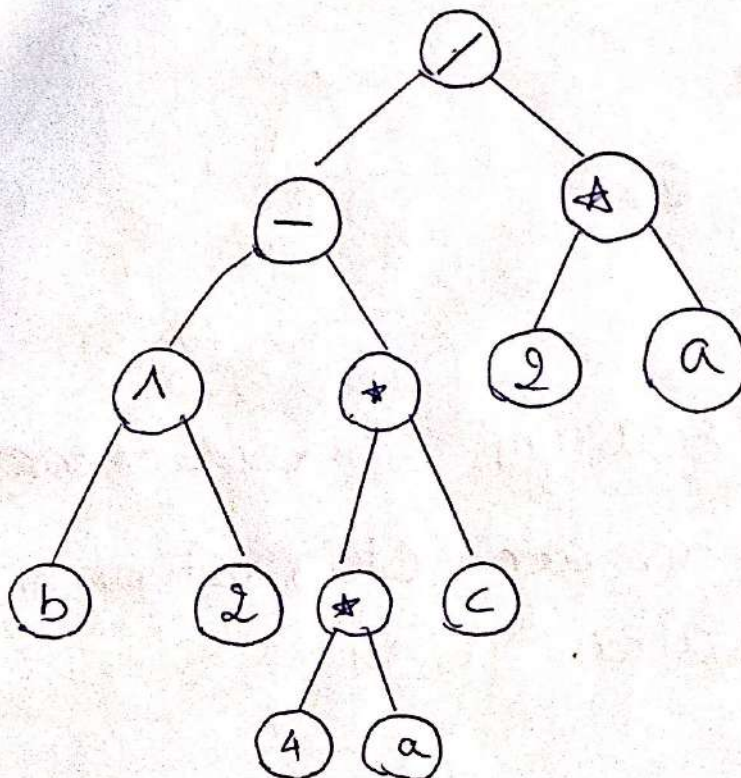
=> 6. Heapsort

χρονική πολυπλοκότητα $O(\log n)$.

4 Δέντρα Εκφράσεων (expression trees)

Δυαδικά δέντρα που χρησιμοποιούνται για να αναπαραστήσουν αλγεβρικές εκφράσεις που σχηματίζονται με δυαδικούς τελεστές

$$(b^2 - 4 * a + -c) / (2 * a)$$

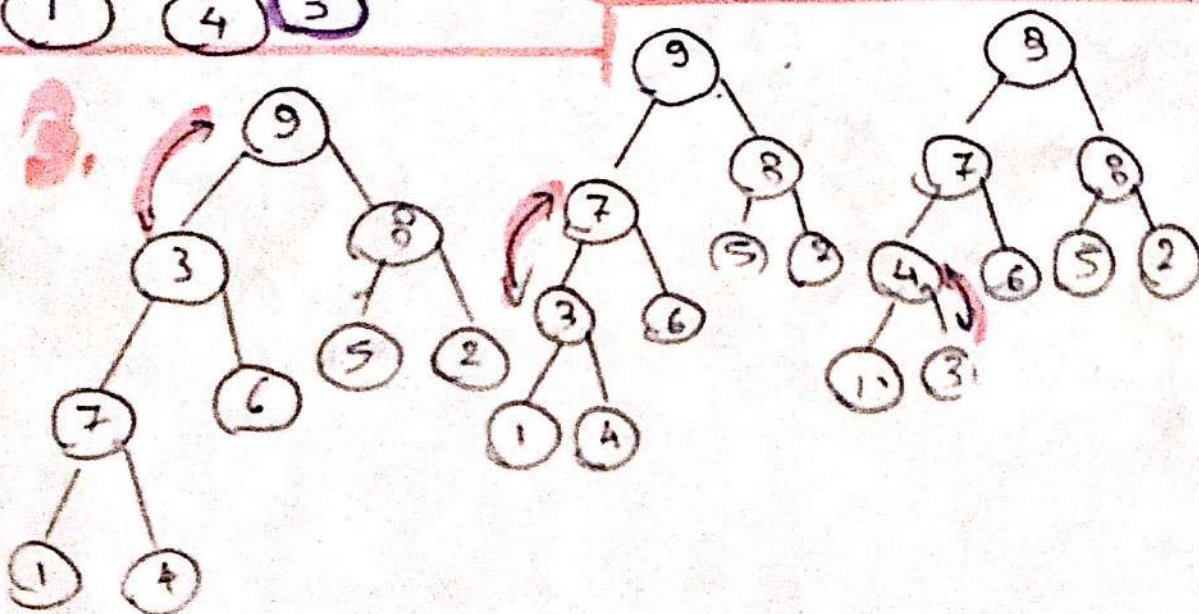
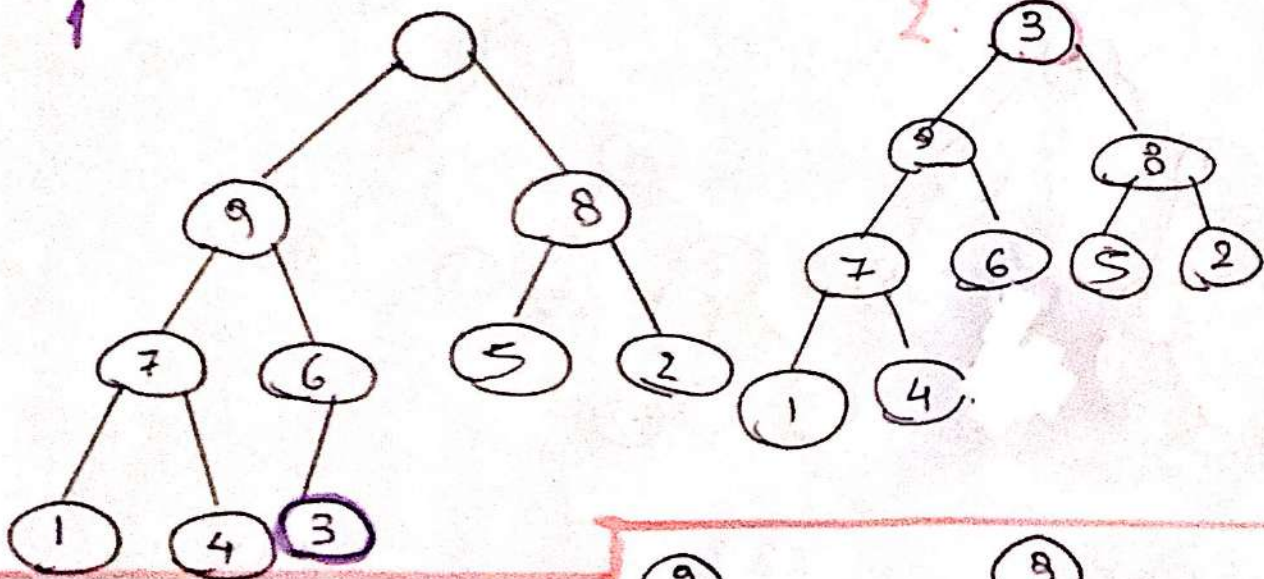


1. Αναδιάρθρωση του δέντρου.

1. → Διαγράφουμε το δεξιότερο κόμβο στη χαμηλότερη γραμμή (δηλαδή το τελευταίο φύλλο στη διατάξη του ευρέους)

2. → Ενοθετούμε την αμμ του διαγραφμένου κόμβου ως κόμβο της ρίζας.

3. → Αποκαθιστούμε την ιδιότητα του σωρού σε δέντρο ξεκινώντας από τον κόμβο της ρίζας και επαναλαμβάνοντας την ανταλλαγή της τιμής του με την μεγαλύτερη τιμή των παιδιών του μέχρι να μην είναι δυνατές άλλες αλλαγές



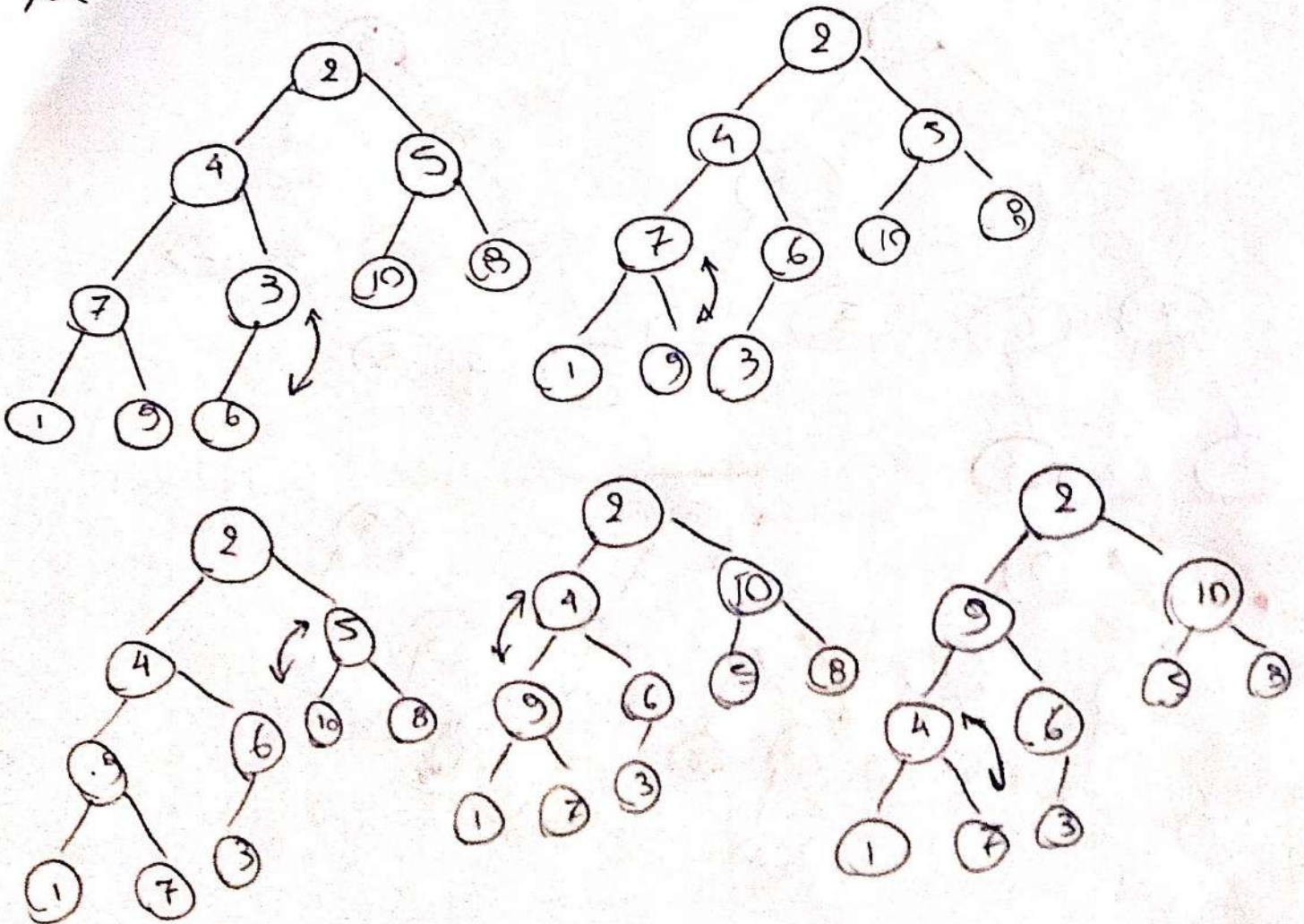
2 Heapifying μετατροπή ενός ημίσφους
διαδικού δένδρου σε σωρα (C++)

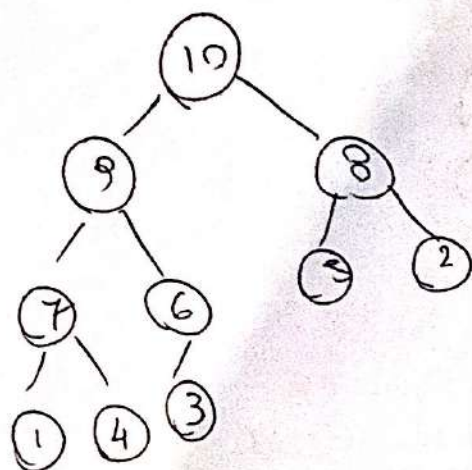
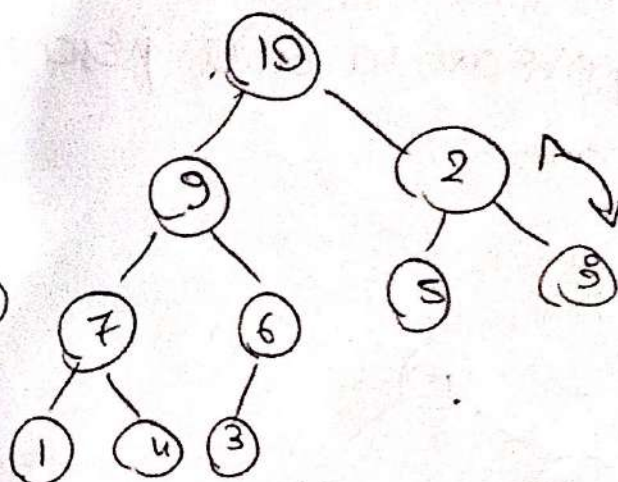
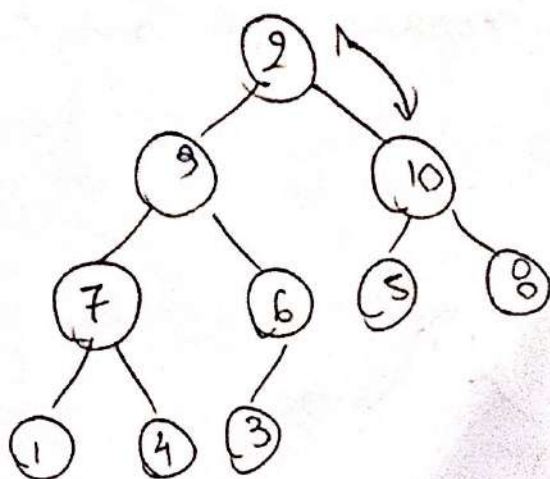
→ Έστω ένας κορβός N με τιμή V . Αν ο M δεν έχει παιδιά, τότε δεν κινούμε τίποτα M

→ Αν ο Ν έχει παιδιά τότε επιλέγουμε το παιδί με την μεγαλύτερη τιμή V_i

→ Αν $V_1 \supset V$ τότε αυτά λίσουμε ως ειγες V_1 και V , αλλιως (αν $V \subseteq V_1$) δεν κλωουμε τιποτα.

→ Επαναλαμβάνουμε την διαδικασία για τον κόμβο μ.





3. Εισαγωγή νέου στοιχείου σε σωρό Ο(λογn)

1. Προσθέτουμε έναν νέο κενό κόμβο ψ στο στην πρώτη διαθέσιμη θέση στα κάτω επίπεδα του δέντρου

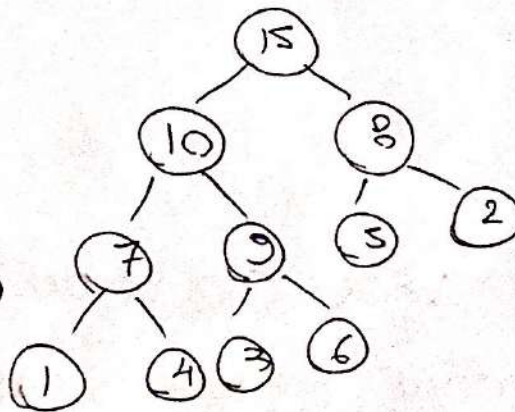
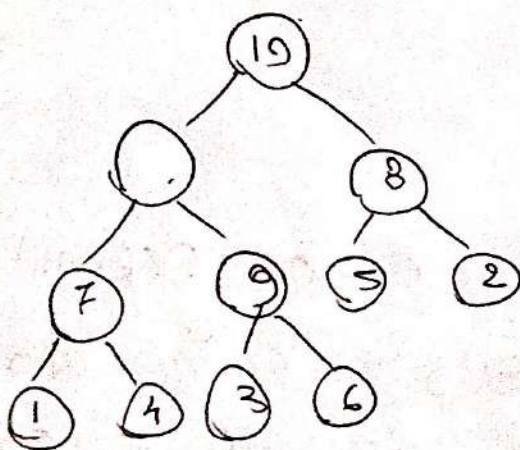
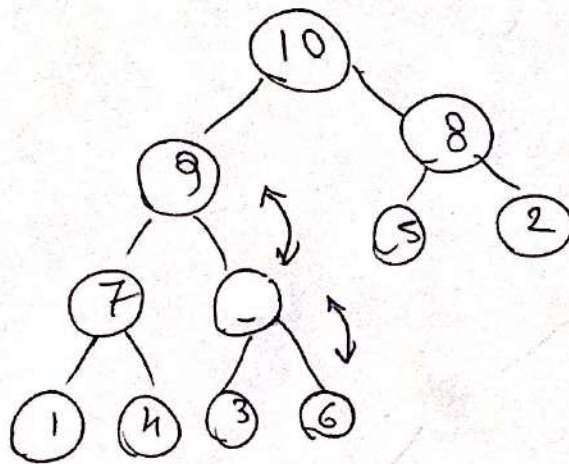
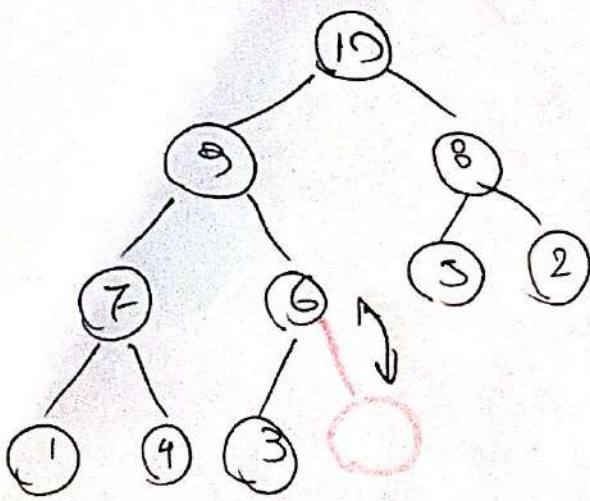
2. κάνουμε heapifying ξεκινώντας από τον πρώτο κόμβο αυτού του νέου κόμβου που προσπαθούμε να έχουμε τη σωστή θέση το νέο στοιχείο.

Συνοψίζοντας:

Η διαδικασία εισαγωγής σε σωρό περιλαμβάνει τη τοποθέτηση του νέου στοιχείου σε κάποιο επίπεδο του δέντρου και την επαναδοσίτασή του, προκειμένου

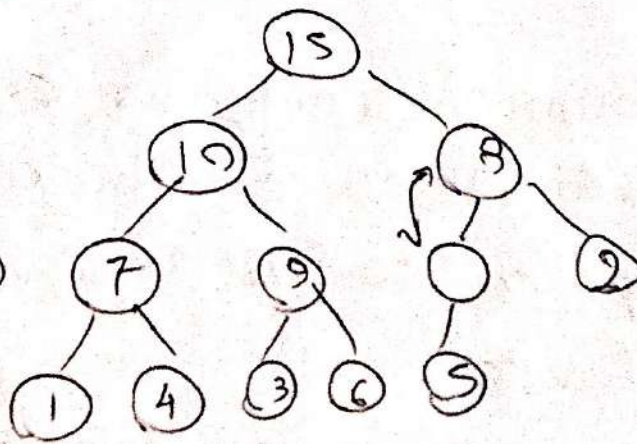
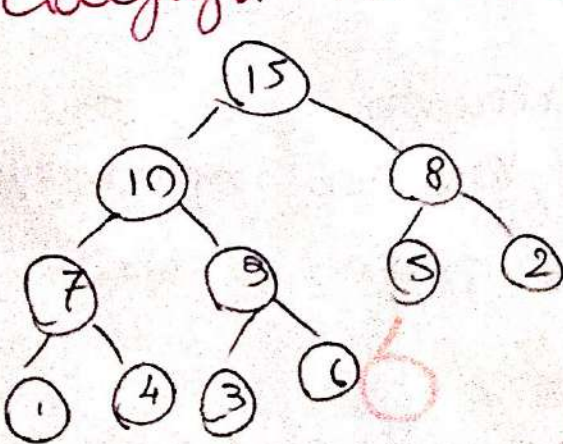
να διατηρηθεί η ιδιότητα του σωρού, δηλαδή κάθε γονέας να είναι μεγαλύτερος ή ίσος από τα παιδιά του.

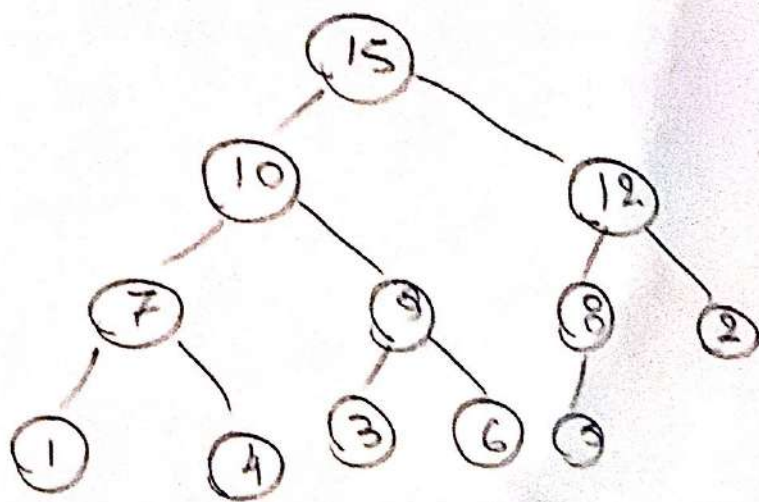
Εισαγωγή 15



κελίμος
ζυγός

Εισαγωγή 12 στον κελίμο σωρό





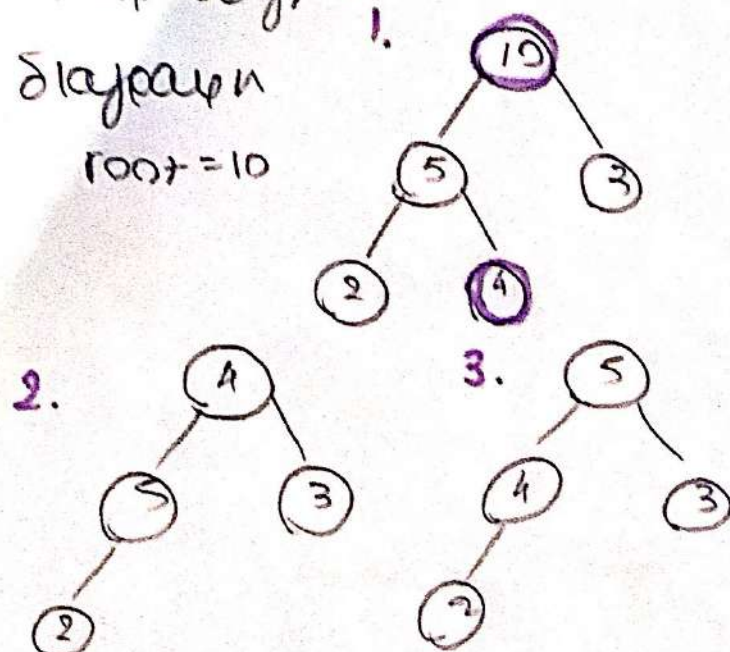
8
 ⇒ Οι ουρές προετοιμάζονται με το $O(N)$ να υλοποιηθούν με ένα max heap

4 Διαγραφή από σωρό (delete)

Η τυπική λειτουργία διαγραφής σε ένα σωρό είναι η διαγραφή του στοιχείου που βρίσκεται στη ρίζα του σωρού. Δηλαδή, αν πρόκειται για Max Heap, η διαγραφή θα αφαιρέσει το μέγιστο στοιχείο, ενώ αν είναι min heap

παραδειγμα:
 διαγραφή

root = 10



Διαδικασία

→ Αντικαθιστούμε το στοιχείο που θέλουμε να διαγράψουμε, είτε είναι ρίζα είτε είναι άλλο στοιχείο, με το τελευταίο στοιχείο του σωρού

→ Διαγράφουμε το τελευταίο στοιχείο από τον σωρό

→ κάνουμε heapify για να αποκατασταθεί η ιδιότητα του σωρού αν χρειάζεται

5 Διασχίσεις δέντρων (Traversals)

Inorder: Αριστερό παιδί, ρίζα, δεξί παιδί

$b^2 - 4 + a * c / 2 * a$ (infix) αναδρομικά ή με

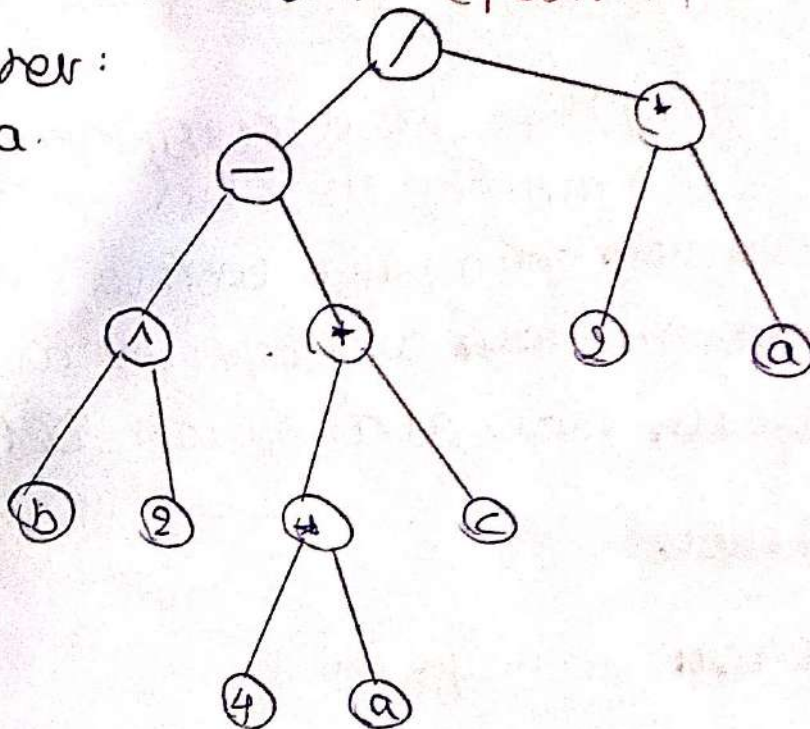
preorder: ρίζα, αριστερό, δεξί παιδί stack

$/ - \wedge b^2 * 4 a c * 2 a$ (prefix)

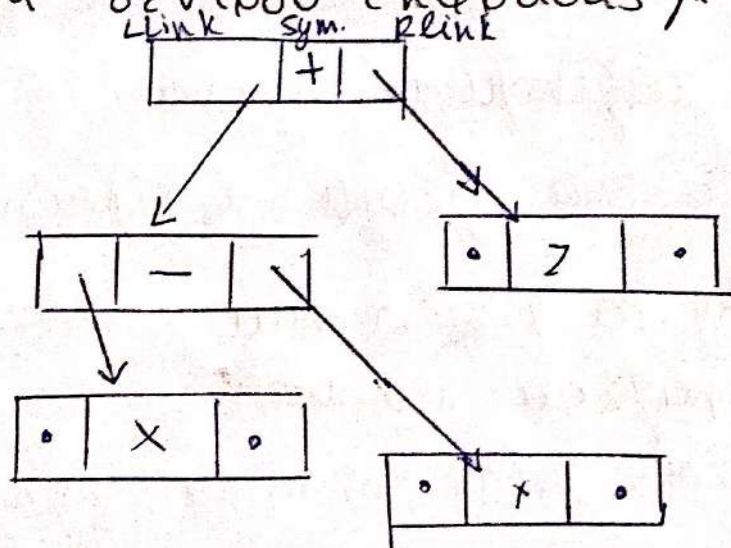
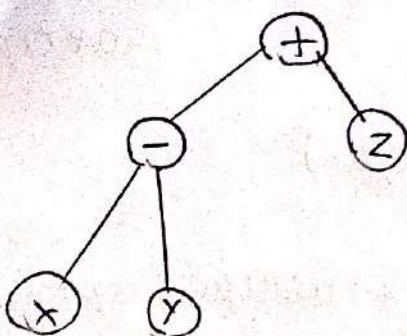
postorder: αριστερό, δεξί, ρίζα (Δυσκολή).

$b^2 \wedge 4 a * c * - 2 a * /$ (postfix)

levelorder:
με ουρά.



6 Αναπαράσταση δέντρου έκφρασης με λίστες



Ξ Δέντρα αναζήτησης (Parse trees)

Οι μεταγλωττιστές αναλύουν αλγεβρικές εκφράσεις (και γενικά δηλώσεις προγραμμάτων) και κατασκευάζουν δέντρα συντακτικής ανάλυσης.

Αυτά διασφαλίζονται από γεννημένες κωδίκια για να παραχθεί ο κωδικός συναρμολόγησης.

Θ Πίνακας συμβόλων (symbol table)

Μια δομή δεδομένων με αντικείμενα που έχουν κλειδιά και υποστηρίζει τις εξής λειτουργίες:

✓ Αφαιρετικότητα του πίνακα ώστε να είναι άδειος

✓ Μέτρηση πλήθους αντικειμένων του

✓ Εισαγωγή νέου αντικειμένου ~~OK~~

✓ Αναζήτηση

✓ Διαγραφή αντικειμένου.

✓ Εύρεση k-οστού μικρότερου αντικειμένου ~~OK~~

✓ Ταξινόμηση πίνακα ~~πολλά πράγματα~~

Αντικείμενα / ζεύγη: (κλειδί, τιμή)

! Αν τα κλειδιά είναι μοναδικά ο πίνακας συμβόλων ονομάζεται και map.

Αν δεν είναι μοναδικά ονομάζεται και lexikon

(dictionary)

ελληνική symbol to table: φοιτητές πανεπιστημίου

9 Αναζήτηση (search)

Η αναζήτηση μιας συγκεκριμένης πληροφορίας από μεγάλους όγκους ήδη αποθηκευμένων δεδομένων.

1. Συστήματα πληροφοριών τραπεζών
2. Συστήματα πληροφοριών αεροπορικών εταιριών
3. Αναζήτηση στο web.

9.1 Αναζήτηση με δείκτες (key-indexed search)

Εισαγωγή: $O(1)$

Αναζήτηση: $O(1)$

Ευρεση: $O(n) \rightarrow n: \max \alpha_{km}$

9.2 Σειριακή αναζήτηση (sequential search)

Εισαγωγή: $O(n)$

Αναζήτηση: $O(n)$

Ευρεση: $O(1)$

n : αριθμός αιτιημάτων
ταξινομημένος πίνακας με βάση το κλειδί

Εισαγωγή: $O(n)$

Αναζήτηση: $O(n)$

Ευρεση: $O(1)$

ταξινομημένα συνδεδεμένα λίστα

9.3 Δυαδική Αναζήτηση

Βασίζεται στο πρώτο divide and conquer.

Γίνεται με:

1. Ελακτιστική
2. Αναδρομικά

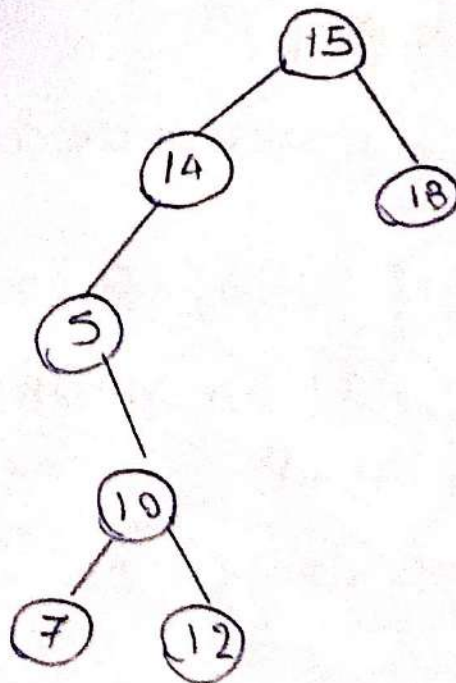
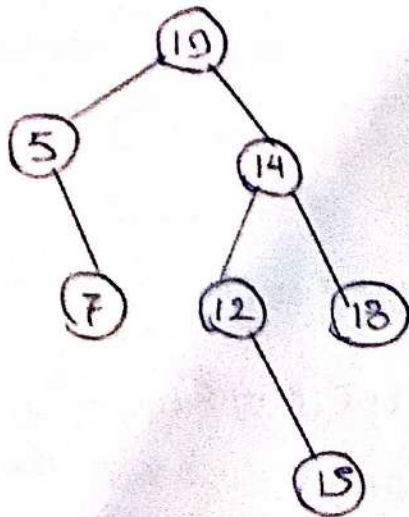
-11-

Παθητικότητα: οσλοση)

10 Δυαδικά δένδρα αναζήτησης BST

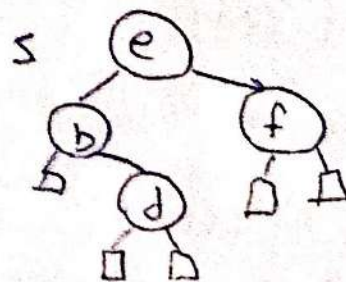
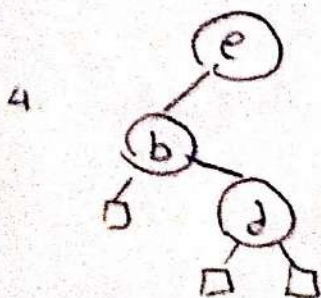
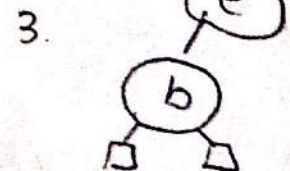
Δυαδικά δένδρα στα οποία για κάθε κόμβο N ισχύει ότι τα κλειδιά στο αριστερό υποδένδρο του N είναι μικρότερα από το κλειδί του N και τα κλειδιά στο δεξιο υποδένδρο είναι μεγαλύτερα ή ίσα από το κλειδί του κόμβου N (BST property)

↳ μπορούμε να έχουμε double keys (distinct values)



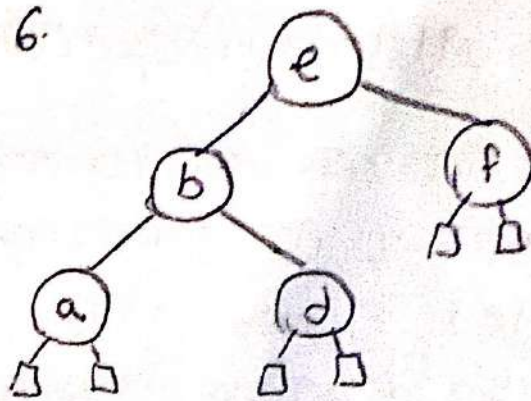
10.1 Εισαγωγή σε BST (αναδρομικά)

1. □ κενό δένδρο

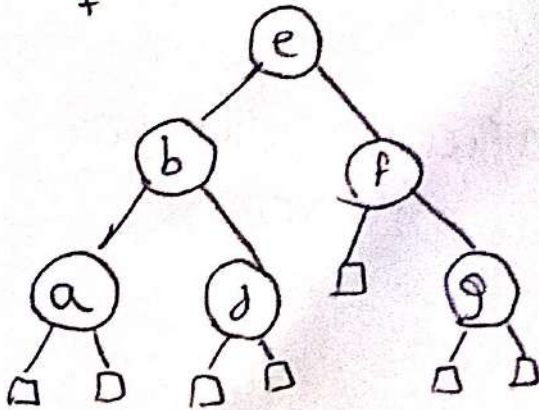


σειρά εισαγωγής:
e b d f a g c

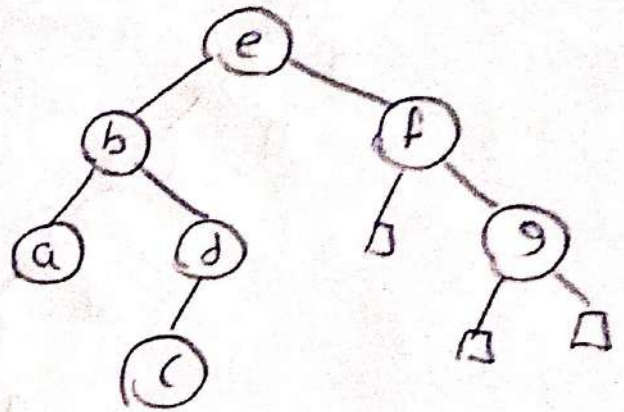
6.



7.



8.

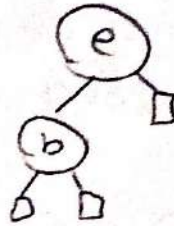


! Αν έχουμε double keys

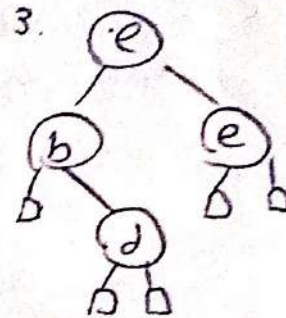
1.



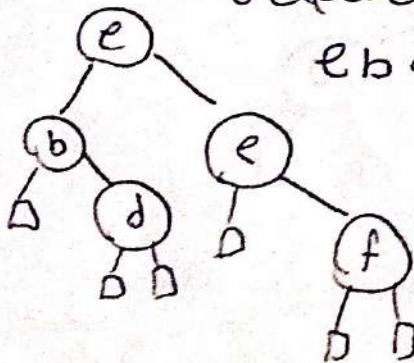
2.



3.

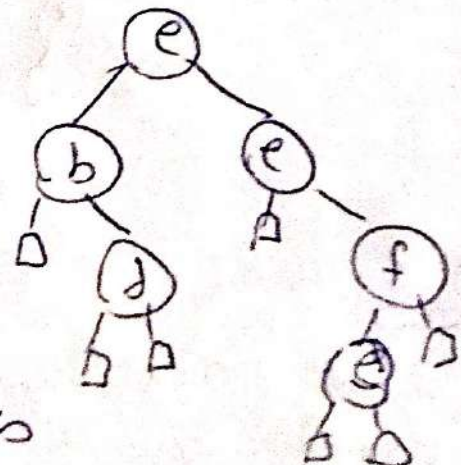


4.



σειρά εισαγωγής:
e b d e f e s.

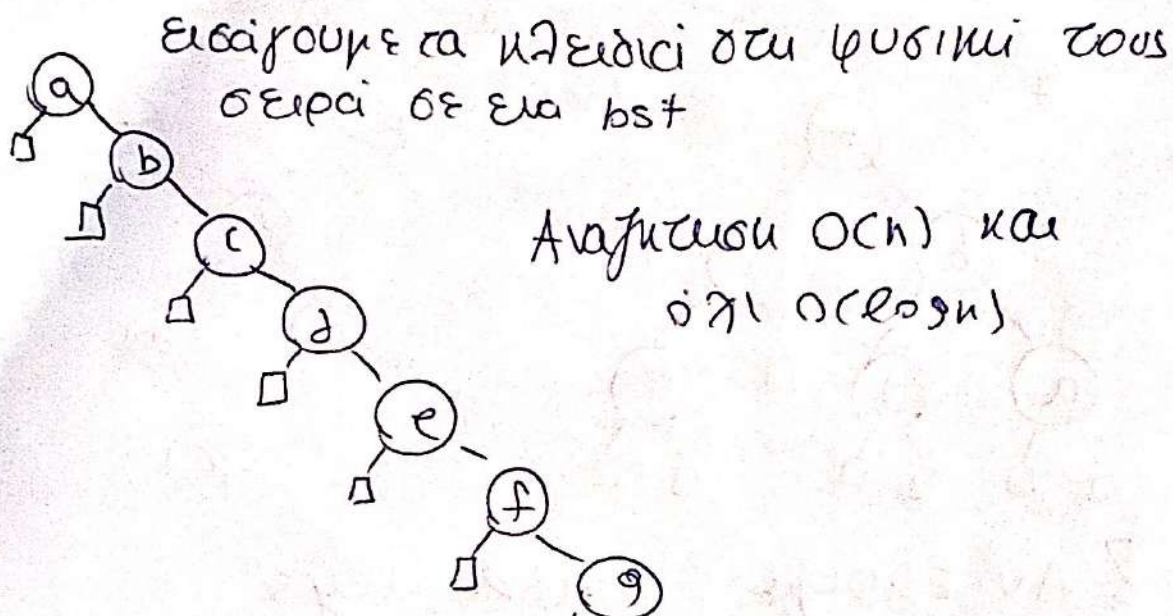
5.



Η αναζήτηση θα επιστρέψει το στοιχείο που αντιστοιχεί στο πρώτο διάνυσμα κλειδιού σε αυτό το μοναδικό.

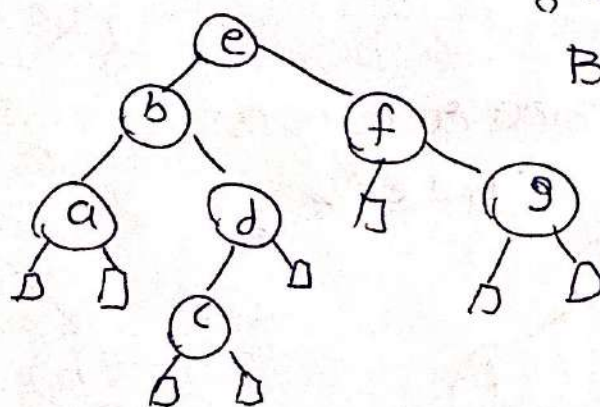
10.2 Πότε έχουμε αναρροαζευκί αναζήτηση

1. Αν θα έχουμε τα κλειδιά σε φυσική τους σειρά :
a, b, c, d, e, f, g, το δέντρο που θα δημιουργηθεί είναι
μα αλυσίδα (worst case). Γιατί τότε δεν πρέπει να



2. Αν τα κλειδιά είναι σε αντιστροφή σειρά ή
αν είναι σχεδόν ταξινομημένα.

! Σε ισορροπημένο
BST search O(log n)



10.3 Insert σε BST

→ Όταν διασχίζουμε ένα BST με inorder οι αλυσ
των κόμβων εμφανίζονται σε αυξανόμενη σειρά.

11. Περιστροφές (Rotations)

♥ Μας επιτρέπουν να ανταλλάξουμε τον ρόλο του κόμβου ρίζας της περιστροφής με έναν από τους κόμβους-παιδιά της ρίζας σε ένα δέντρο, διατηρώντας παράλληλα τη σειρά των κλειδιών στους κόμβους του BST.

♥ Μας βοηθά να κάνουμε τα δέντρα ισορροπημένα

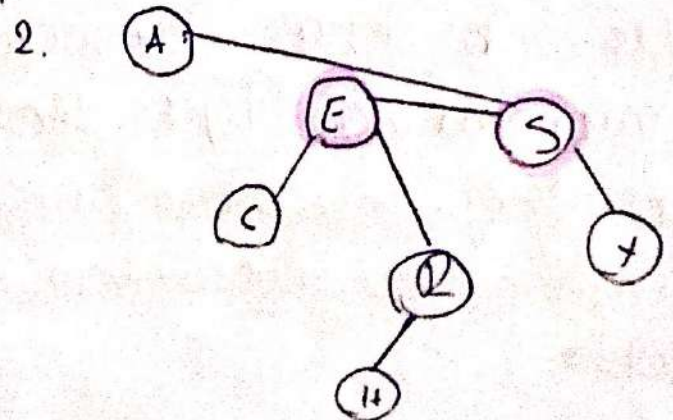
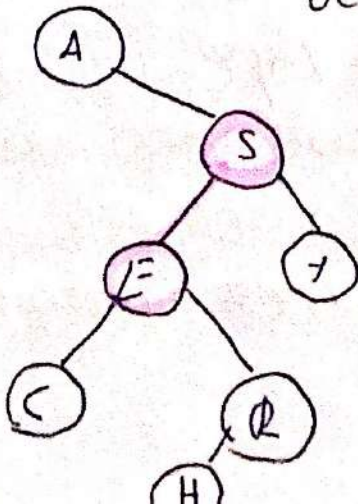
11.1 Δεξιά περιστροφή

♥ Είναι μια κίνηση που γίνεται σε ένα δέντρο για να ανεβάσουμε ένα αριστερό παιδί και να κατεβάσουμε τη ρίζα προς τα δεξιά.

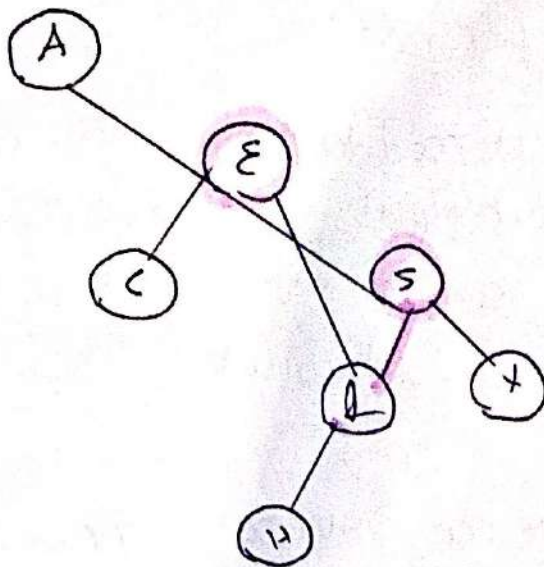
♥ Αυτή η κίνηση βοηθά το δέντρο να ισορροπώσει όταν έχει "γείρει" πολύ προς τα αριστερά

Στην ουσία τι λέμε: Παιρνουμε έναν κόμβο που έχει αριστερό παιδί και του λέμε: Άσε το παιδί σου να πάρει τη θέση σου. Εσύ πήγαινε δεξιά του. Αν το παιδί σου είχε δεξί παιδί, κράτα το αριστερό σου.

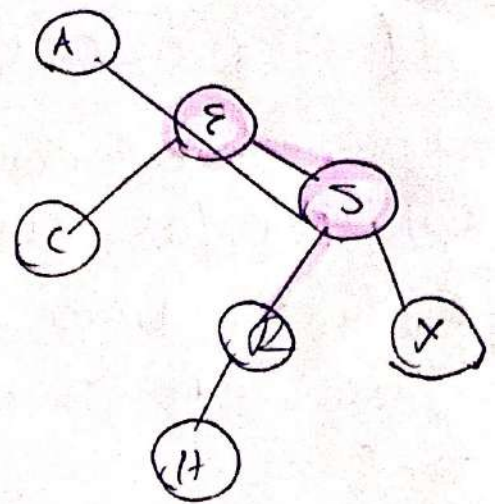
ΠΑΡΑΔΕΙΓΜΑ έστω ότι θέλουμε να εμπεδώσουμε δεξιά περιστροφή στον S.



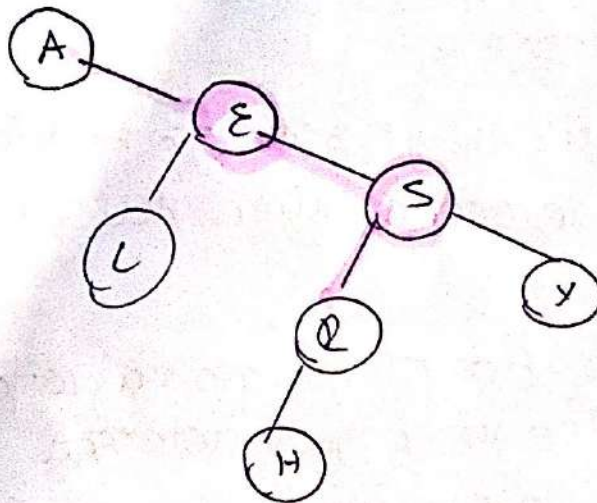
3.



4.



5.



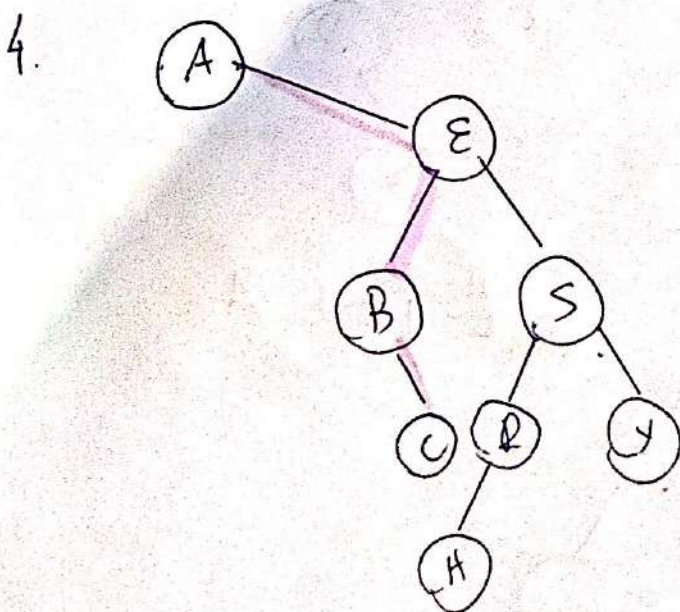
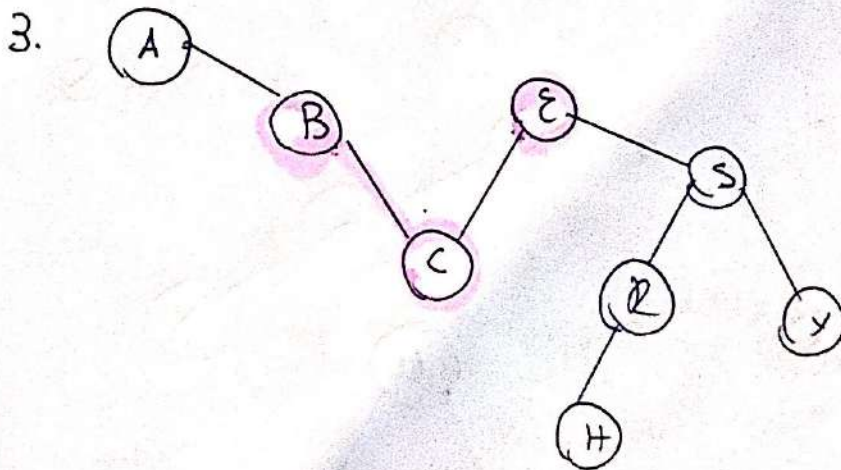
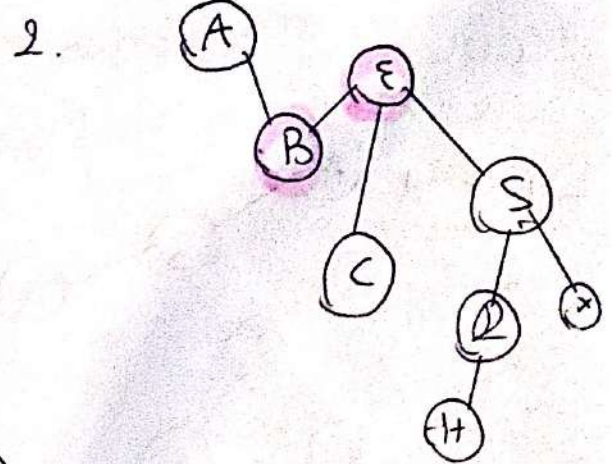
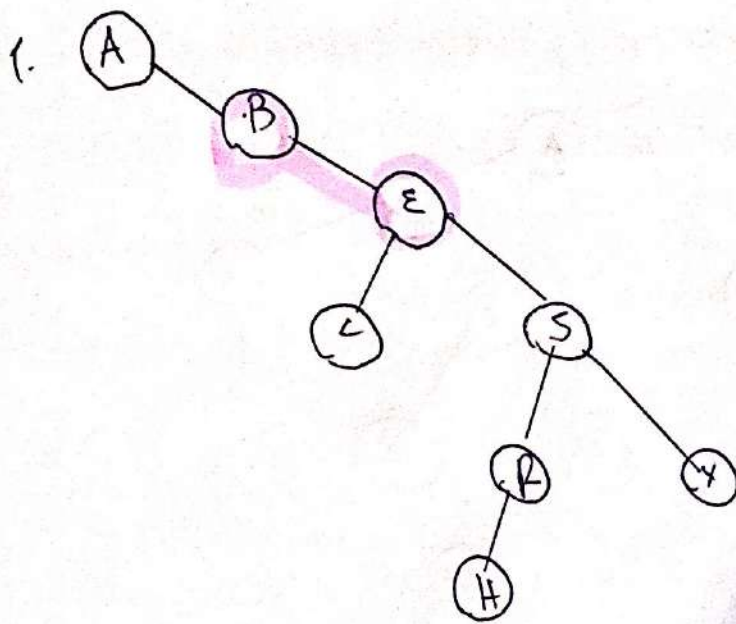
11.2 Αριστερή περιστροφή

♥ Είναι μια κίνηση που γίνεται σε ένα δέντρο όταν έχει γείρει πολύ προς τα δεξιά, για να το ισορροπήσουμε.

Με άλλα λόγια: Παίρνουμε έναν κόμβο που έχει δεξί παιδί και του λέμε: Ασε το παιδί σου να πάρει τη θέση σου Εσύ πήγαινε αριστερά του. Αν το παιδί έχει αριστερό παιδί, τότε κρατά το δεξιά σου.

ΠΑΡΑΔΕΙΓΜΑ

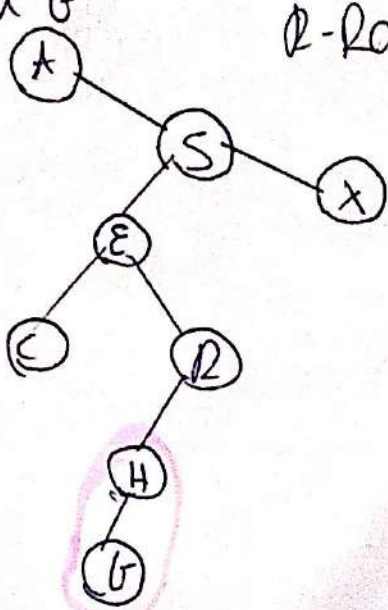
ΠΕΡΙΣΤΡΟΦΗ ΑΡΙΣΤΕΡΑ
ΣΤΟΝ ΚΟΥΒΟ Β



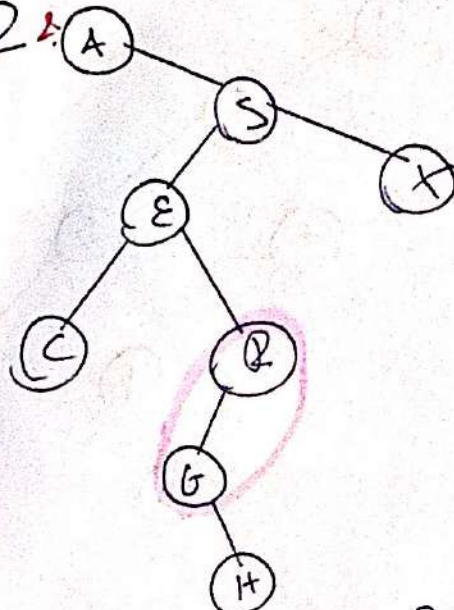
11.3 Εισαγωγή στην AVL

Τα πρόσφατα εισαχθέντα κλειδιά βρίσκονται στα κορυφαία (πρόσφατα σε ορισμένες εφαρμογές).

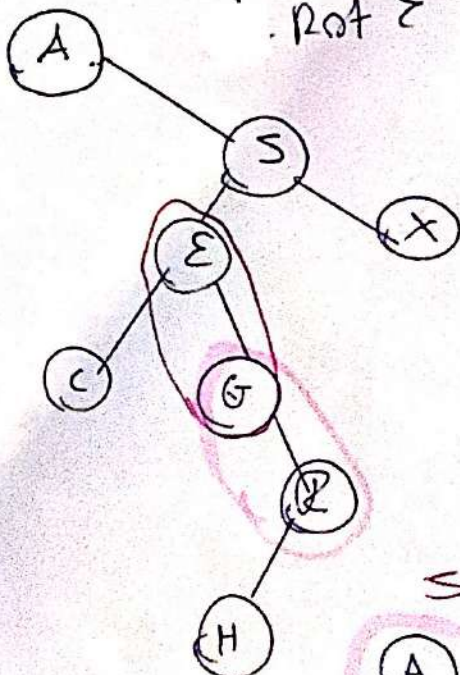
Εισαγωγή G
R-rot H



R-rot D

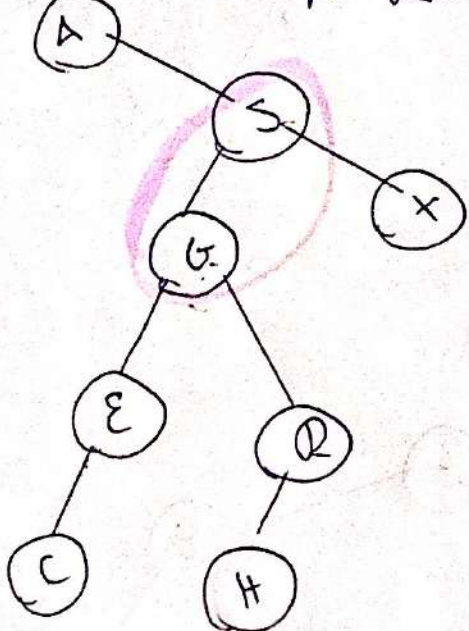


3.



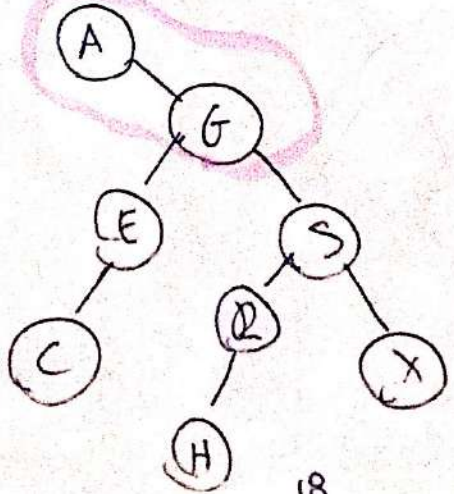
L-rot E

4.

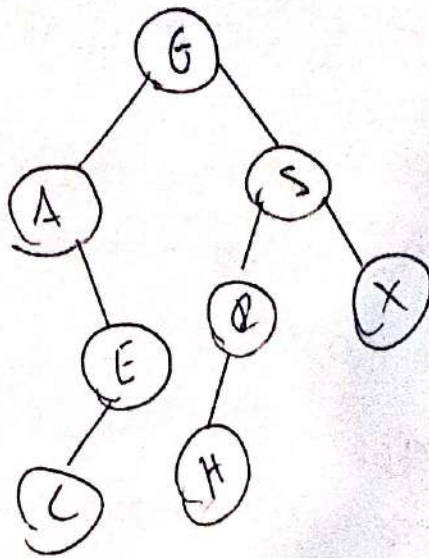


R-rot S

L-rot A



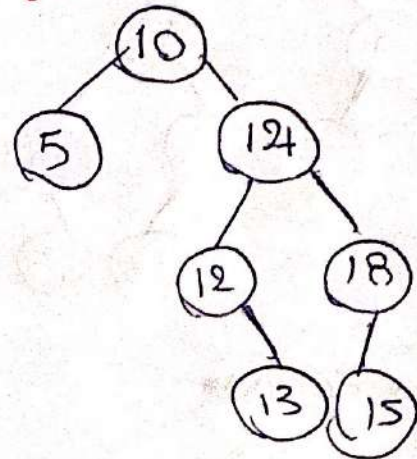
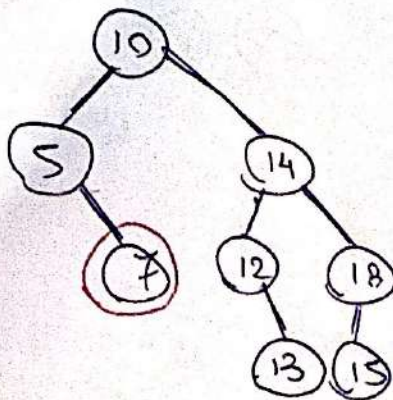
6.



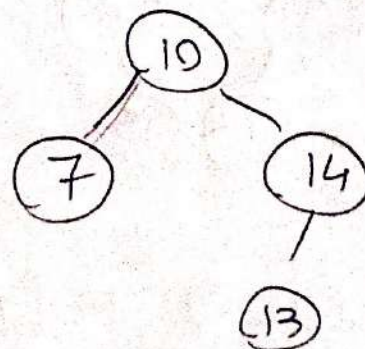
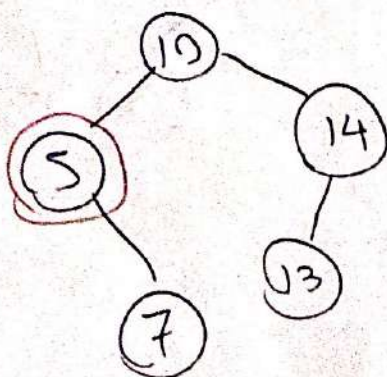
Δεν έχει παιδιά

11.4 Διαγραφή από ένα BST

Περίπτωση 1: Ο κόμβος προς διαγραφή είναι φύλλο
 Διαγραφή 7 *Αντως αλείφουμε τον κόμβο*



Περίπτωση 2: Ο κόμβος έχει μόνο ένα παιδί
 Διαγραφή 5: *Αντως ενώνουμε τον γονέα του 5 με το παιδί του 5 και αλείφουμε τον κόμβο 5.*

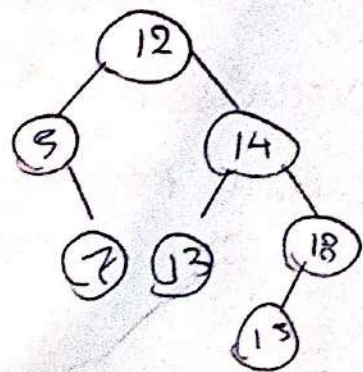
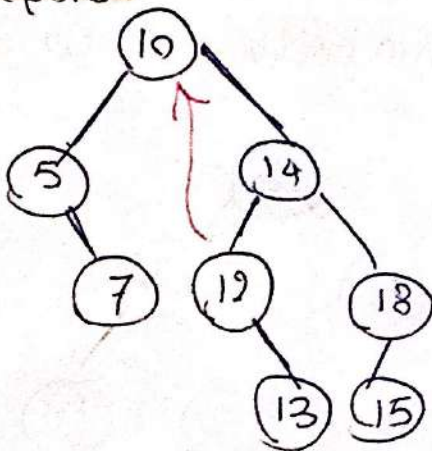


Περίπτωση 3: Ο κόμβος προς διαγραφή έχει 2 παιδιά

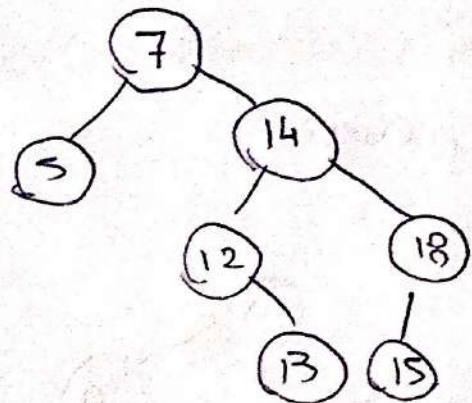
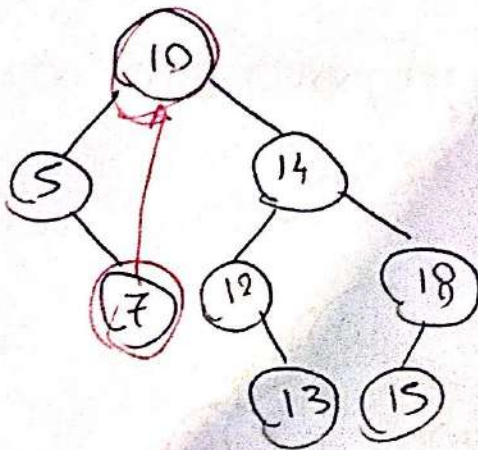
✓Βρίσκουμε το μικρότερο κλειδί από το δεξιο υποδένδρο και αντικαθιστούμε αυτό το κλειδί με τον κόμβο προς διαγραφή, διαγράφοντας τον κόμβο.

✓Εναλλακτικά, θα μπορούσαμε να βρούμε το μεγαλύτερο κλειδί από το αριστερό υποδένδρο.

Διαγραφή 10.
Με 1ο τρόπο:



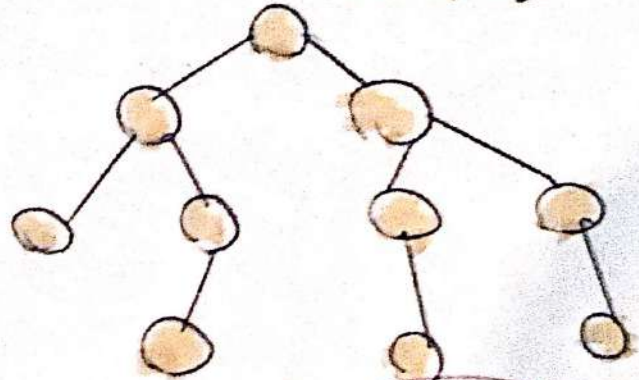
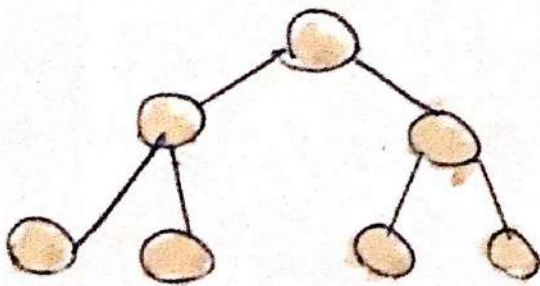
Με 2ο τρόπο:



11.5 Ένωσε 2 BST σε 1
πρωτοδικότητα: $O(n)$

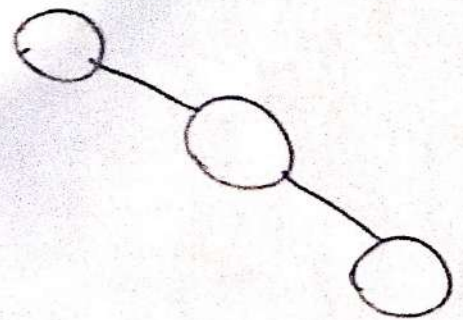
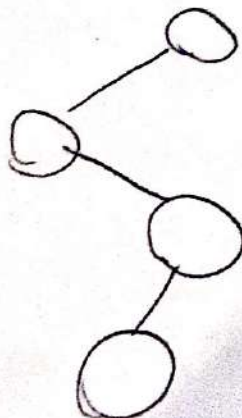
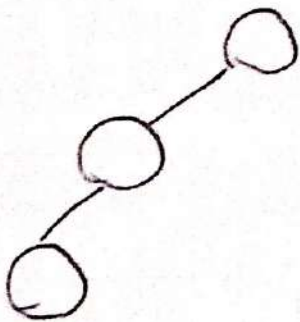
11.6 Αναζήτησης σε BST best case / worst / average case

1. Best case (ισορροπημένο δέντρο)



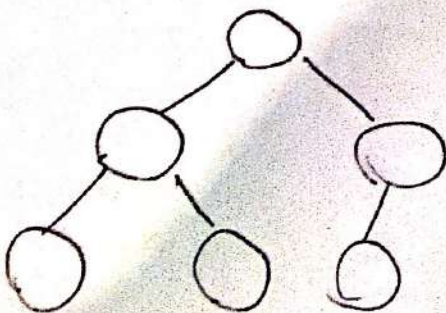
Μεθυστική: $O(\log n)$ αναζήτηση / $O(\log n)$ εισαγωγή
 $O(\log n)$ διαγραφή

2. Worst case (αλυσίδα - στενό δέντρο)



Μεθυστική: $O(n)$ αναζήτηση, διαγραφή, εισαγωγή

3. Average case



Μεθυστική:
αναζήτηση: $O(\log n)$
εισαγωγή: $O(\log n)$
διαγραφή: $O(\log n)$

11.7 Διαμερίσις (partition)

Μια τεχνική με την οποία αναδιατάσσουμε ένα BST, έτσι ώστε το στοιχείο που είναι το k -οστό μικρότερο να γίνει ρίζα του δέντρου.

Γιατί να το κάνουμε αυτό;

Αν θέλουμε να αποκτήσουμε γρήγορη πρόσβαση σε k -οστό στοιχείο. (για λίγους αποδόσεις)

Μπορεί να συνδυαστεί με: splay trees και βοηθά σε υλοποιήσεις όπως order statistics trees.

Μαθηματικότητα (best case)
Worst case).

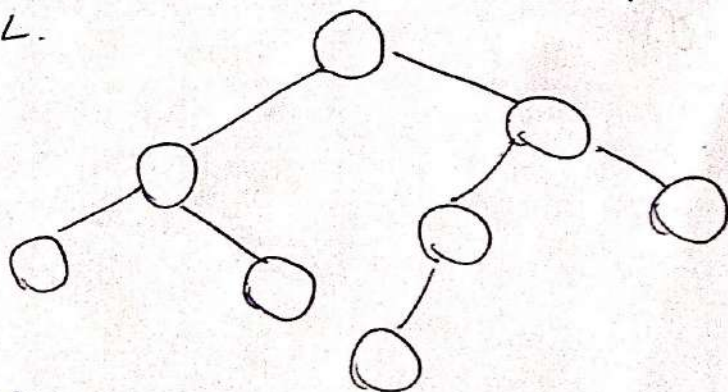
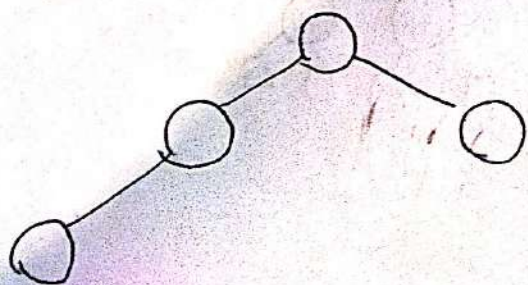
12 Ισορροπημένα δυαδικά δένδρα αναζήτησης

AVL δένδρα

Adelson - Velskii Landis

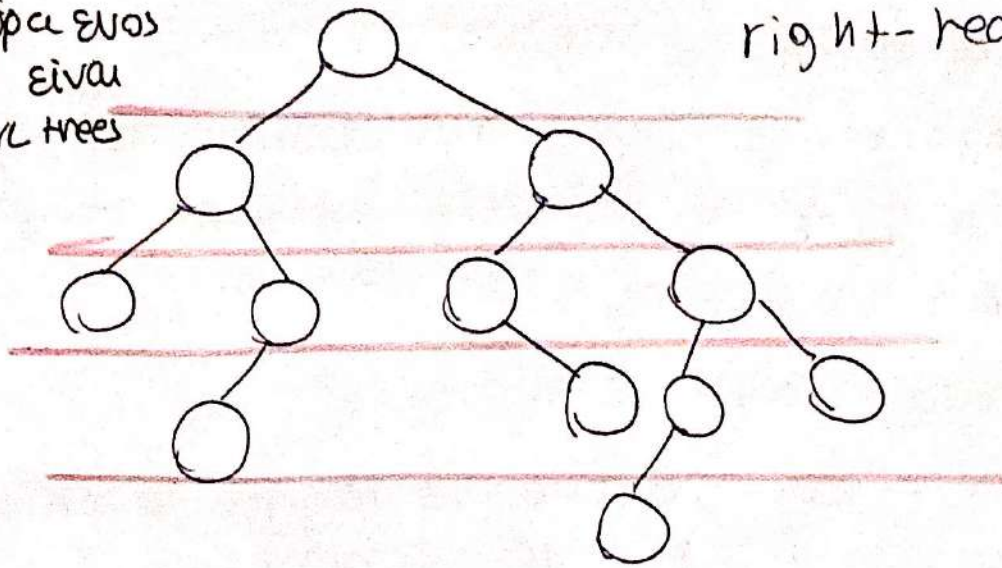
⇒ Αν ο N είναι ένας κόμβος σε ένα δυαδικό δένδρο αναζήτησης, τότε λέμε ότι ο N ικανοποιεί την ιδιότητα AVL αν τα ύψη του αριστερού και του δεξιού υποδένδρου διαφέρουν κατά το πολύ 1. (ιδιότητα ισορροπίας ύψους)

Ένα **AVL** δέντρο είναι ένα BST όπου κάθε κόμβος πληροί την ιδιότητα AVL.

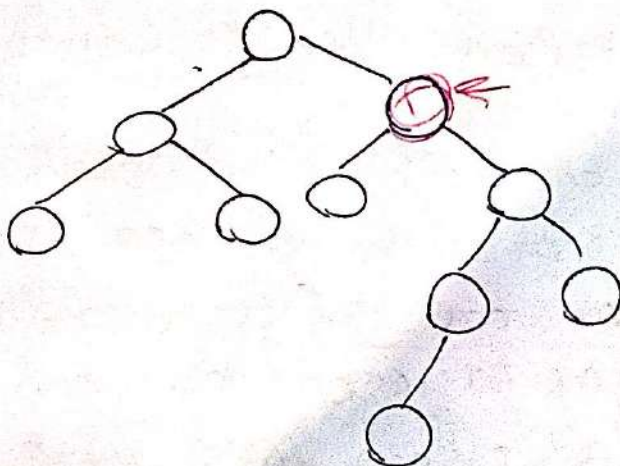
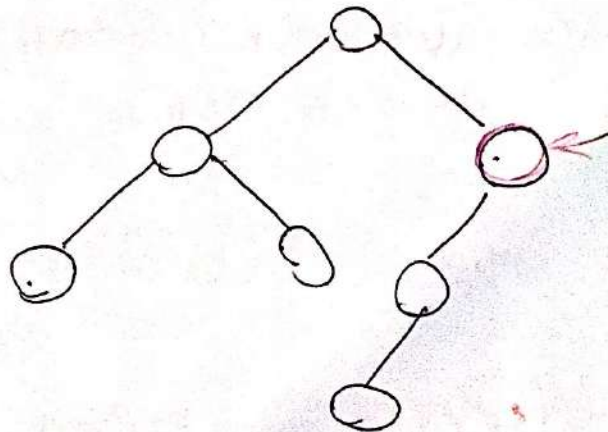
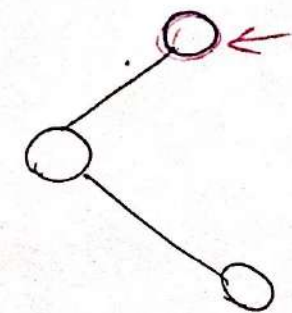


Τα υποδένδρα ενός
AVL tree είναι
επίσης AVL trees

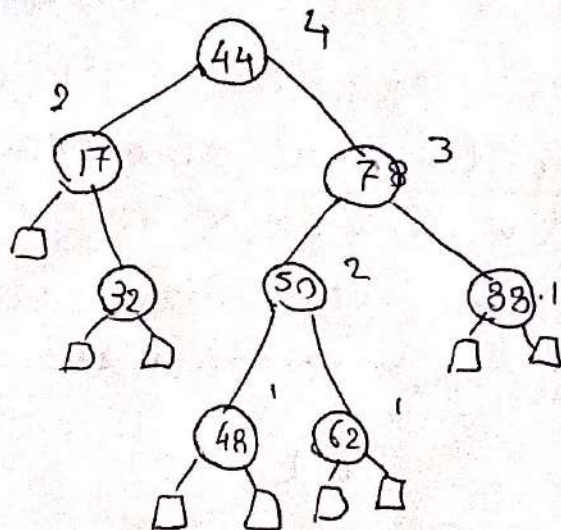
right-heavy



Ποιοι είναι οι κόμβοι που παραβιάζουν την ιδιότητα
ισοσομίας ύψους σε ένα AVL δέντρο;



Σημειώνετε AVL trees



Υψος $h = O(\log n)$

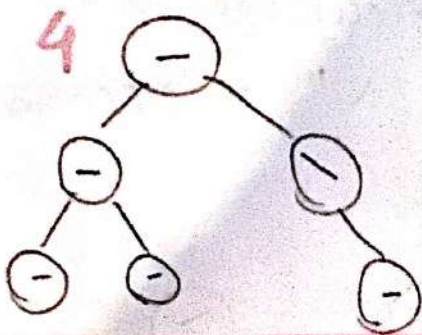
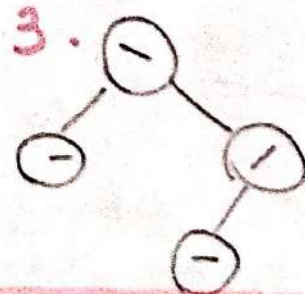
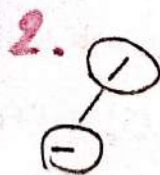
Αριστερά υψηλότερος κόμβος ενός AVL δέντρου, αν το ύψος του αριστερού παιδιού είναι 1 μονάδα μεγαλύτερο από το ύψος του δεξιού υποδένδρου. Συμβολίζεται με $"/$

Δεξιά υψηλότερος κόμβος ενός AVL δέντρου, αν το ύψος του δεξιού υποδένδρου είναι 1 μονάδα μεγαλύτερο από το ύψος του αριστερού υποδένδρου. Συμβολίζεται με $" \backslash "$ Ανισοδ.

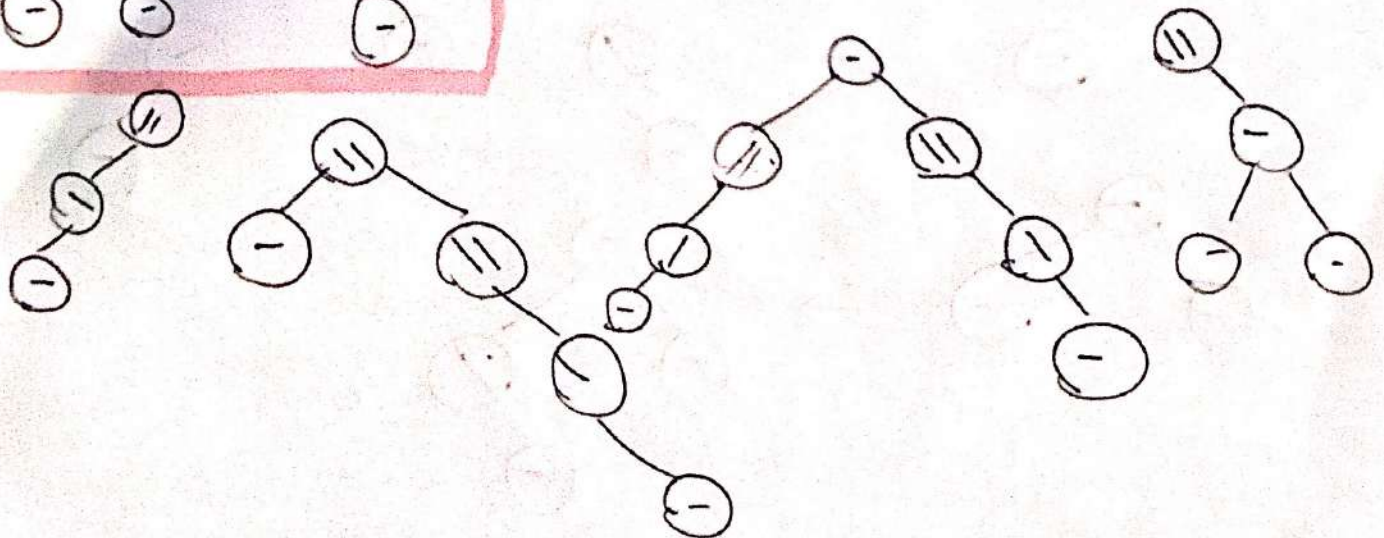
Παραγοντας ισοζυγίου ισός με 0 $" - "$

Δεν πληρούν ιδιότητα AVL $" // " " \backslash "$

AVL Tree 1. 



non AVL Tree



α είναι ο παράγοντας ισοζυγίου (Balance Factor)
σε AVL δέντρο

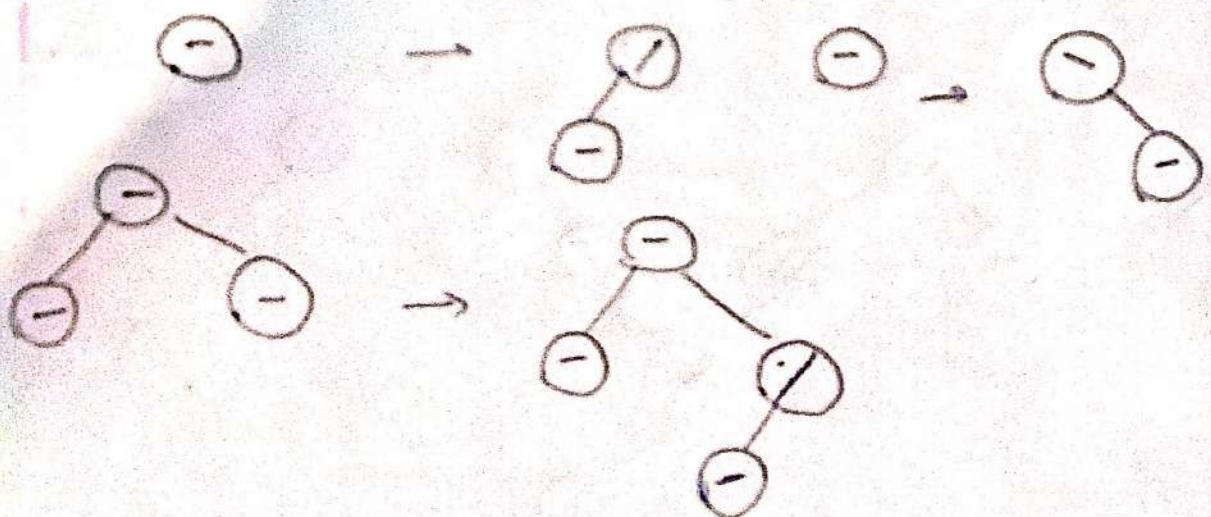
Στα AVL δέντρα θέλουμε να μην γίνει. Ποιο υψόμε-
τρο είναι γραμμές (αριστερό & δεξί) ή αν έχουν το
ίδιο υψόμετρο τότε ο Balance Factor

Rebalancing σε AVL

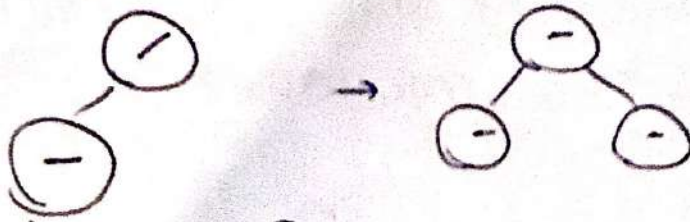
Τα AVL δέντρα είναι BST που διατηρούν την
ισορροπία τους. Αν κατά την εισαγωγή ή διαγραφή
ένος κόμβου χάσουν την ισορροπία τους, τότε εισαγ-
ρούνται περιστροφές (rotations) για να την επαναφέρουμε.
Αυτός ο αλγόριθμος ονομάζεται rebalancing.

Εισαγωγή σε AVL

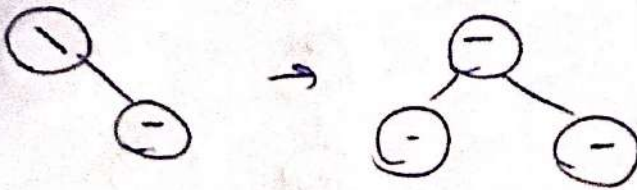
Η περίπτωση Ο κόμβος N έχει ισορροπία -1 . Αν
εισαγούμε νέο κόμβο στα αριστερά του δένδρου γίνεται
ο N "left high" ενώ αν εισαγούμε στο
δεξί, ο δένδρος γίνεται right high "\". Η ισορροπία
δεν αλλάζει, αλλά επιδεινώνεται.



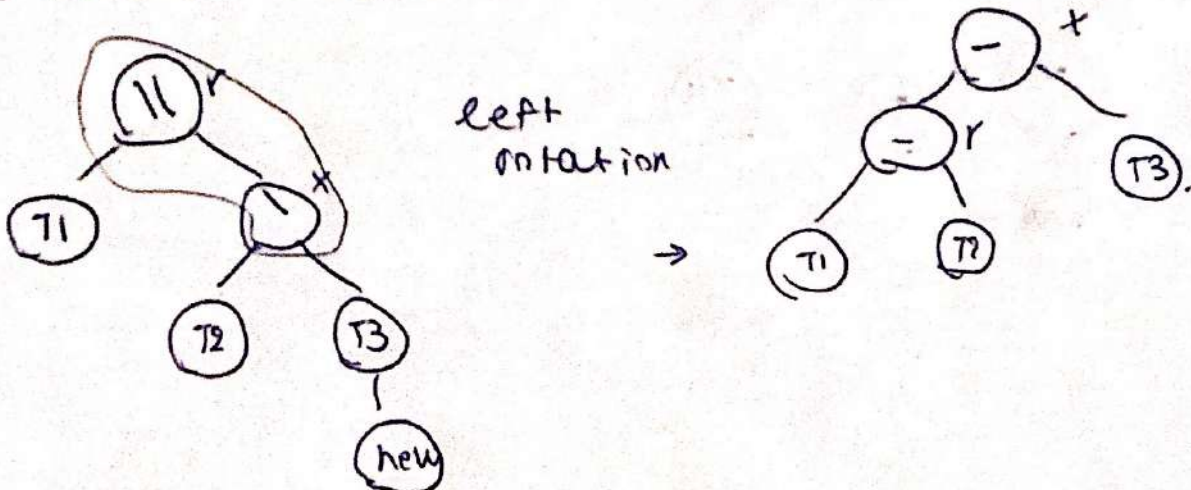
2η περίπτωση: Ο κόμβος είναι αριστερά υψηλότερος και εισάγουμε κόμβο στο δεξί του υποδένδρο. Η ισορροπία δεν γίνεται.



3η περίπτωση: Ο κόμβος είναι δεξιά υψηλότερος και εισάγουμε κόμβο στο αριστερό υποδένδρο. Η ισορροπία δεν γίνεται.

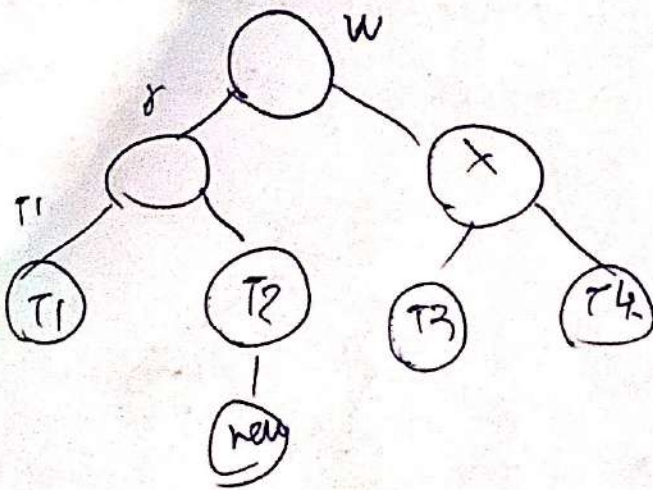
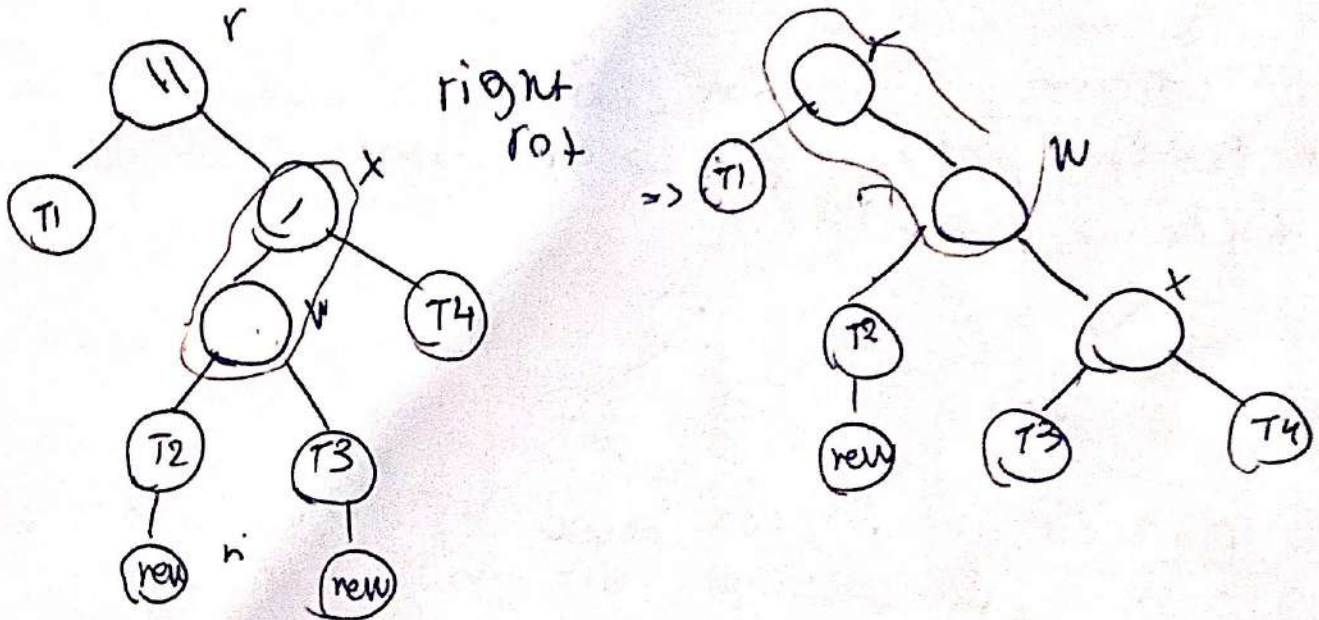


4η περίπτωση: Ο κόμβος είναι δεξιά υψηλότερος και εισάγουμε κόμβο στο δεξί του υποδένδρου. Αν συμβεί αυτό, το ύψος του δεξιού υποδένδρου αυξάνεται περισσότερο και η διαφορά ύψους μεταξύ δεξιού και αριστερού υποδένδρου γίνεται 2 και γίνεται η ισορροπία.



Στη περίπτωση Το x είναι αριστερά υψηλότερο
και εισαγούμε κόμβο στο αριστερό του υποδένδρο

Λύση: 1) Δεξιά περιστροφή στον x
2) Αριστερή περιστροφή στον w } right
left
rotation

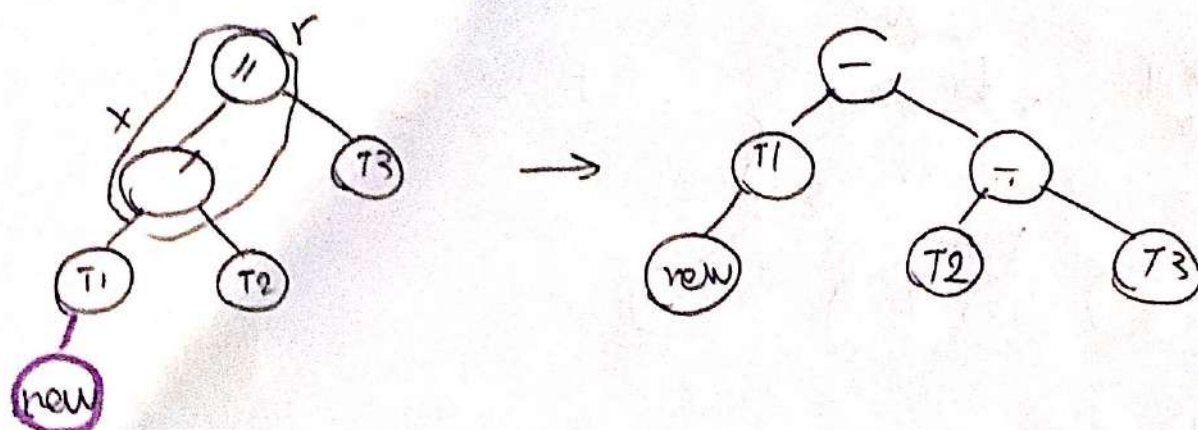


6η περίπτωση Ισα ύψη δεν μπορεί να συμβεί

Δεν χρειάζεται να εξετάσουμε αυτή τη περίπτωση

7η περίπτωση Ο κόμβος r είναι αριστερά-υψηλότερος και ο νέος κόμβος έχει εισαχθεί στο αριστερό υποδένδρο του αριστερού παιδιού του r .

Από δεξιά περιστροφή στον r



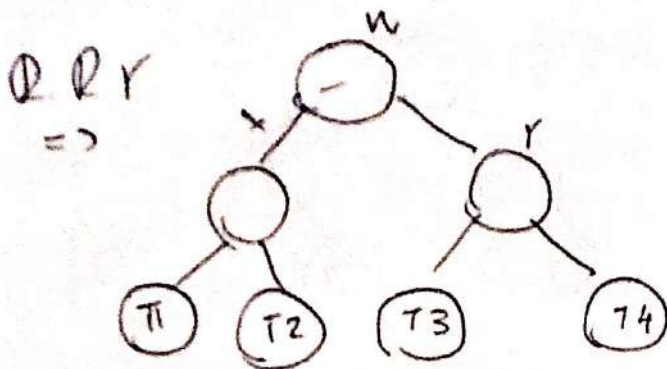
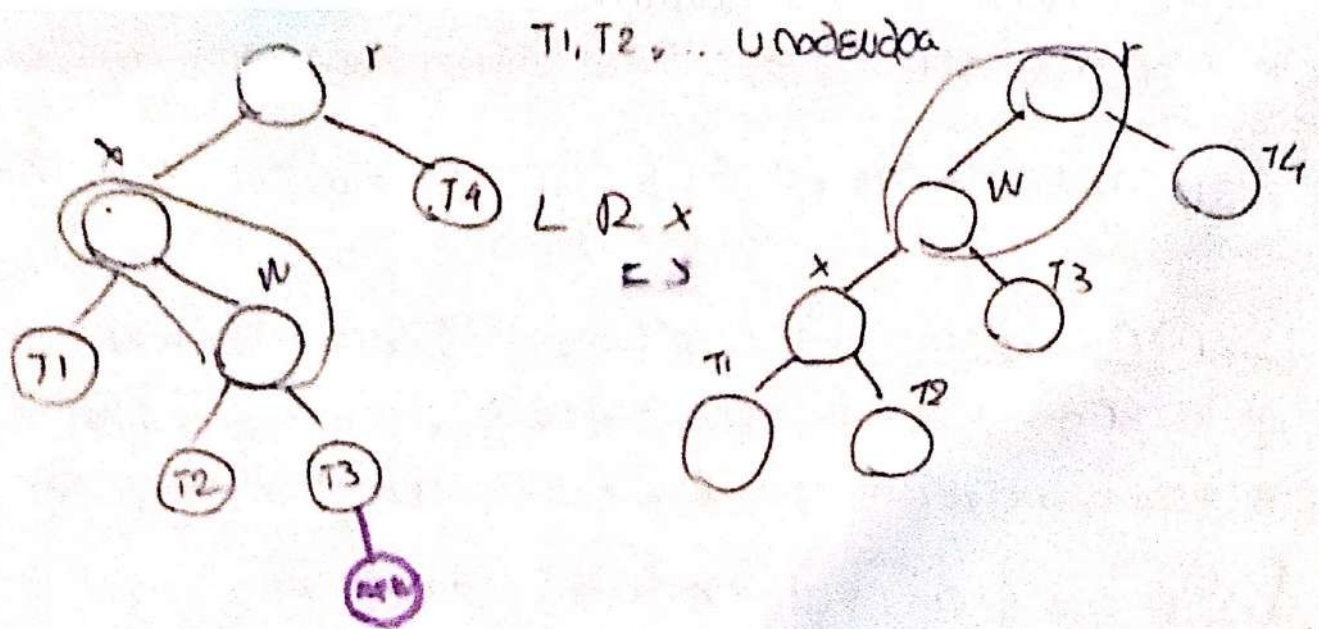
8η περίπτωση Ο κόμβος r είναι αριστερά-υψηλότερος και ο νέος κόμβος έχει εισαχθεί στο δεξί υποδένδρο του αριστερού παιδιού του r

Δίνω Αριστερά-Δεξιά περιστροφή στους x και r .

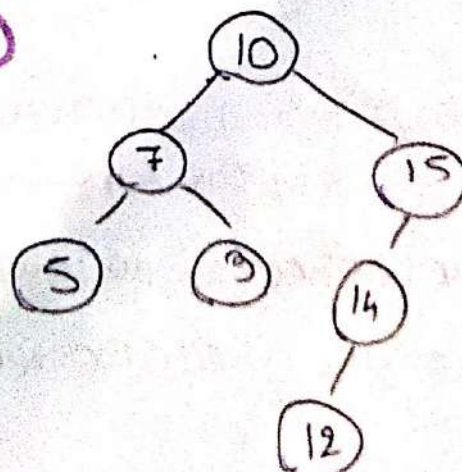
Βήματα: 1. Αριστερή περιστροφή στον x

2. Δεξιά περιστροφή στον r

Σημείωση: οι περιστροφές είναι τοπικές (πραγμαζονται μόνο στα 2 ή 3 κόμβους ενός υποδένδρου)
→ Δεν απαιτούνται επιπλέον περιστροφές ή αλλαγές στις παράγοντες ισορροπίας (balance factors) σε αυτόματη εξέταση του δένδρου.



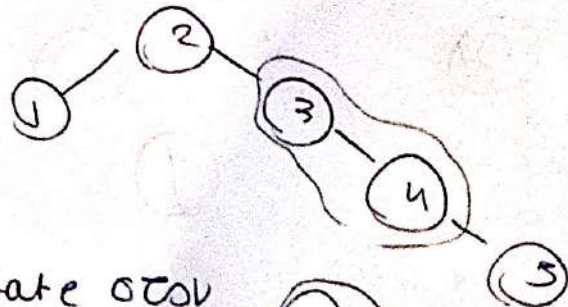
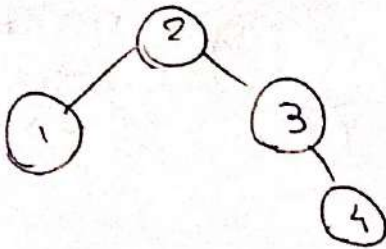
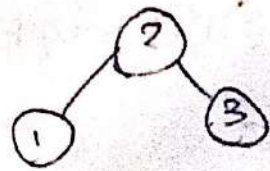
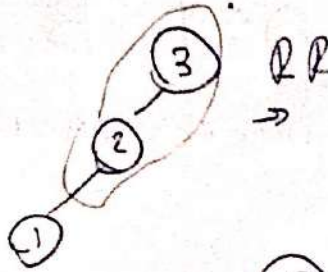
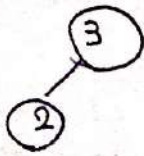
Άσκηση 1



Το δέντρο δεν είναι AVL, ο κόμβος με κλειδί 15 δεν έχει ιδιότητα AVL, ο κόμβος με κλειδί 14 είναι αριστερά υψηλότερος.

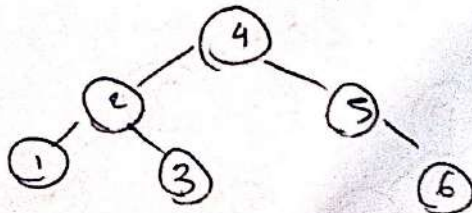
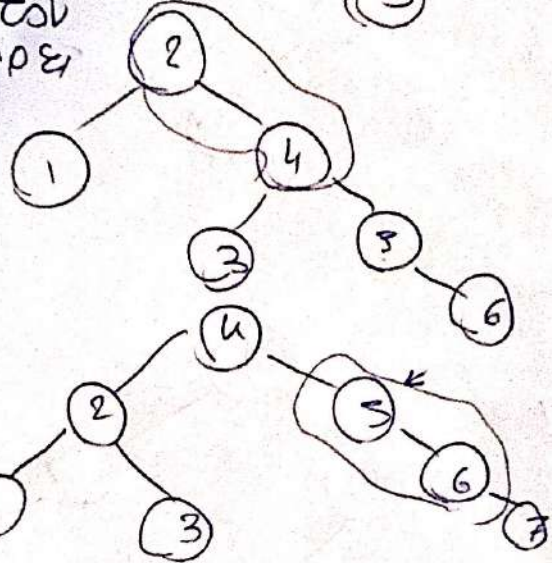
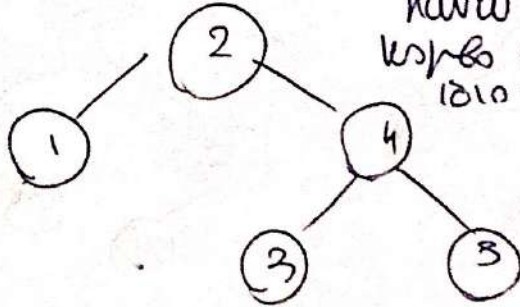
- 2) Αν εισαγάγουμε τα κλειδιά 3, 2, 1, 4, 5, 6, 7, 16, 15 σε ένα δέντρο AVL αρχικά κενό, ποιο δέντρο AVL θα προκύψει;

3

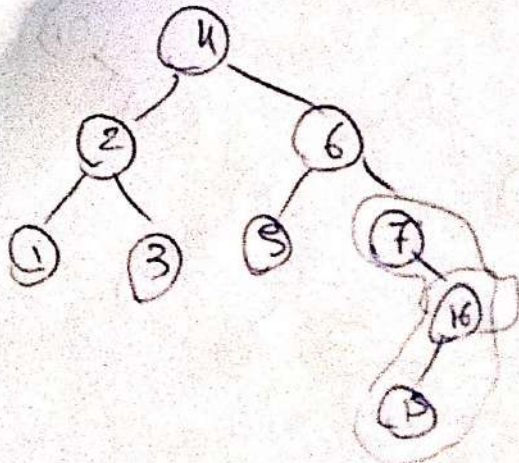
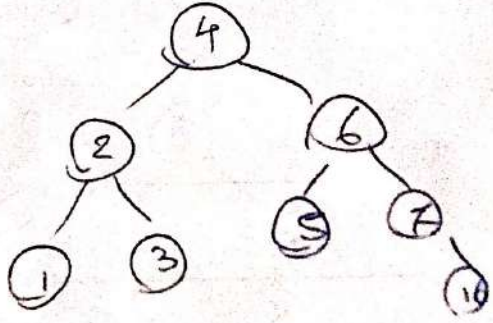
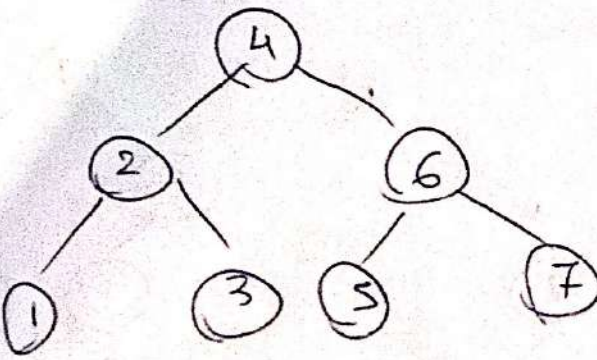


LR
=>

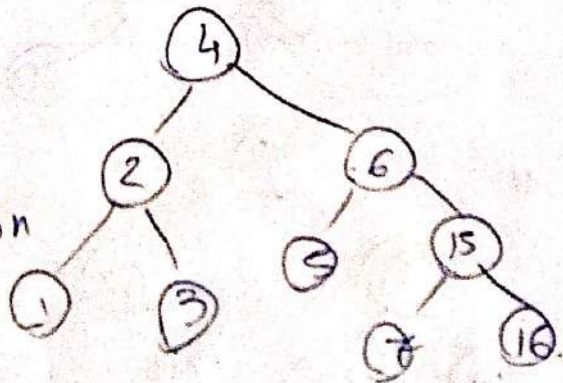
κατω rotate στο
υψηλο που δεν ειναι
ιδιο ευραχυλ



Left rot
=>



Right
Left rotation



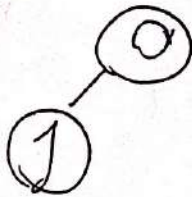
③ Εισαγωγή O, J, B, D, M, Z, OD, X, GL^A

A, G (A < B < C < D < E < F < G < H < I < J < K < L < M < N < O < P < Q < R
S < T < U < V < W < X < Y < Z)

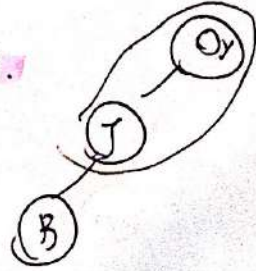
1.



2.

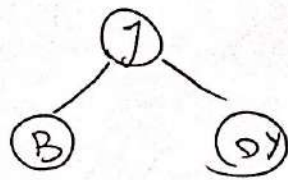


3.

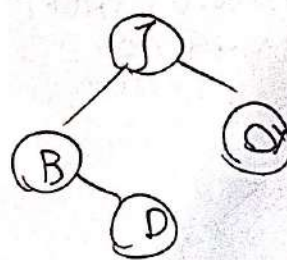


rightrot
O O V O
=>

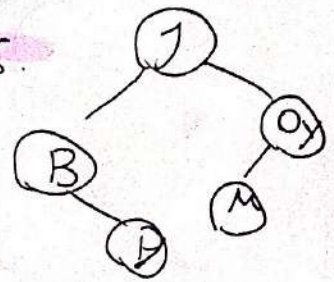
4.



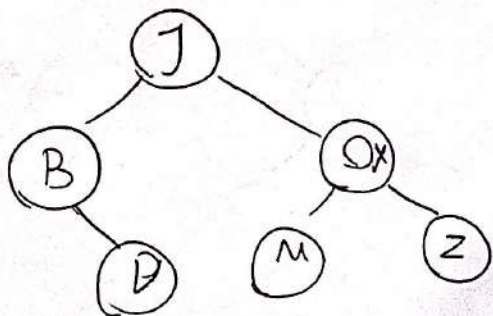
5.



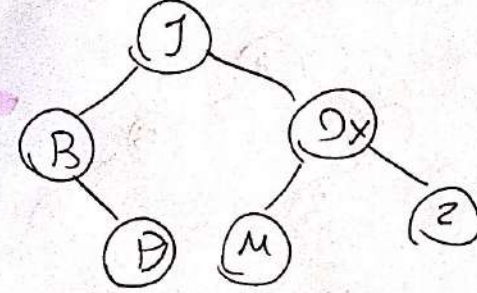
6.



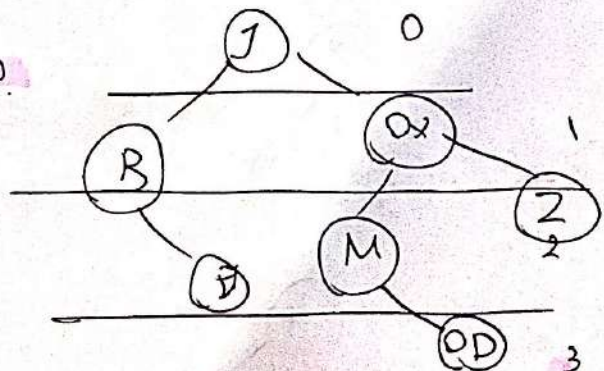
7.



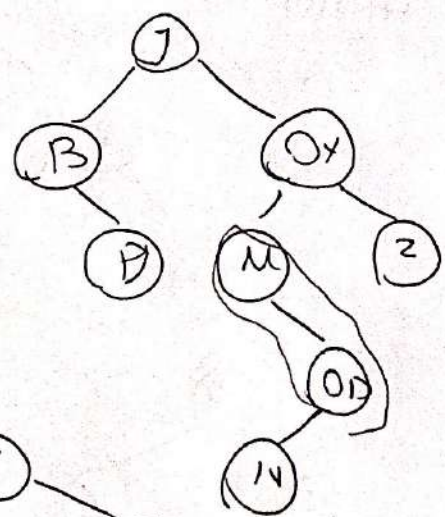
8.



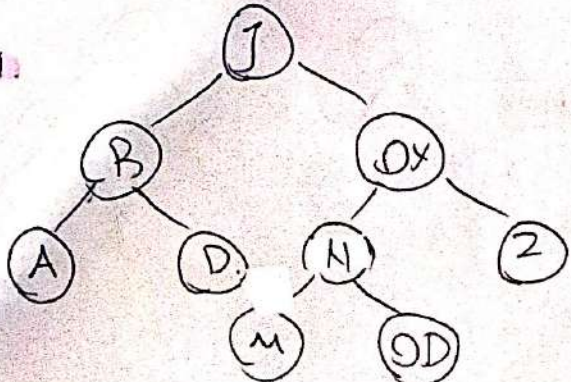
9.



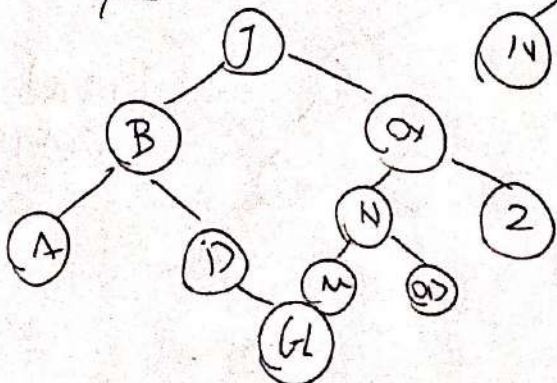
10.



11.



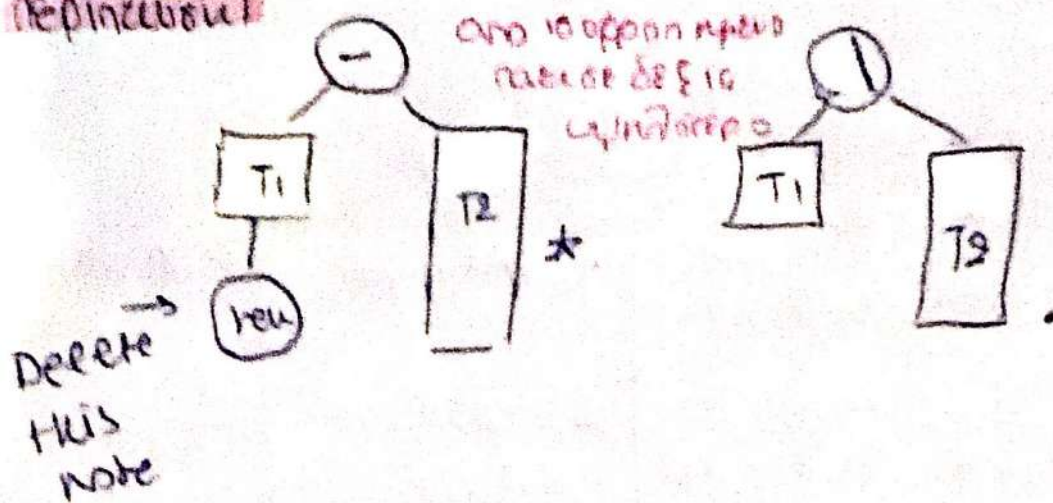
12.



Διαγραφή από AVL

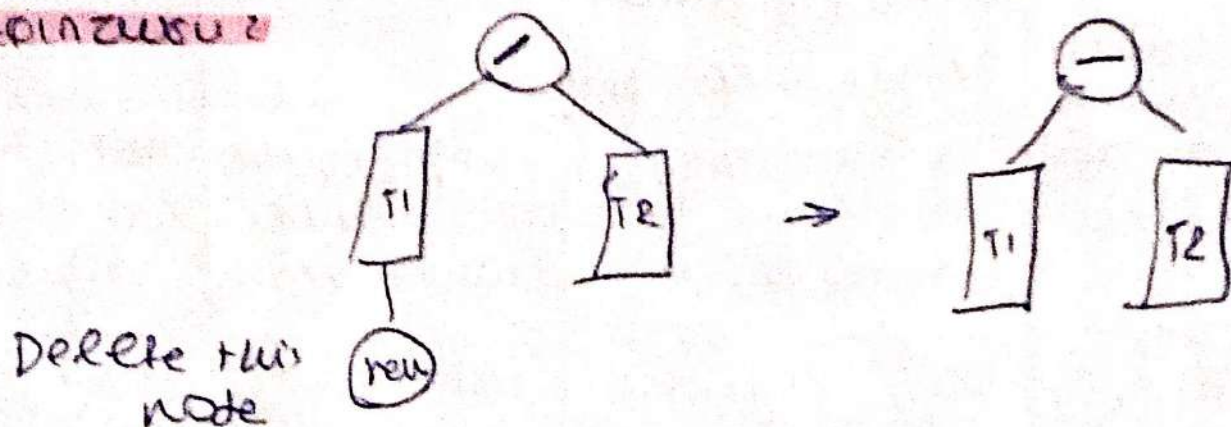
Περιπτωση 1. Ολα καλα	Πότε αφαιρείται	Τι κάνουμε	Συνέχεια
	Το ύψος δεν αλλάζει πάλι και το δέντρο είναι ακόμα ισορροπημένο	Δεν κάνουμε αμτα	Τελος
Ανάλυση 2 Ανάλυση	Το ψηλότερο παιδί έγινε ίδιο σε ύψος με το αλφ	Ευθυγραμμίζουμε πάλι η πλευρά ήταν πιο ψηλή	Συνεχίζουμε προς τα πάνω στο δέντρο
Ανάλυση 3α Περιστροφή	Το δέντρο γέρνει αλλά το βαρύ παιδί του είναι ουδέτερο	rotation (δεξιά ή αριστερά)	Συνεχίζουμε προς τα πάνω
Ανάλυση 3β Περιστροφή αλλά συνεχίζουμε	Το δέντρο γέρνει και το παιδί του σφραγίζεται με τη ηλικία του.	κάνουμε μια περιστροφή και συνεχίζουμε (αριστερά)	Συνεχίζουμε προς τα πάνω
Ανάλυση 3γ Περιστροφή	Το δέντρο γέρνει αλλά το παιδί γέρνει αντίθετα.	κάνουμε διπλή περιστροφή πρώτα δεξιά μετά αριστερά	Συνεχίζουμε προς τα πάνω
Συμμετρικές περιπτώσεις.	Το πρόβλημα είναι στη δεξιά αυτί για την αριστερή πλευρά ή αντίστροφα.	κάνουμε αμτα τα ίδια βήματα αλλά αλλα (πρ δεξιά αυτί για αριστερά)	ίδια αντιμετώπιση

непринципи

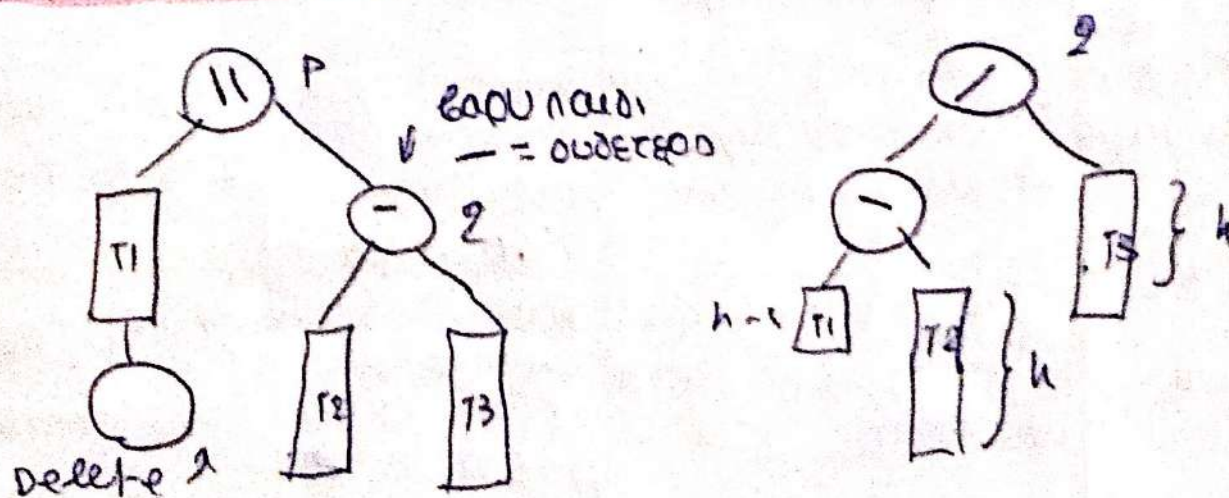
[illegible]

* Θέλει να
δείξει ότι
ο υποδεδω-
το είναι
μεγαλύτερο
έχει μεγαλύτερο
βάρος
T2: h
T1 = h-1

2024/11/19

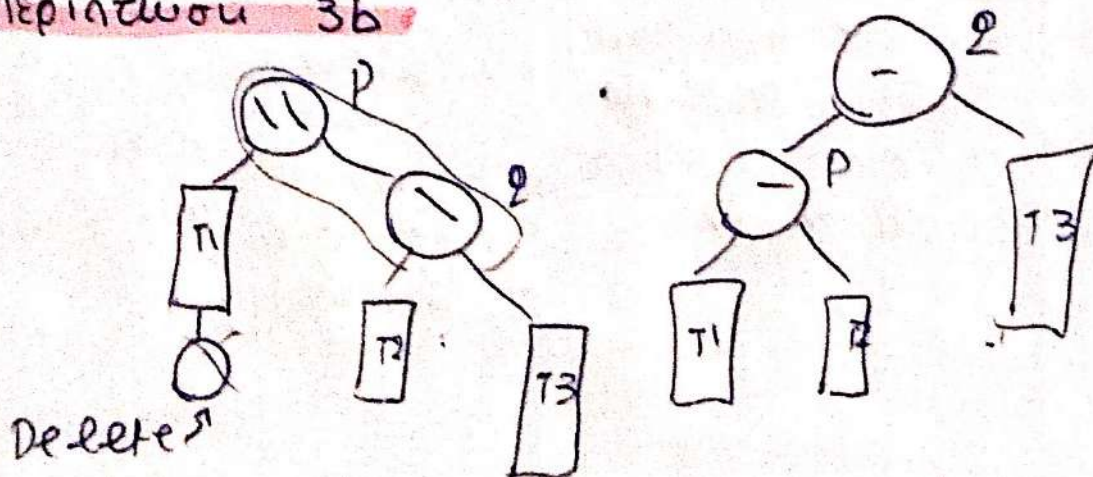


ΛΕΞΙΛΟΓΙΟ 3α



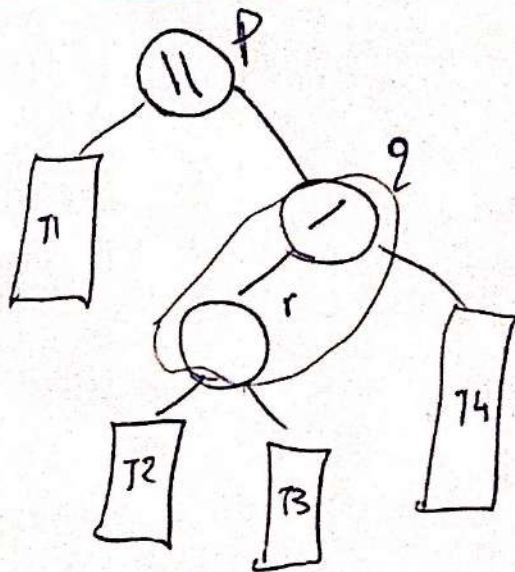
ΕΑΡΥ ΠΑΙΔΙ
 — = ΟΥΔΕΤΕΡΟ

περίπτωση 3b

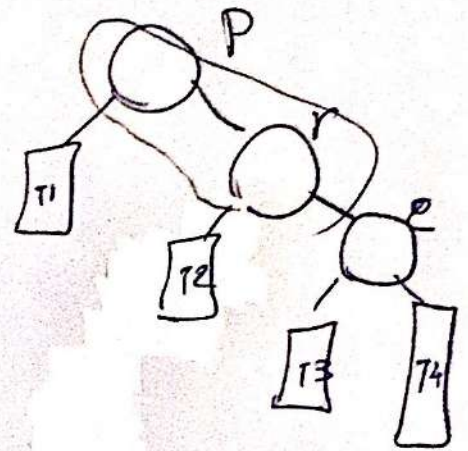


Δομές AVL

Περίπτωση 3c

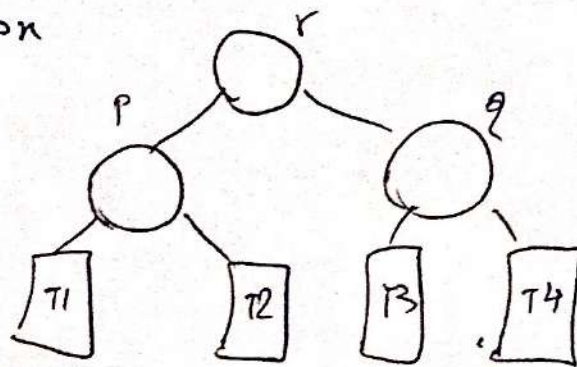


right
rotation
=>



Left rotation

=>



Ανάλυση AVL

Λεδομένου ότι ένα AVL είναι ένα BST,
ο αλγόριθμος αναζήτησης είναι ίδιος με αυτόν
των BST

Multi way search trees

δέντρα στα οποία κάθε εσωτερικός κόμβος μπορεί να αποθηκεύει περισσότερα από 1 κλειδιά και να έχει περισσότερα από 2 παιδιά

Διατεταγμένο (ordered) δέντρο. Όταν υπάρχει γραμμική διατάξη για τα παιδιά του κάθε κόμβου. Δηλαδή, μπορούμε να ταυτοποιήσουμε τα παιδιά ενός κόμβου ως 1ο, 2ο, 3ο κτλ

d-κόμβος

Έστω v ένας κόμβος σε ένα διατεταγμένο δέντρο. Θα λέγεται ότι ο v είναι d -κόμβος αν έχει d παιδιά.

! Ένα δυαδικό δένδρο αναζήτησης (BST) είναι μια ειδική περίπτωση δένδρου αναζήτησης με $m=2$ διαδοχικών όπου κάθε εσωτερικός κόμβος αποθηκεύει μια καταχώρηση και έχει 2 παιδιά ($m=2$)

! Δεν επιτρέπονται διπλά κλειδιά $\rightarrow$ κατάλληλη δομή δεδομένων για μια υλοποίηση map

! Εισαγωγή - διαγραφή multiway search $\rightarrow O(\log t)$

Βαθιά αναζήτηση $\rightarrow O(\log m)$ Αν το εξισορροπώσουμε καλύτερα νομίζουμε πιθανά και δυαδική αναζήτηση

Σχέσεις με την αποτελεσματικότητα

Πρόβλημα

Η διατήρηση τέλει ισορροπίας στα BST οδηγεί σε μικρότερους χρόνους αναζήτησης. Όσοι, η προσπάθεια για την τέλεια ισορροπία κατά την εισαγωγή ή διαγραφή κόμβων, μπορεί να οδηγήσει σε rebalancing όπου κάθε κόμβος πρέπει να αναδιοργανωθεί.

Λύση

με AVL: τα δέντρα AVL μας δείχνουν έναν τρόπο να λύσουμε το πρόβλημα υιοθετώντας την προσέγγιση να διατηρούμε τα δέντρα σχεδόν ισορροπημένα. και όχι τέλεια.

με m-way search: διατηρούν το ύψος όσο το δυνατόν μικρότερο επιτρέποντας να ποικίλει ο αριθμός των κλαδιών σε κάθε κόμβο. Εδώ θέλουμε, το **ύψος** του δέντρου να είναι μια λογαριθμική συνάρτηση του συνολικού αριθμού των καταχωρήσεων που αποθηκεύονται στο δέντρο.

Ένα δέντρο αναζήτησης με λογαριθμικό ύψος ονομάζεται **ισορροπημένο δέντρο αναζήτησης**.

Ισορροπημένα δέντρα αναζήτησης

$\Rightarrow (2, 4)$ trees

$\Rightarrow$ Red black trees

$\Rightarrow (a, b)$ trees

$\Rightarrow B+, B-$ trees

Χρησιμοποιούμε την ελεγκταμένη μορφή τους

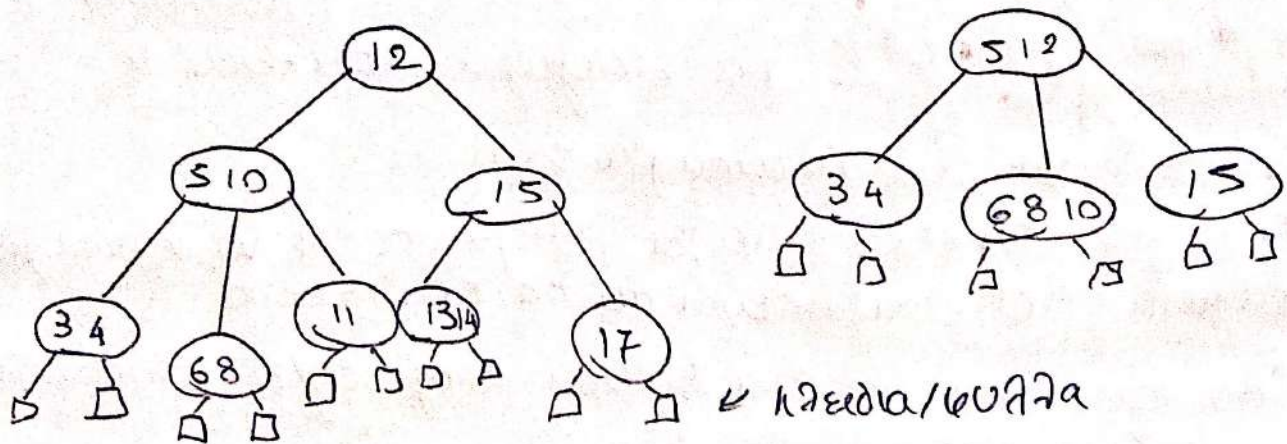
(2,4) Δέντρο

Ένα (2,4) δέντρο ή 2-3-4 δέντρο είναι ένα δέντρο αναζήτησης με 2 ή 3 ή 4 παιδιά με τα 2 παρακτωστά:

κάθε εσωτερικός κόμβος περιέχει τουλάχιστον 1 και το πολύ 3 κλειδιά

κάθε εσωτερικός κόμβος έχει τουλάχιστον 2 και το πολύ 4 παιδιά. (μέγιστο 4) ορί 4

† Όλοι οι εξωτερικοί κόμβοι είναι NULL (βρίσκονται στο ίδιο κατώτερο επίπεδο - έχουν το ίδιο βάθος)



← κλειδιά/κλειδιά

Υψος: $O(\log n)$ $n \rightarrow$ καταχωρίσεις

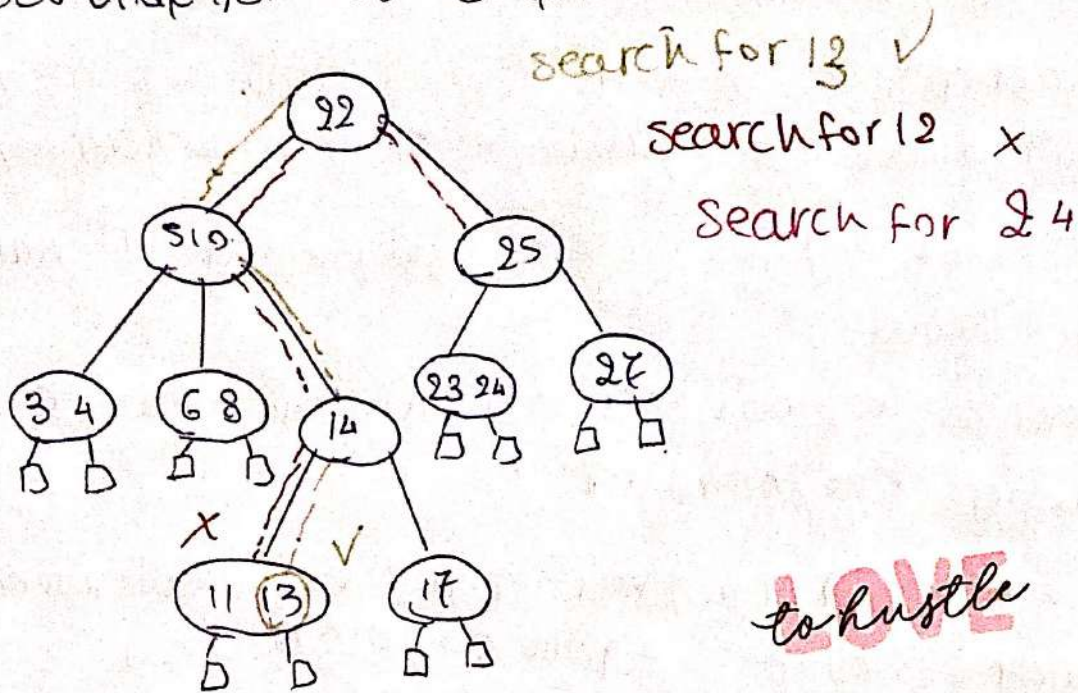
Αναζήτηση

Εστω ότι ψάχνουμε το k

1. Ξεκινάμε από τη ρίζα και πάμε προς τα κάτω
2. Όταν βρίσκουμε σε έναν d -κόμβο v συμπερινομε το k με τα αποθηκευμένα κλειδιά.
3. Αν βρούμε $k = k_i$ τότε η αναζήτηση ολοκληρώνεται επιτυχώς.

4. Διαφορετικά συνεχίζουμε τη αναζήτηση στο παιδί v_i τέτοιον ώστε $k_{i-1} < k < k_i$ (αν $k > k_{\text{κόμβου}} \rightarrow$ δεξιά, αν $k < k_{\text{κόμβου}} \rightarrow$ αριστερά)

5. Αν προσέχουμε σε εξωτερικό κόμβο, τότε το κλειδί δεν υπάρχει στο δέντρο



Εισαγωγή $O(\log n)$ για εισαγωγή στοιχείου k

1. Εκτελούμε αναζήτηση για k
2. Αν το στοιχείο υπάρχει δεν μπορούμε να κάνουμε εισαγωγή γιατί αναφορεύονται τα διλήρητα.
3. Αν δεν υπάρχει, η αναζήτηση τερματίζει αποκρίνοντας σε έναν εξωτερικό κόμβο z .
4. Έστω v ο γονέας του z . Εισάγουμε το κλειδί k στον κόμβο v και προσθέτουμε έναν νέο εξωτερικό κόμβο σε v στη αριστερή πλευρά του z .

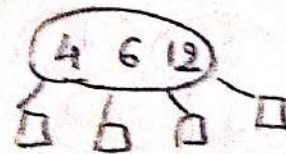
! Η μέθοδος εισαγωγής διατηρεί τα ιδιότητα εαθους

Όσοι, μπορεί να παραβιάσει τα ιδιότητα του μεγέθους
 Εάν ένας v κόμβος ήταν προηγούμενος 4-κόμβος τότε μπορεί να γίνει 5-κόμβος μετά τη εισαγωγή, κάτι που καθιστά το δέντρο μη $(2,4)$

Αυτή η παραβίαση της ιδιότητας μεγέθους ονομάζεται υπερχείλιση (overflow) στον κόμβο.

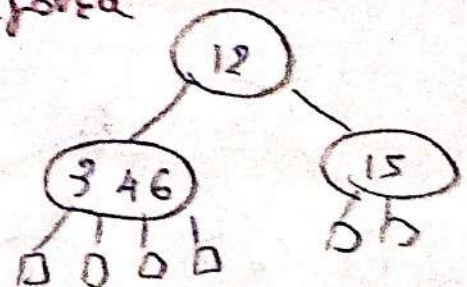
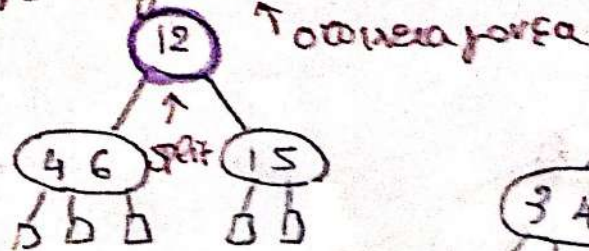
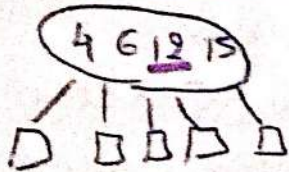
Παραδείγματα. Εισάγουμε τα 4, 6, 12, 3, 5, 10, 18 Νο 10
(2,4) δέντρο θα προκύψει

1.

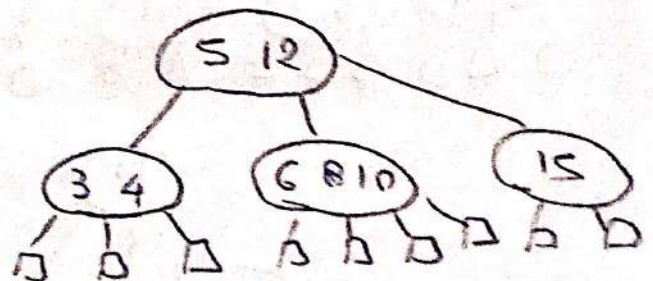
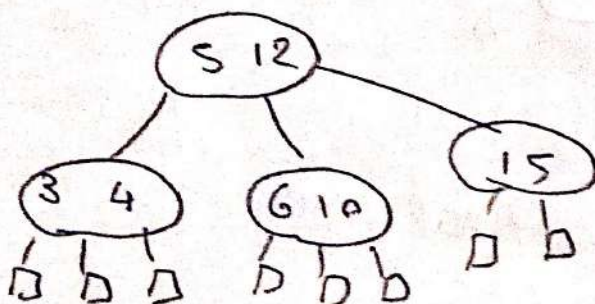
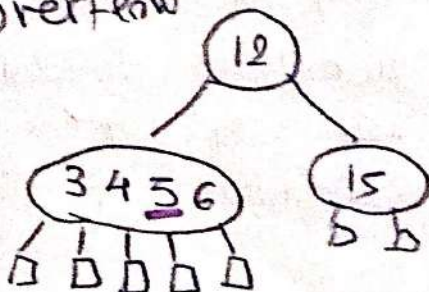


Εξωτερικοί κόμβοι = γονέας + 1

overflow



overflow

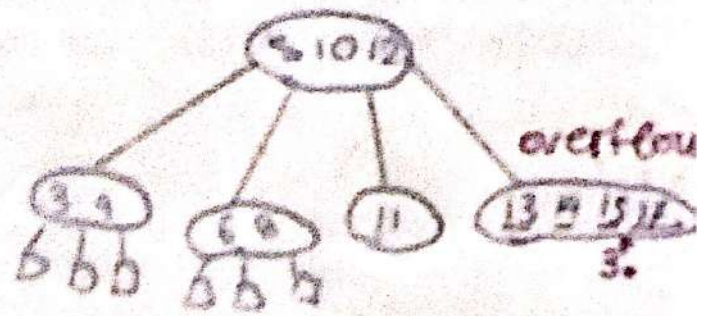
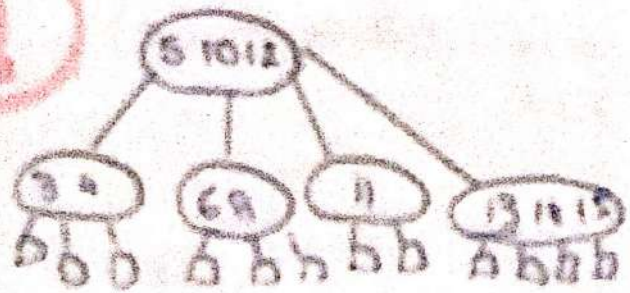


2.

(μη συνθετός) Εισάγουμε το κλειδί 15 στο παρακάτω δέντρο, Νο 10 (2,4) δέντρο θα προκύψει;

(Αν δεν καταλαβαίνετε πως δουλεύει το split ως μέθοδος αντιμετώπισης του overflow, το εξηγούμε παρακάτω :)

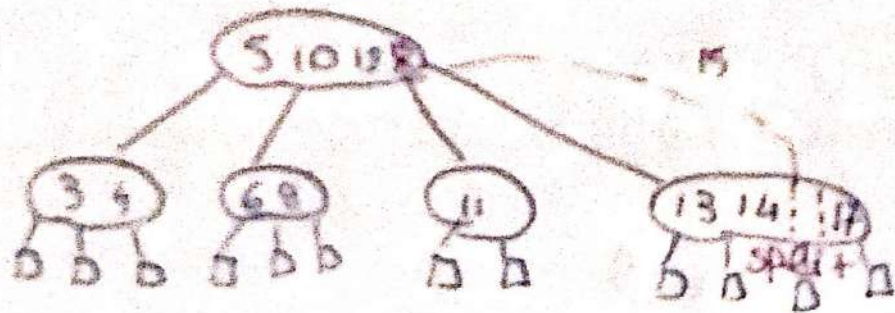
Αρχικό δέντρο



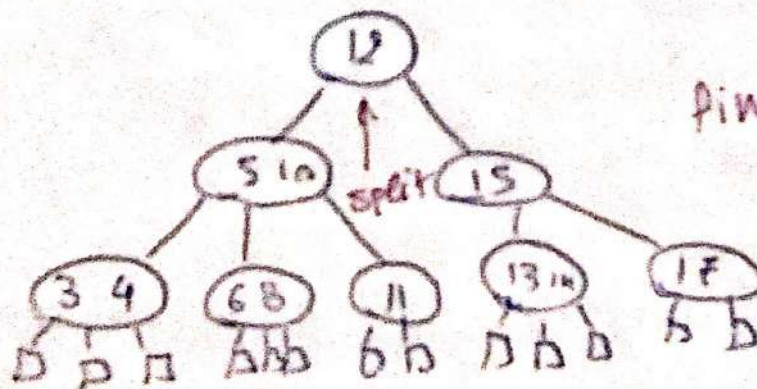
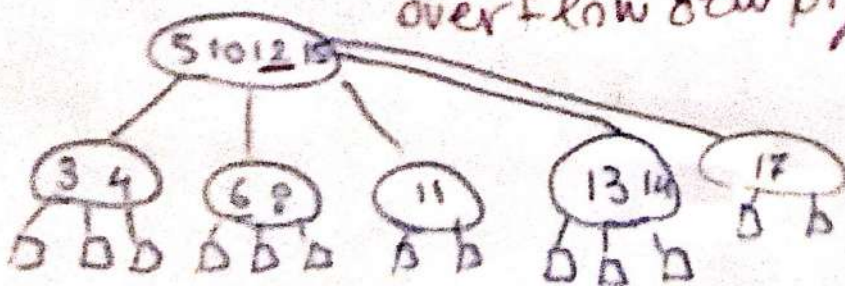
Overflow:

το 3ο στοιχείο
πρέπει να γίνει
κόμβος

πρωτεύοντα (10-20)
και 3ο κόμβος



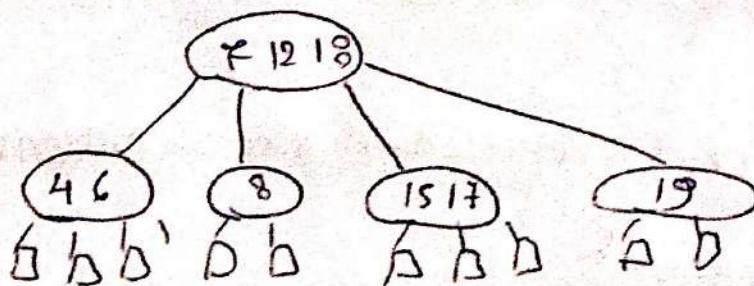
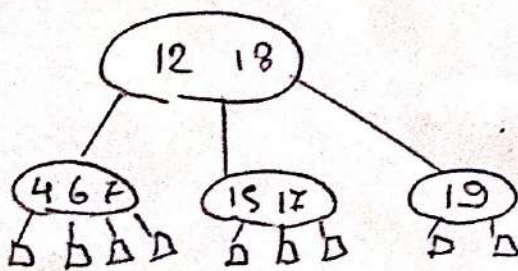
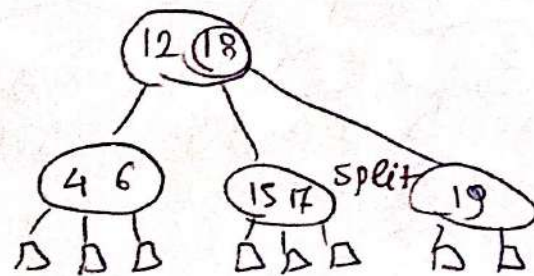
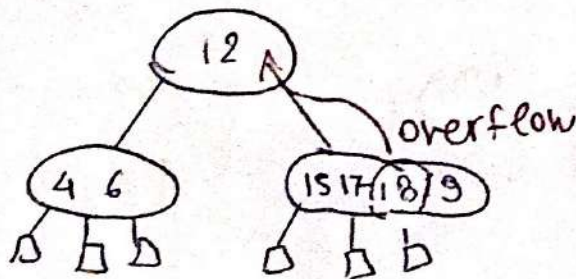
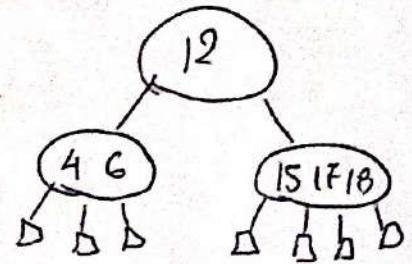
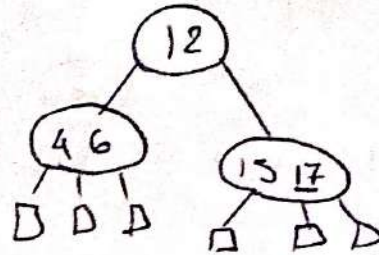
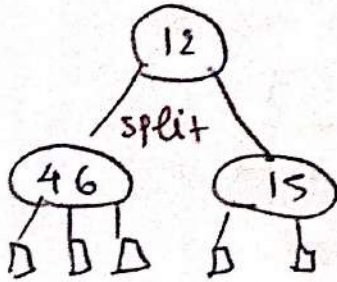
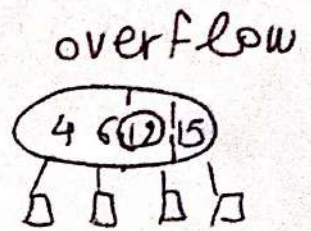
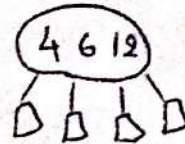
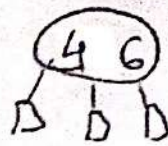
overflow σελή split!

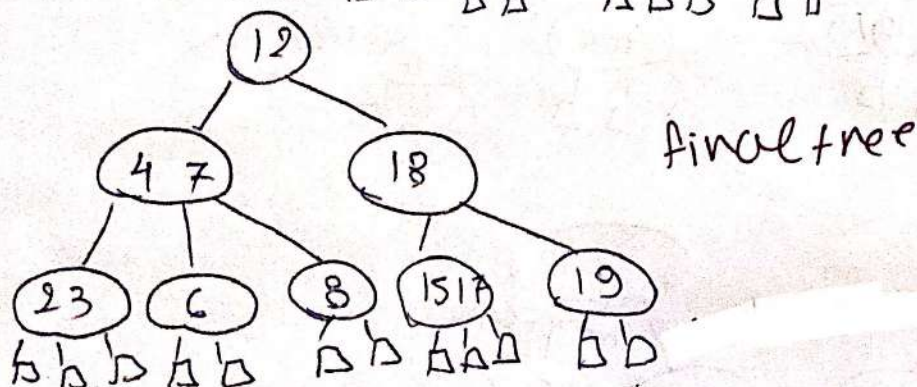
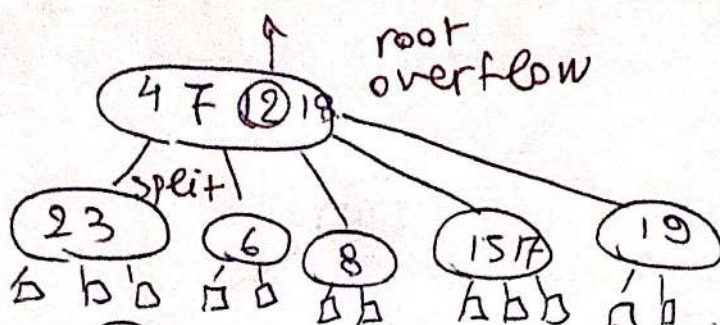
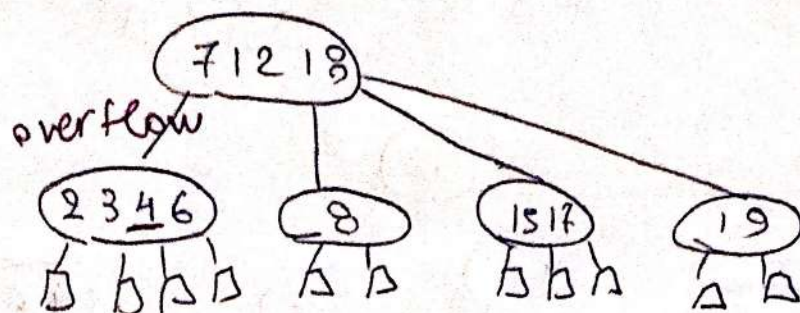
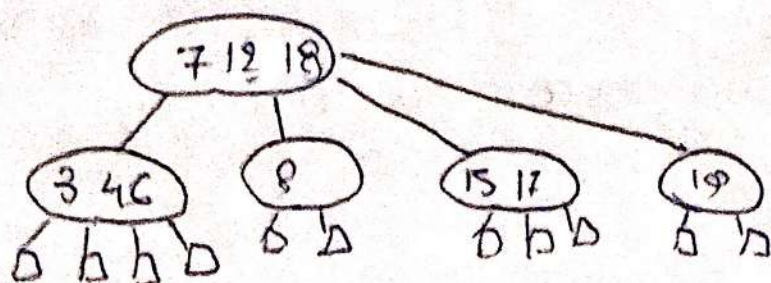


final tree



Εισαγούμε τα 4, 6, 12, 15, 17, 18, 19, 7, 3,
3, 2, Ποιο δέντρο θα προκύψει;





Διαφορετικά λίγο tricky αλγόριθμο

Ξέρει αρκετές περιπτώσεις χειρισμών και οι διαβάσεις το βιβλίο λίγο μεθόδευα?

1. Αρχικά εξετάζουμε αναζήτηση για να δούμε αν υπάρχει

Η περίπτωση

Αν το στοιχείο προς αφαίρεση είναι σε κόμβο με ^{NULL} φύλλα ως παιδί τότε $(4) \leftarrow$ στοιχείο προς αφαίρεση
 $\Delta \leftarrow$ φύλλα

→ Αφαιρούμε το κλειδί από αυτόν τον κόμβο

→ Αφαιρούμε το αντίστοιχο παιδί φύλλο που σχετίζεται με το κλειδί  underflow

Η "ισορροπία" του δέντρου ως προς το βάθος δεν αλλάζει.

Πως μπορεί να προκύψει πρόβλημα αν:

Ο κόμβος έχει μόνο ένα κλειδί προς διαγραφή.
Τότε μένει άδειος και αυτό ονομάζεται υπορροή

ΤΙ ΚΑΝΟΥΜΕ ΑΝ ΕΧΟΥΜΕ ΥΠΟΡΡΟΗ

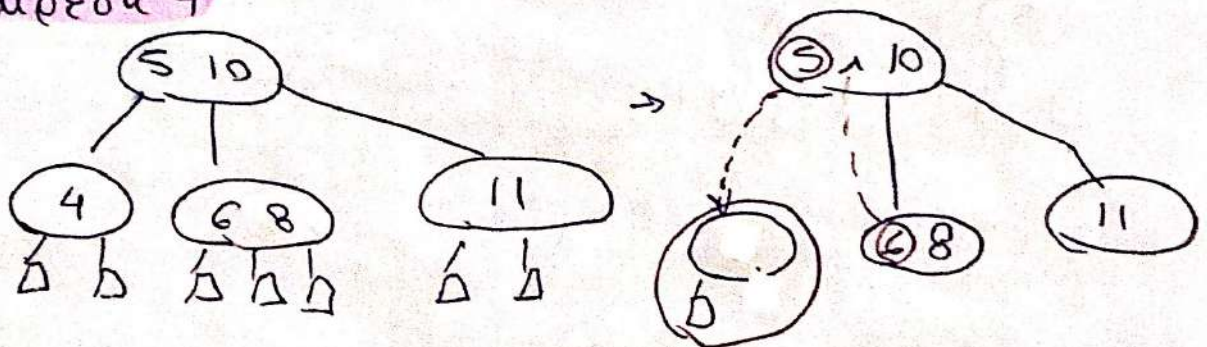
ΠΕΡΙΠΤΩΣΗ Α3 ^{(3 ή 4) ~} Ο γειτονικός κόμβος έχει παραπάνω κλειδιά
→ Δανειζόμαστε ένα κλειδί από τον "γείτονα" αδελφό και το παίρνουμε στον πατέρα κόμβο

→ Μετακινούμε ένα κλειδί από τον πατέρα κόμβο στον κόμβο του οποίου το στοιχείο αφαιρέσαμε

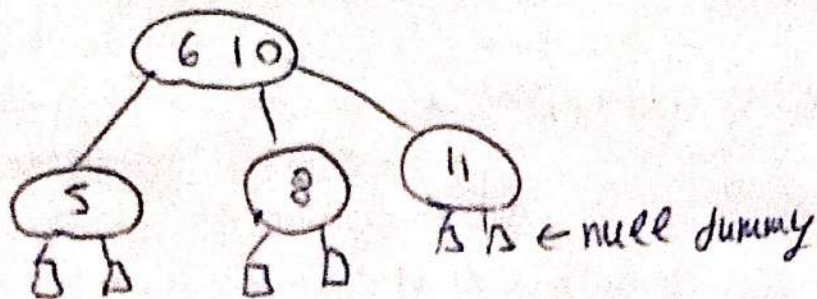
Έτσι: Συναρμολογεί ο άδειος κόμβος και όλα ισορροπούν

Αυτός λέγεται transfer (μεταφορά)

π.α.αφαίρεση 4



Στο transfer μεταφέρεται και ένα παιδί
transfer αριστερά → δεξιά, κουβαλάω το τέρμα δεξιά
παιδί από τον αριστερό κόμβο και ελαιώσω

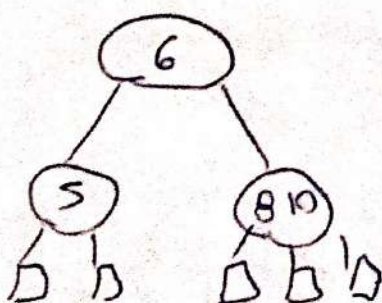
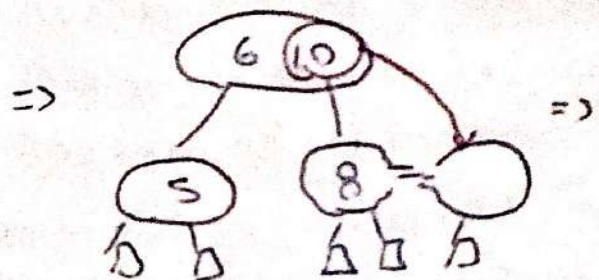
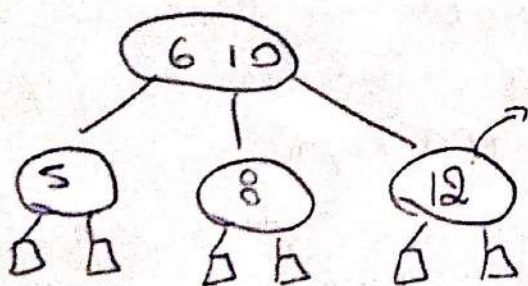


Περίπτωση Β Οι κόμβοι αδελφάκια έχουν κλάους μόνο 2 κλειδιά (δηλαδή στοιχείο και 2 NULL)

→ τότε ενώνουμε τον κόμβο με έναν από τους αδελφούς του

→ παίρνουμε ένα κλειδί από τον πατέρα και το βάζουμε στον νέο ευνομένο κόμβο

Αυτός λέγεται: συγχώνευση (fusion)
διαγραφή 12



**Τι γίνεται αν το πρόβλημα
βρεθεί στη ρίζα**

Αν μετά από πολλές συγχωνεύσεις και αλλαγές
αδελφάκια και η ρίζα, τότε η ρίζα διαγράφεται εντελώς

το δέντρο γάνει ένα ευκλειδ και συνεχίζει από τον επόμενο κόμβο

2η περίπτωση Αν το κλειδί που θέλουμε να απαρέσουμε δεν είναι σε φύλλο αλλά σε εσωτερικό κόμβο :

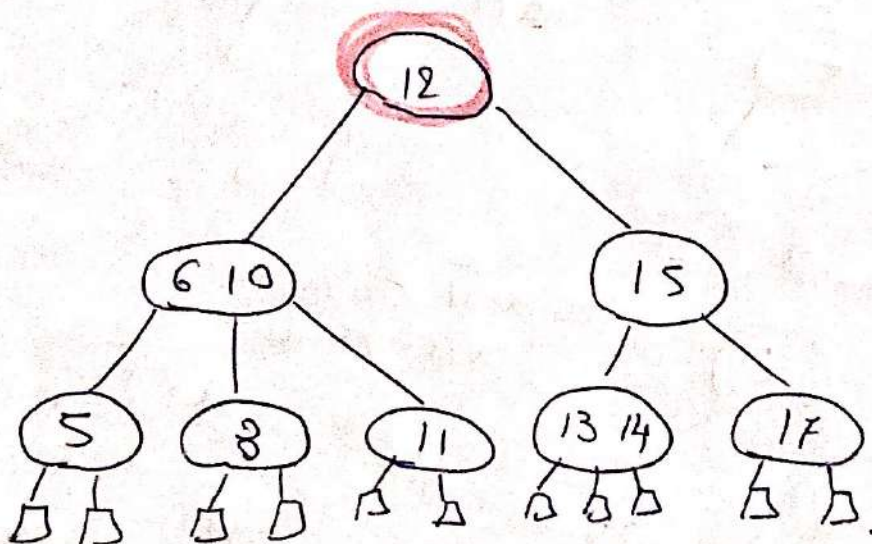
→ Πηγαίνουμε στο παιδί του που είναι πιο δεξιά και ακολουθούμε το μονοπάτι μέχρι να φτάσουμε σε έναν κόμβο που έχει μόνο φύλλα για παιδιά.

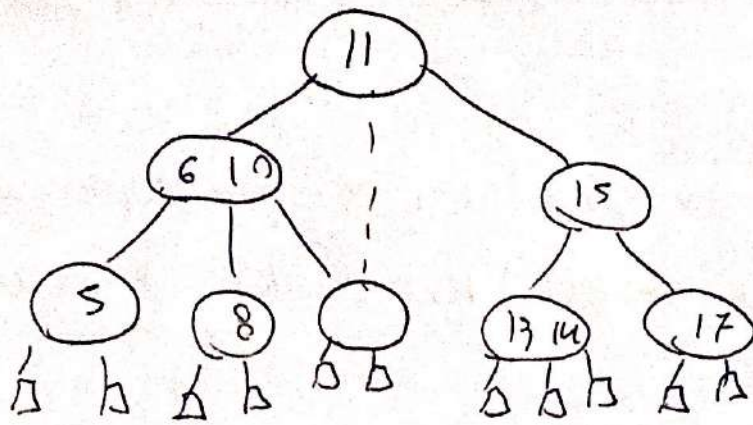
→ Εκεί βρίσκουμε το μεγαλύτερο κλειδί από εκείνο το υποδέντρο

→ Ανταλλάσσουμε τα δυο κλειδιά. **3.**

Τώρα έχουμε φέρει το πρόβλημα σε κόμβο με φύλλα και συνεχίζουμε όπως πριν.

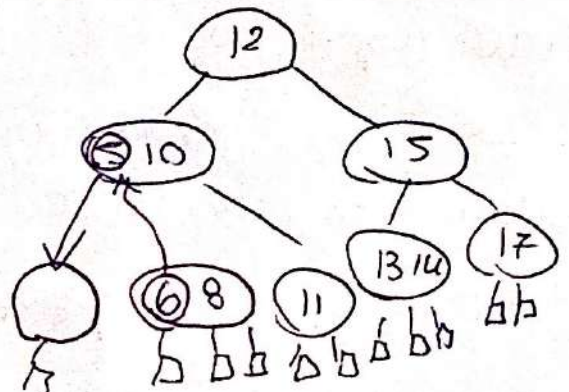
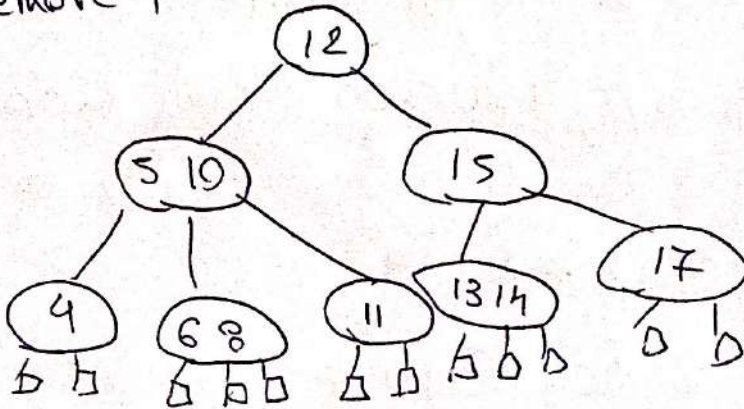
αφαίρεση το 12 (ρίζα) **1, 2, 3 method**



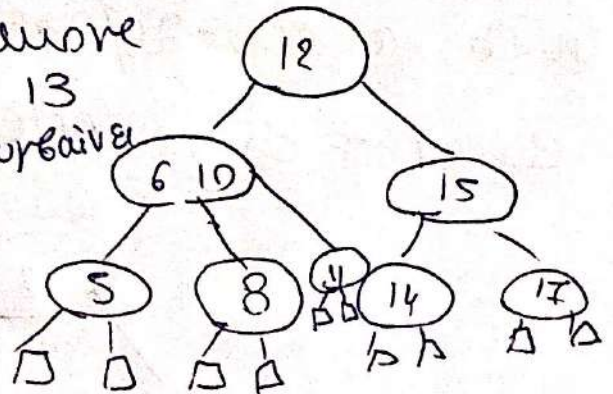
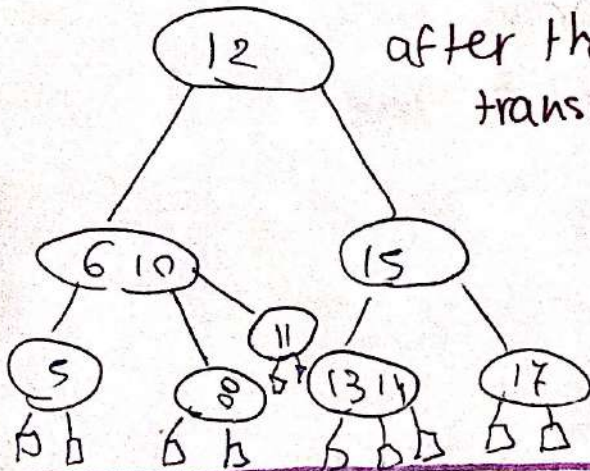


πείραμα παραδείγματα:

remove 4

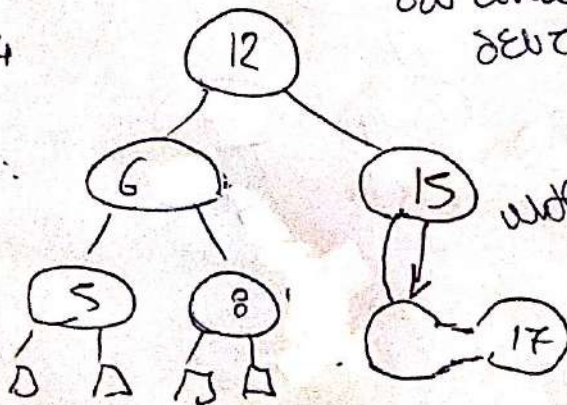


after the remove transfer
13
δεν υπάρχει
πια

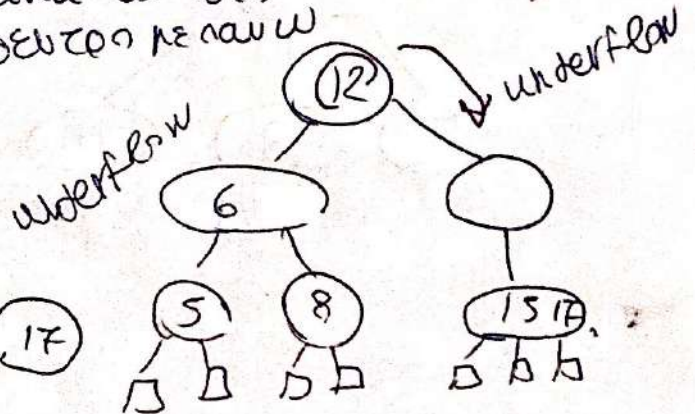


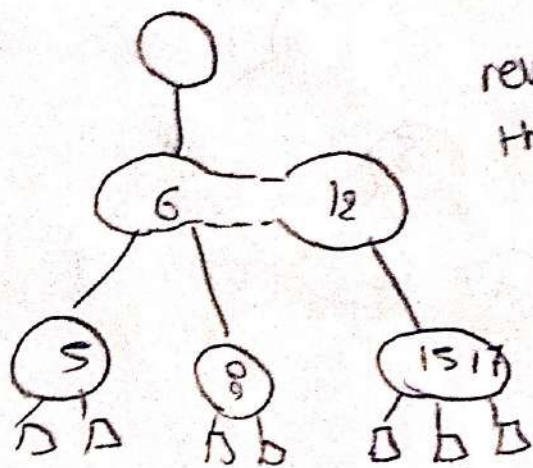
remove 14
Αφίηκε

δεν το

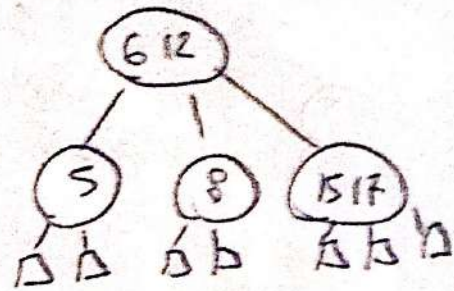


δεν είναι το ίδιο
δεν τον ξαναω

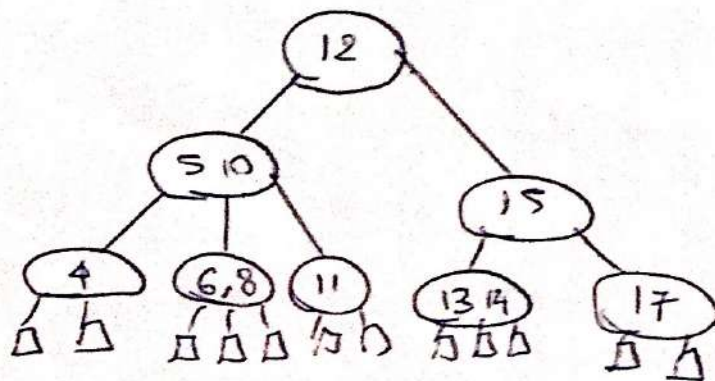




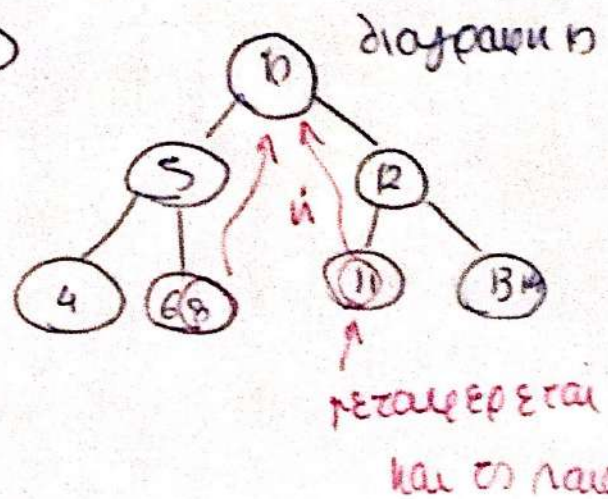
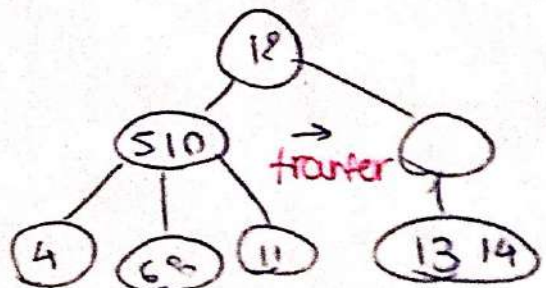
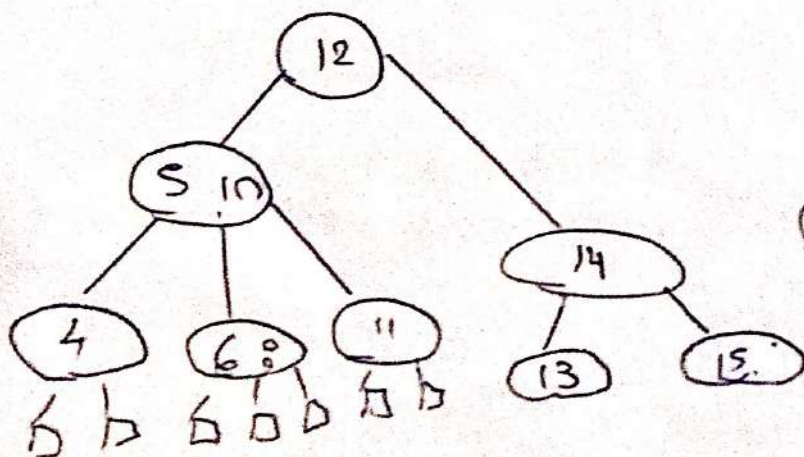
remove
the root

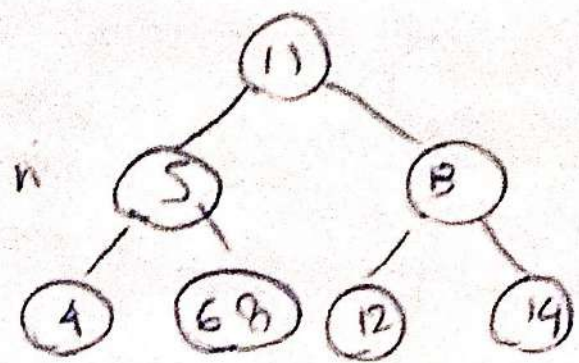
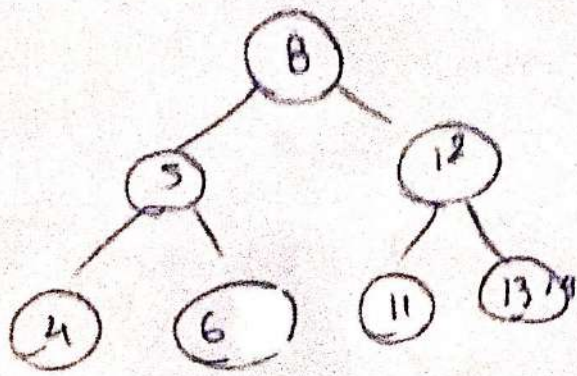


Αν διαγράψουμε τα κλειδιά 15, 10 που είναι δεύτερο
αποκλώνει (παλιό θέμα
δυσκόλο)



remove 17





Red Black trees

Τα AVL δέντρα και τα (2,4) έχουν μαζί κάποιες ιδιότητες αλλα

Τα AVL μπορεί να χρειαστούν μαζί περιστροφές μετά από μια διαγραφή

Τα δέντρα (2,4) μπορεί να απαιτήσουν μαζί διαχωρισμούς ή συγχωνεύσεις μετά από μια ενημέρωση (εισαγωγή ή διαγραφή)

Τα red black trees είναι μια δομή δέντρων που απαιτεί O(1) δομητικές αλλαγές μετά από μια ενημέρωση ώστε να παραμείνει ισορροπημένη.

Ορισμός

Ένα red black trees είναι ένα bst με κόμβους χρωματισμένους κόκκινους και μαύρους

Ιδιότητες

↑ ο αδελφός ενός εξωτερικού κόμβου είναι εξωτερικός κόμβος ή κενός κόμβος

=> Ιδιότητα ρίζας (root property) Η ρίζα είναι μαύρη
Εξωτερική ιδιότητα: κάθε εξωτερικός κόμβος (null) είναι μαύρος εννοείται null

=> Εσωτερική ιδιότητα: τα παιδιά ενός κόκκινου κόμβου είναι μαύρα

=> Ιδιότητα βάθους (depth property) Όλοι οι εξωτερικοί κόμβοι έχουν το ίδιο μαύρο βάθος, το οποίο ορίζεται ως ο αριθμός των μαύρων προγόνων μιν 1. (Θυμίζουμε ότι ένας κόμβος θεωρείται προγόνος του εαυτού του). Δεν συζητώνται τα δίντα κ παιδιά

Αναζήτηση

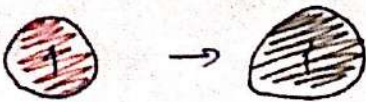
Ο αριθμός για τη αναζήτηση σε ένα δέντρο
κοκκυνο-μαύρο είναι αριθμώς ίδιος με τον αριθμώς
αναζήτησης σε bst. $O(\log n)$

Εισαγωγή

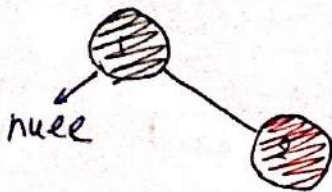
Εισαγωγή 1

η ρίζα είναι κοκκίνη $\rightarrow$ τω πάνω μαύρη

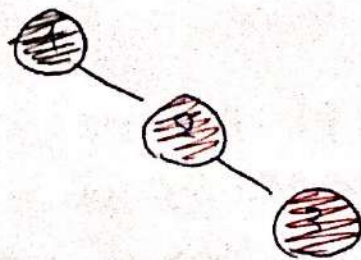
Εισαγωγή
 $O(\log n)$
αποκατάσταση
 $O(\log n)$



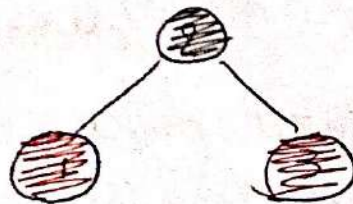
Εισαγωγή 2



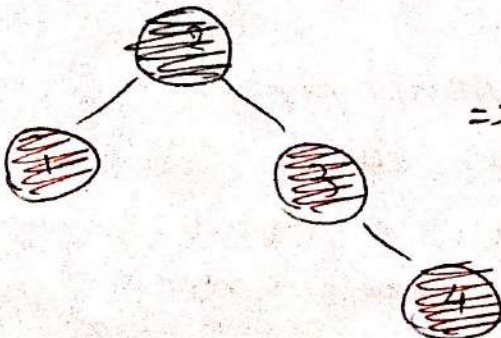
Εισαγωγή 3



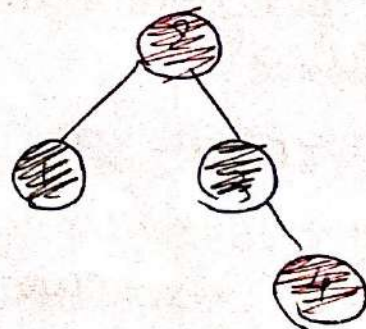
$\Rightarrow$



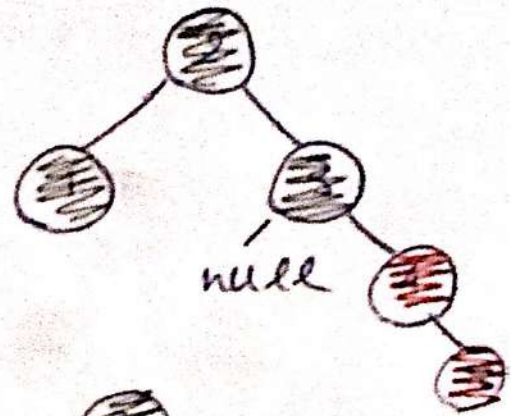
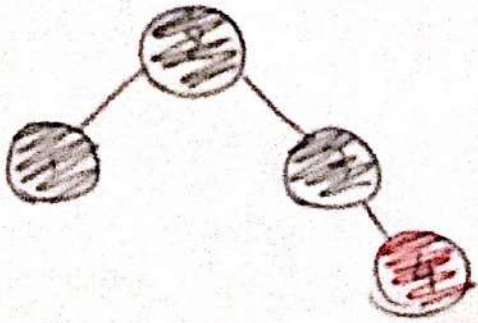
Εισαγωγή 4



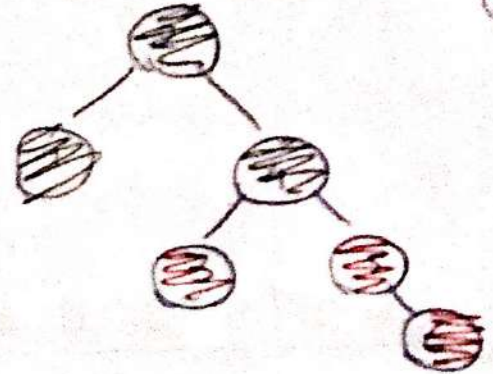
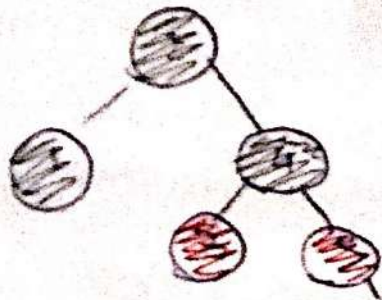
$\Rightarrow$



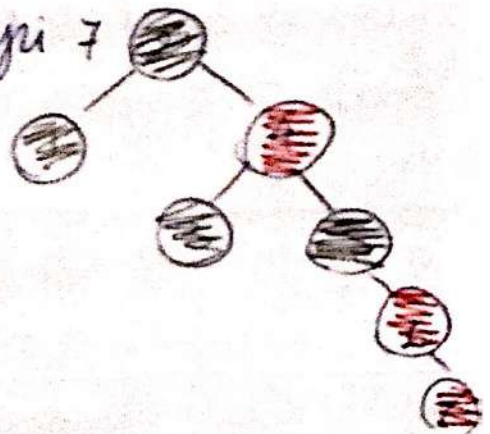
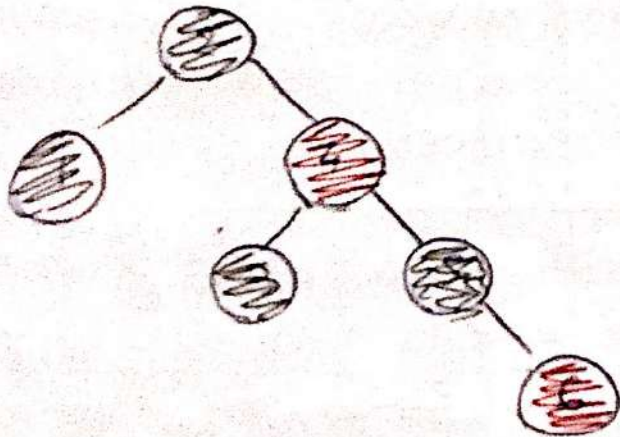
Εισαγωγή 5



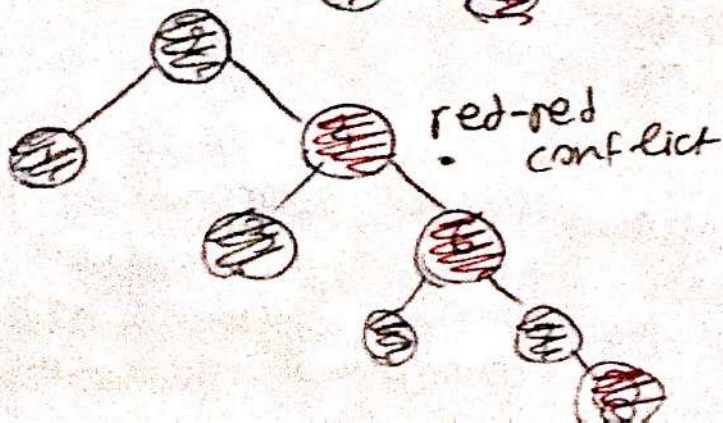
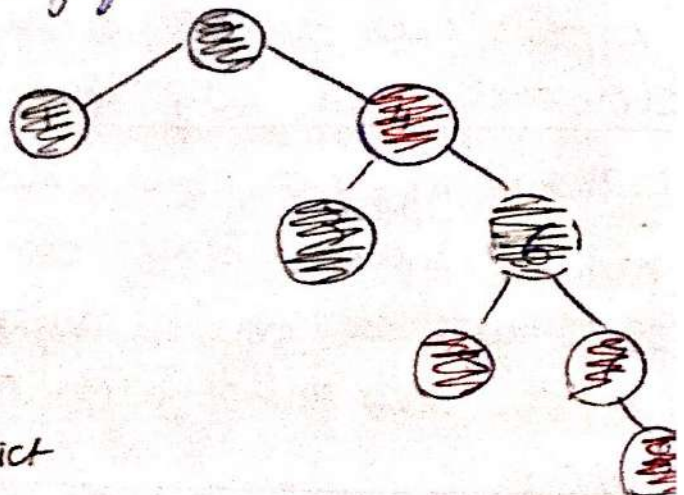
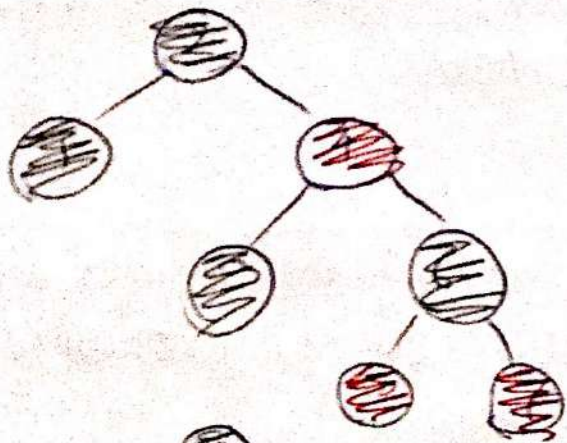
Εισαγωγή 6

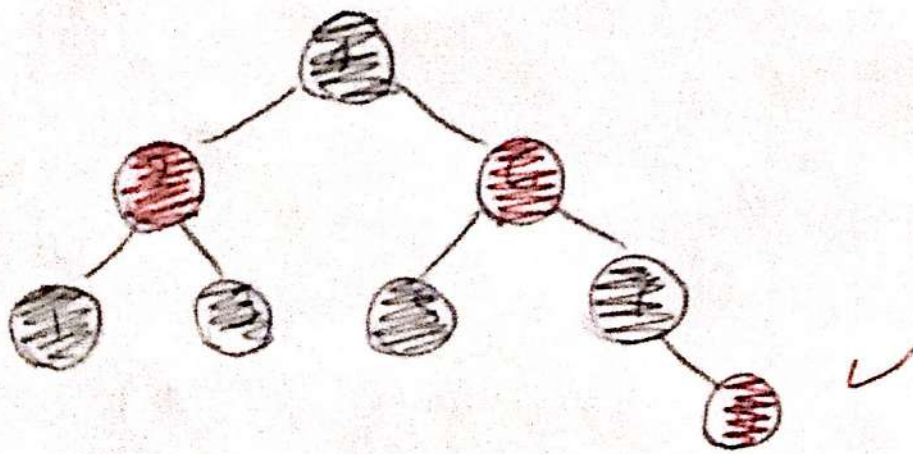


Εισαγωγή 7



Εισαγωγή 8





Κανόνες Εισαγωγής

σταματάω double
red

1 ^η η ρίζα είναι κόκκινη	την κάνουμε μαύρη
2 ^η ο θείος είναι κόκκινος, ο γονέας είναι το αριστερό παιδί του παππού	μαύρος, θείος γίνεται μαύρος, ο παππούς κόκκινος current = G
3 ^η ο θείος είναι κόκκινος, ο γονέας είναι το δεξί παιδί του παππού	current = G μαύρος, θείος γίνεται μαύρος, ο παππούς κόκκινος
(left-left) 4 ^η ο θείος είναι μαύρος, ο γονέας είναι το αριστερό παιδί του παππού, ο καυκάσιος κόμβος που εισάγεται είναι το αριστερό παιδί του γονέα	γονέας μαύρος, παππούς κόκκινος, δεξιά περιστροφή στον παππού
(right-right) 5 ^η ο θείος είναι μαύρος, ο γονέας είναι το δεξί παιδί του παππού, ο καυκάσιος κόμβος είναι το δεξί παιδί του γονέα	γονέας μαύρος, παππούς κόκκινος, αριστερή περιστροφή στον παππού
6 ^η (left-right) ο θείος είναι μαύρος, ο γονέας είναι το αριστερό παιδί του παππού, ο καυκάσιος κόμβος είναι το δεξί παιδί του γονέα	αριστερή περιστροφή στον γονέα, ο καυκάσιος γίνεται μαύρος, ο παππούς κόκκινος, δεξιά περιστροφή στον παππού

(right-left) *rw*

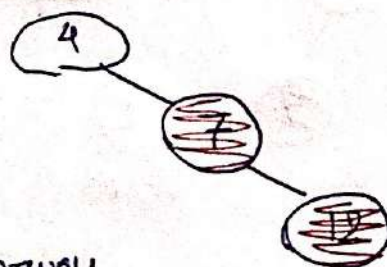
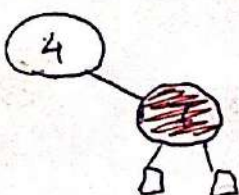
ο θένος είναι μαύρος, ο γένεας
είναι το δεξί παιδί του παππού,
ο καυούργος κόρη είναι το
αριστερό παιδί του γένεα

δεξιά περιστροφή του
γενεα, ο καυούργος κόρη
γίνεται μαύρος, ο παππούς
κόκκινος, αριστερή περιστροφή
στον παππού

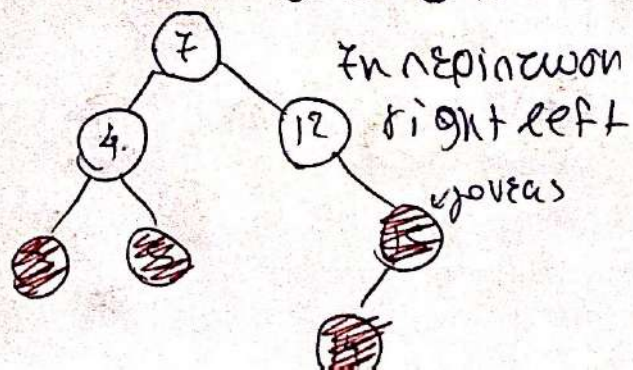
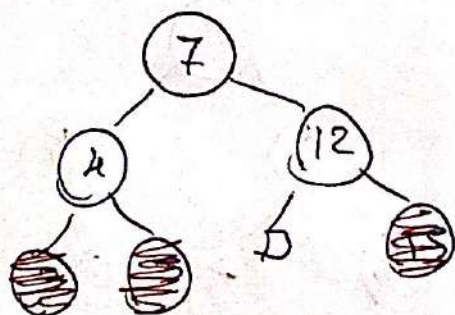
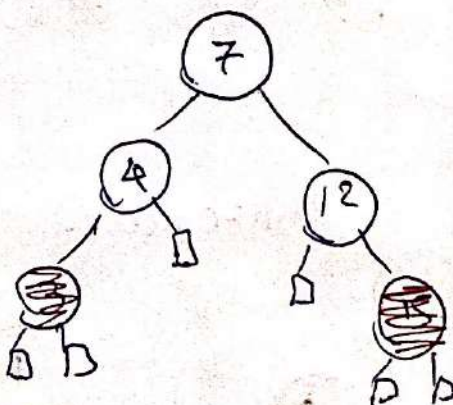
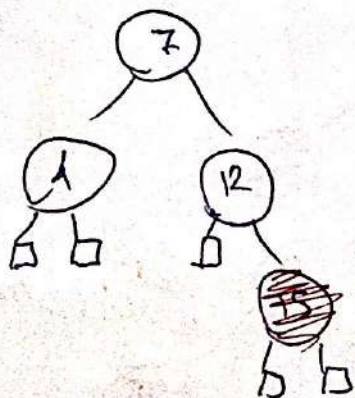
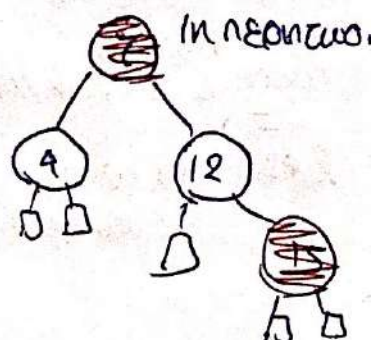
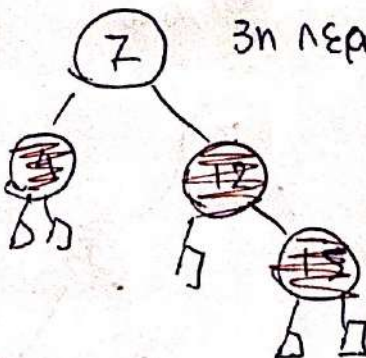
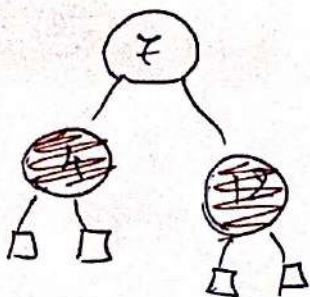
παράδειγμα διαφανειών

Εστω ότι εισαγούμε τα: 4, 7, 12, 15, 3, 5, 14, 12, 13, 16, 17
into a redblack tree πούκινται;

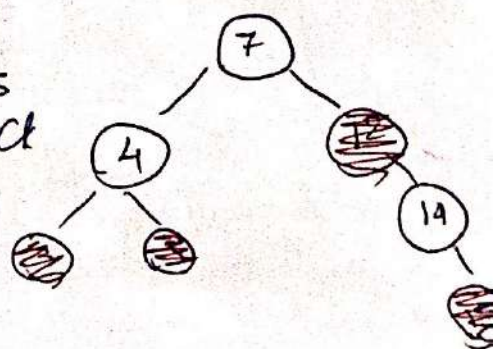
1η περίπτωση



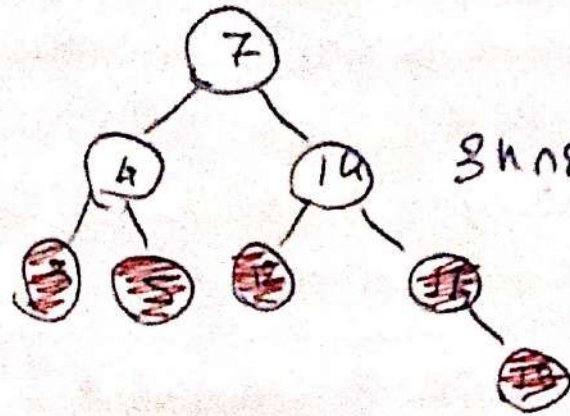
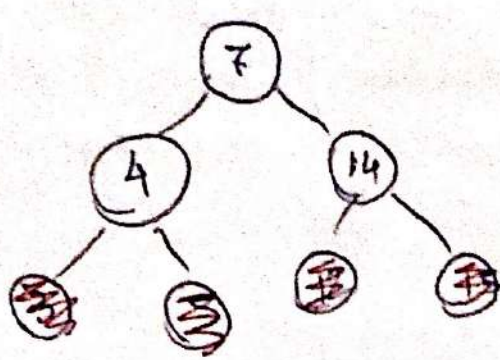
3η περίπτωση



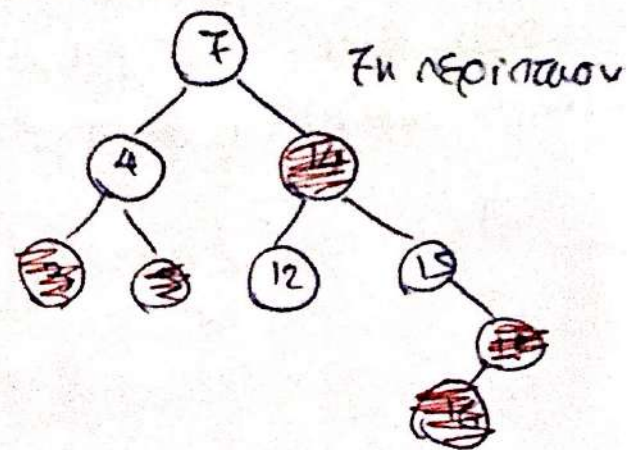
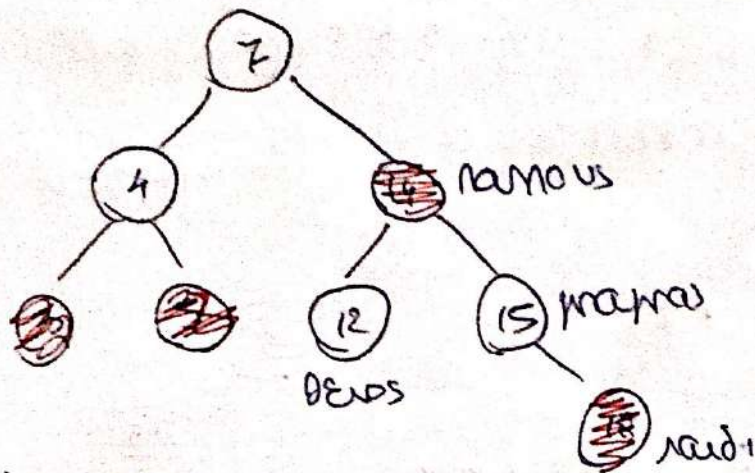
right rot is
14 → black
=>
narrow → red



αριστερή στενή στο νήμα

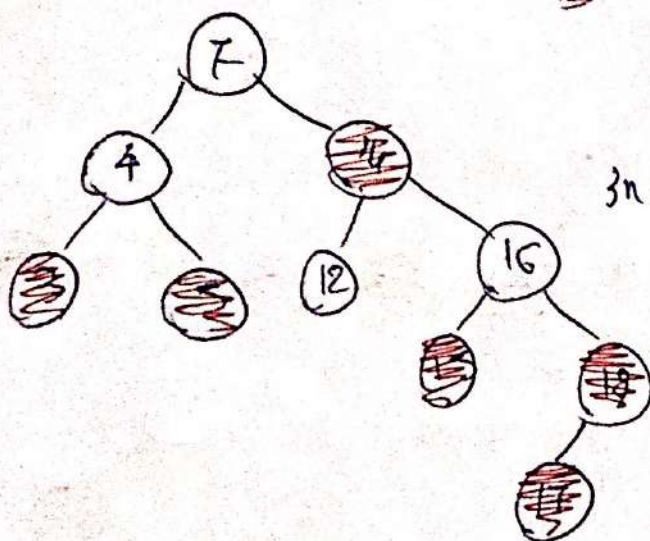
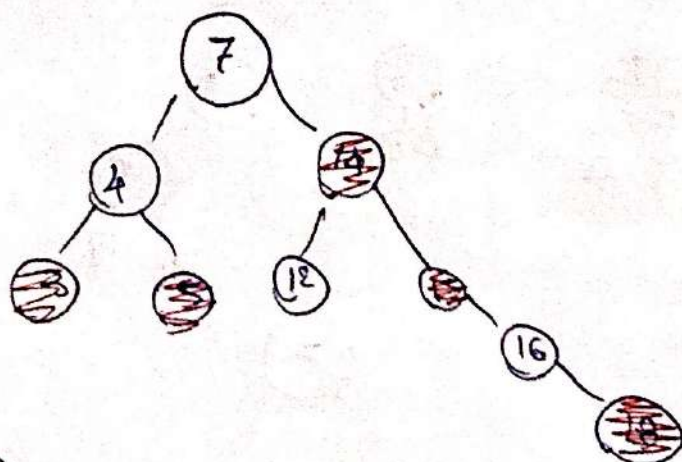


3η περίπτωση

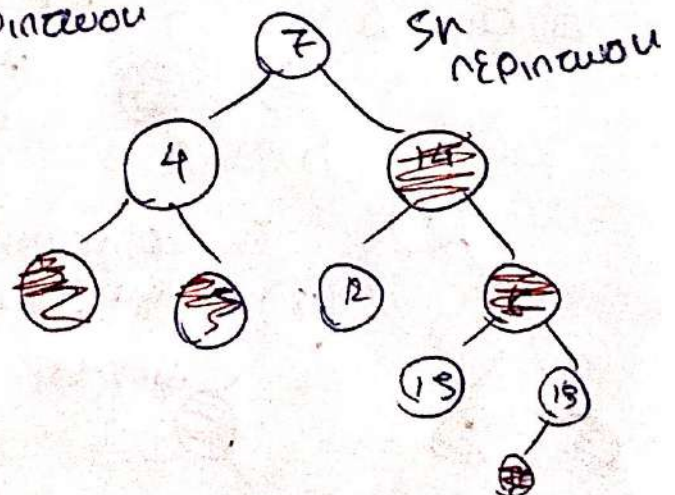


7η περίπτωση

right rotate 18
16 → black
narrow 15 → μαρμα
⇒
left rotate 15
(narrow)

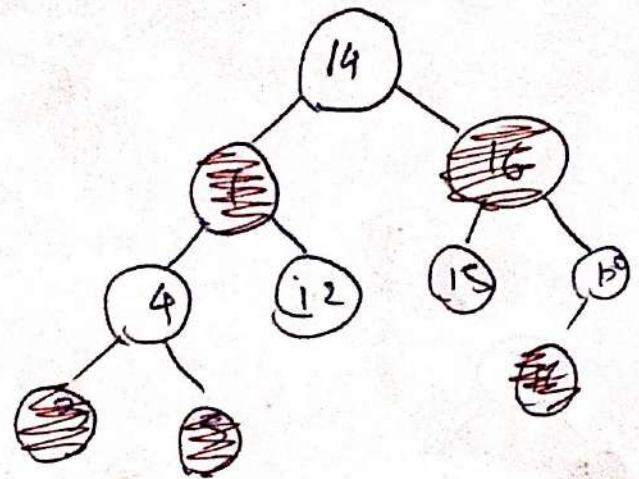
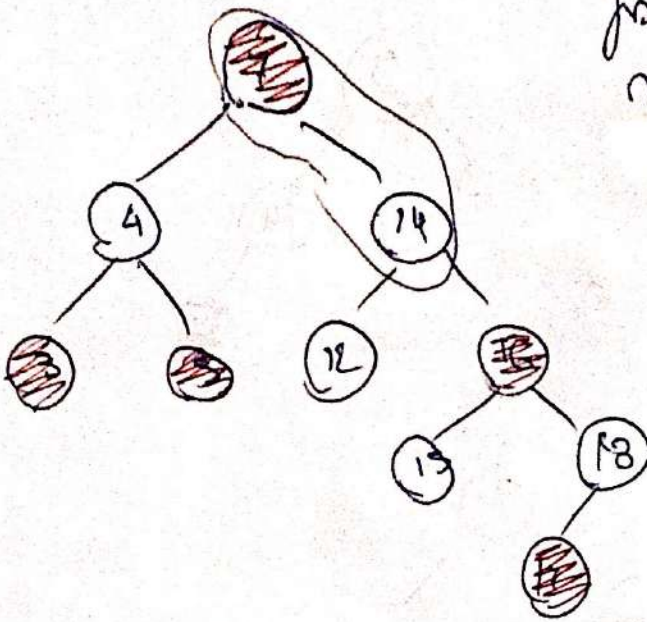


3η περίπτωση



5η περίπτωση

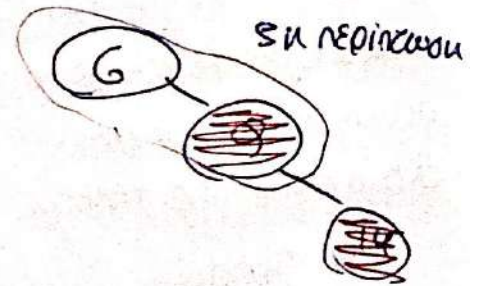
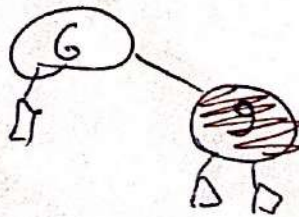
γινεας 14 γαυρος
 γαυρος (7) γαυρος
 left rot στον γαυρο



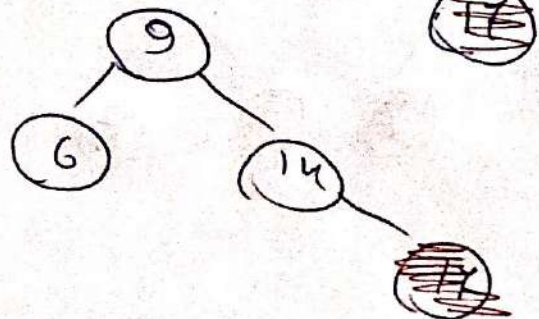
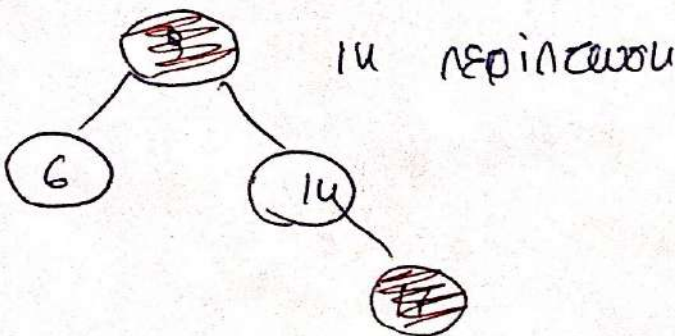
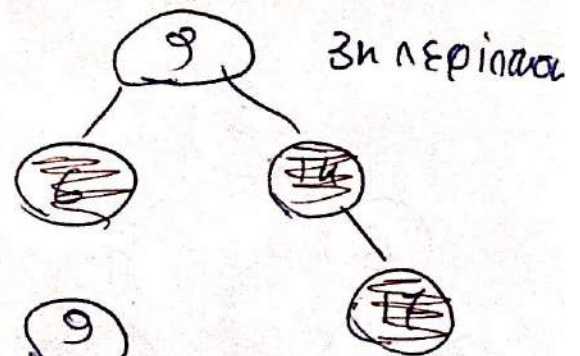
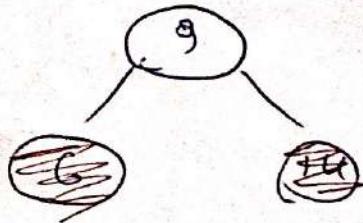
παράδειγμα θεμα.

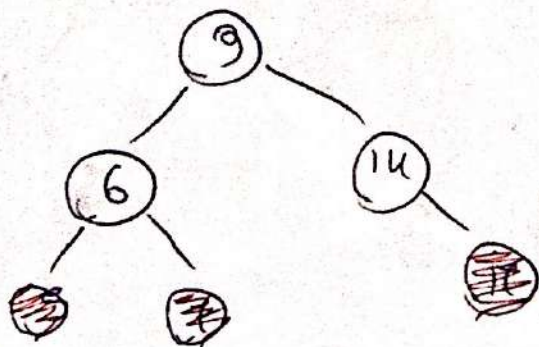
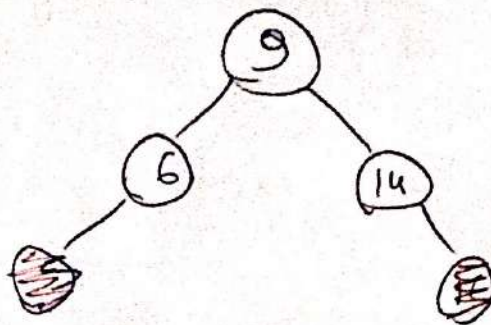
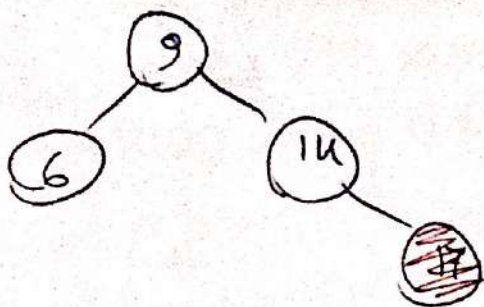
Αν εισαγουμε τα 6, 9, 14, 17, 5, 7, 16 σε ενα κενο
 δεντρο red black, ποιο red-black tree θα προκυψει

1

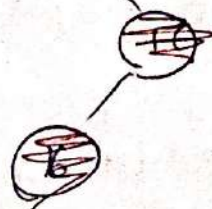


γινεας γαυρος
 γαυρος γαυρος
 αριστερη γερ
 στον γαυρο

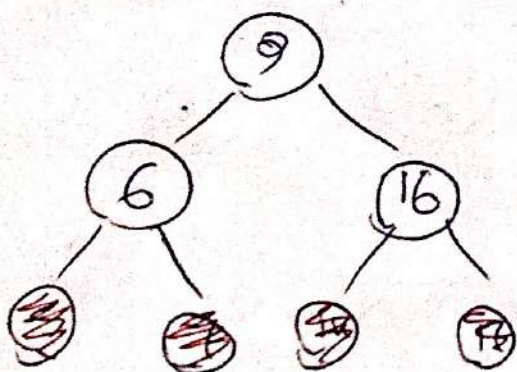
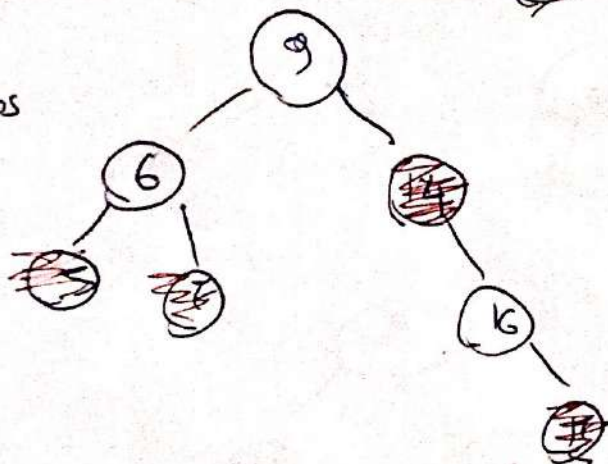




Εκ νέου περιστροφή



right rot 16 (γονέα)
 καυορριος(16) → παυρος
 παυρος → κικκρος
 αριστερὴ περιστροφή
 στὴν αριστερὴ



Διαγραφή από red-black

αλυσή / double black x

Τι συμβαίνει	Τα κάνουμε	γιατί το κάνουμε
1. ο κόμβος που διαγράφουμε είναι κόκκινος	αλλά τον αφαιρούμε	Δεν θα γίνει η ισορροπία του δέντρου
2. Ο κόμβος είναι μαύρος, αλλά ο αδελφός του κόκκινος	χρωματίζουμε τον αδελφό μαύρο	για να μη αλλάξει το συνολικό "βάθος μαυρίδας"
3. και ο κόμβος και ο αδελφός είναι μαύροι	<u>double black</u>	
3.1 Ο αδελφός είναι μαύρος και έχει κι κ-μνο παιδί	αναδιοργανώση με 3 κόμβους (πίσω σε αλυσή)	ισορροπούμε τα βάθη και στρώμα
3.2 Ο αδελφός είναι μαύρος και έχει μαύρα παιδιά	τον κάνουμε κι κόκκινο και αν εξακολουθεί το πρόβλημα στον γονιό.	Αν ο γονιός η ται κόκκινος → τελος Αλλιώς, συνεχίζουμε προς τα πάνω
3.3 αδελφός είναι κόκκινος	Ανταλλάσσουμε θέσεις με τον γονιό και μετά το δέντρο μπαίνει σε περίπτωση 1 ή 2 right rotation + recoloring	φέρνουμε σε πρόβλημα σε κατάσταση που ξέρουμε να λύσουμε

Trinode restructuring

Μια μετακίνηση τριών κόμβων ώστε να ισορροπηθεί ξανά το δέντρο.

Παίρνουμε 3 κόμβους:

ήχ έναν πατέρα, έναν θείο και έναν ανιψιό (παιδί αδελφού), τους βάζουμε σε σωστή σειρά με βάση το μέγεθος των αριθμών.

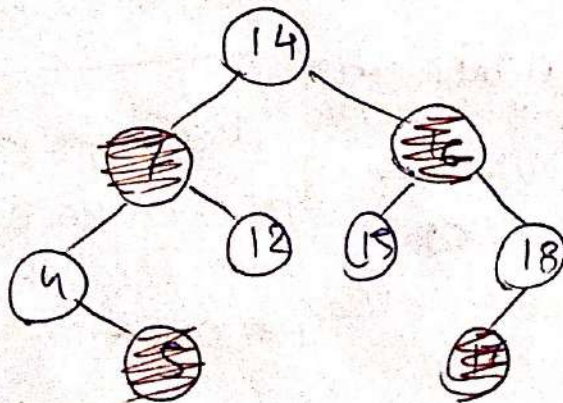
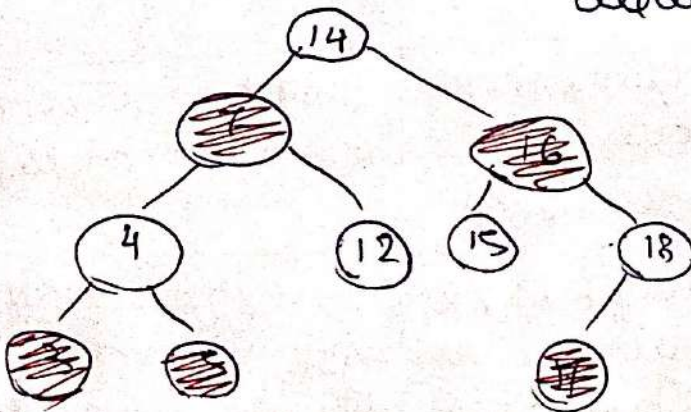
Ο μεσαίος κόμβος ανεβαίνει και γίνεται γονιός.

Οι άλλοι δύο γίνονται παιδιά του.

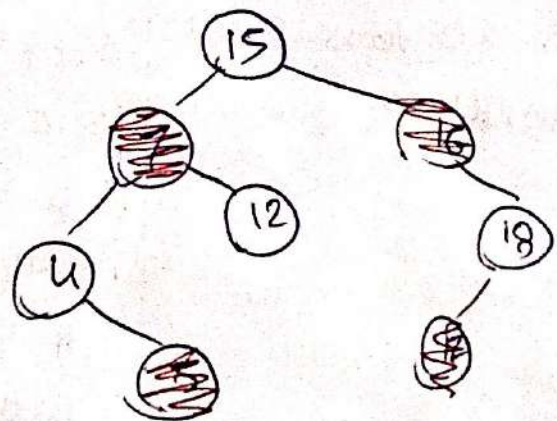
Τους χρωματίζουμε ξανά ώστε να ισορροπηθεί το δέντρο.

Παράδειγμα διαφανειών

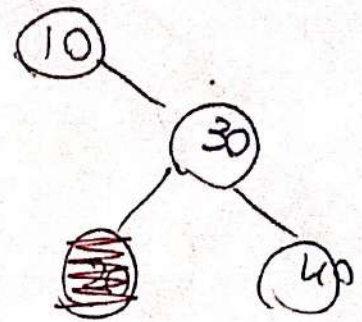
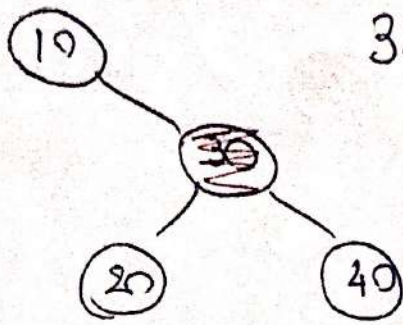
αφαίρεση 3 - 1η περίπτωση



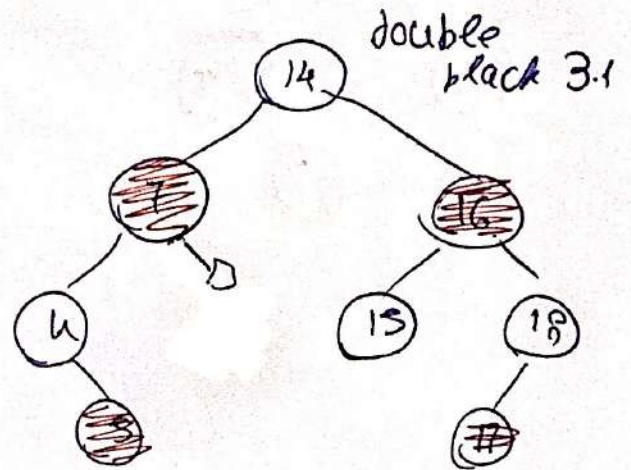
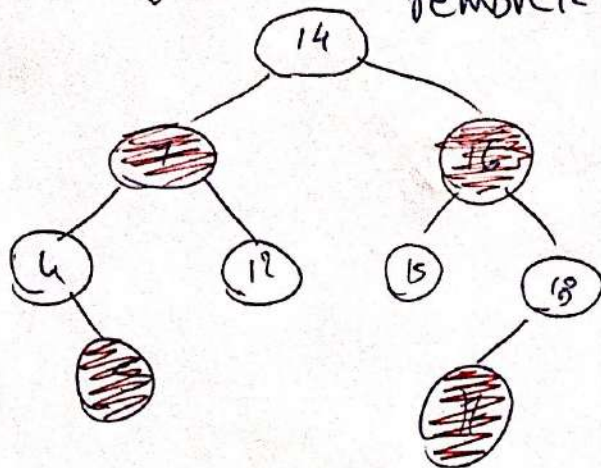
14 όπως σε BST
αφ' αφαίρεση



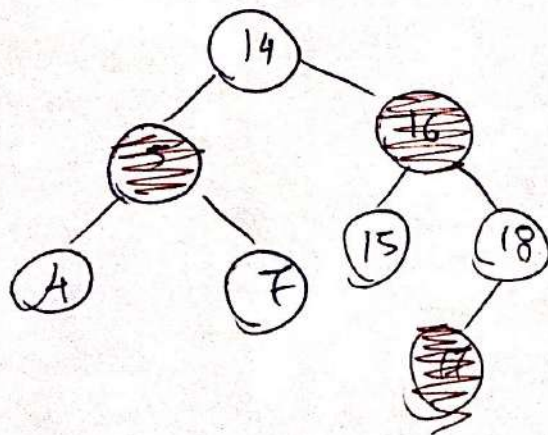
fixing double black problem



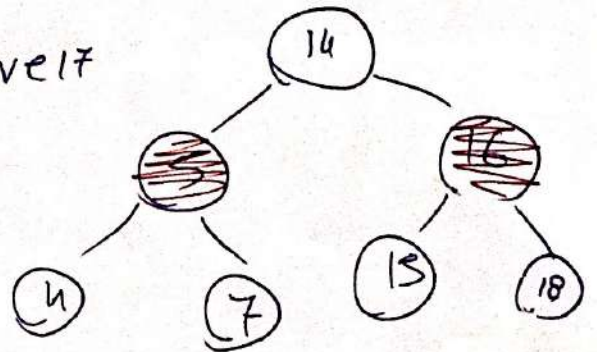
παράδειγμα διαγραφών
remove 12



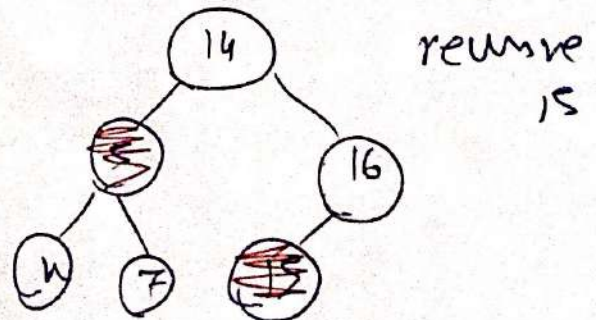
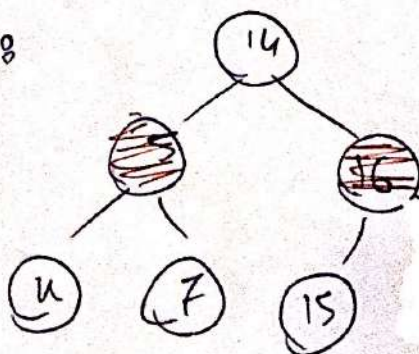
$4 < 5 < 7$



remove 17
case 1

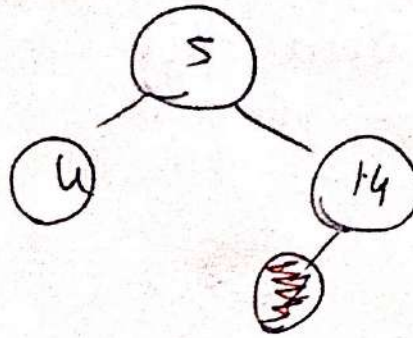
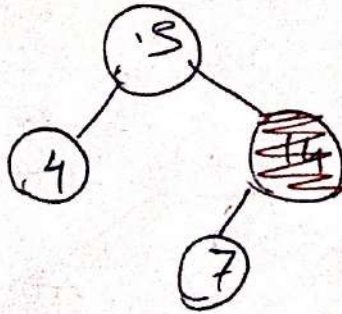
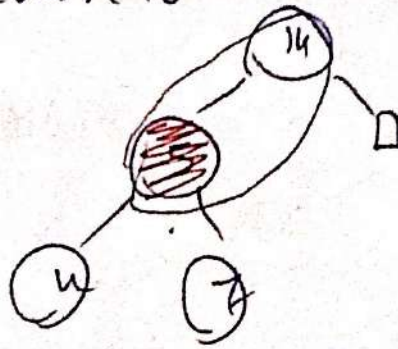
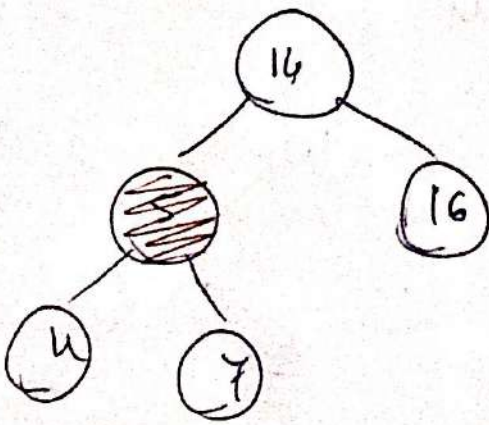


remove 18
case 3.2



remove 16

case 3.3



ΑΝΟ (2,4) red/black

Υπάρχει ένα μοναδικό δέντρο (2,4) που αντιστοιχεί σε red/black tree.

Περίπτωση 1: Ο κόμβος είναι **2-κόμβος** περιέχει 1 κλειδί και 2 παιδιά

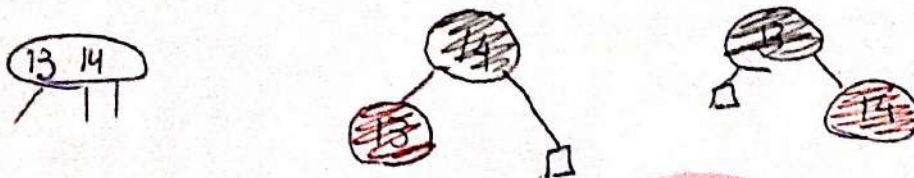


Καμία αλλαγή, κρατάμε τον κόμβο παλιό (τα παιδιά του Δ παραμένουν όπως είναι (μαύρα))



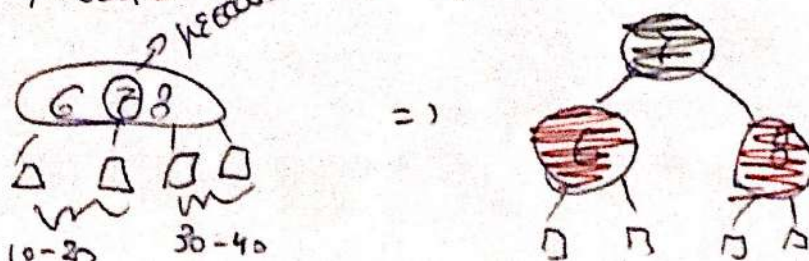
Περίπτωση 2: Ο κόμβος είναι **3-κόμβος**. Περιέχει 2 κλειδιά και 3 παιδιά.

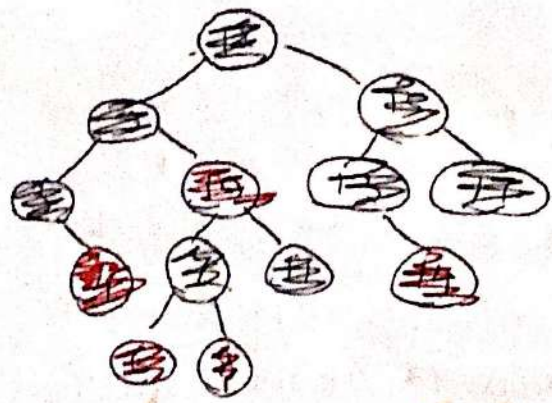
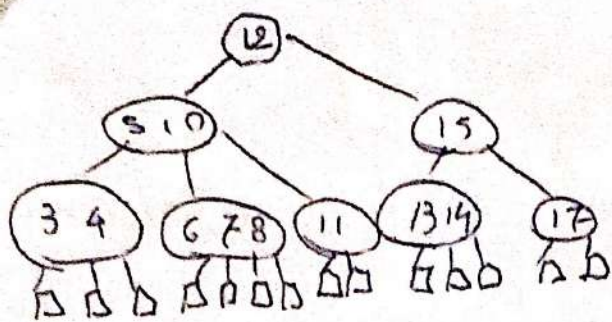
Δημιουργούμε ένα νέο κόκκινο κόμβο w , του δίνουμε τα 2 πρώτα παιδιά του 3-κόμβου. Ο αρχικός κόμβος v κρατά: το μεσαίο κλειδί, το w (κόκκινος κόμβος) ως αριστερό παιδί, το τρίτο παιδί του 3ου κόμβου w ως δεξί παιδί



Περίπτωση 3: Ο κόμβος είναι **4-κόμβος** 3 κλειδιά + 4 παιδιά

Δημιουργούμε δύο νέους κόκκινους κόμβους w και z . Ο w παίρνει τα 2 πρώτα παιδιά (1-2) Ο z παίρνει τα 2 τελευταία παιδιά (3-4). Ο αρχικός κόμβος v κρατά το μεσαίο κλειδί, έχει 2 παιδιά τον w και z





Από red black σε (2,4)

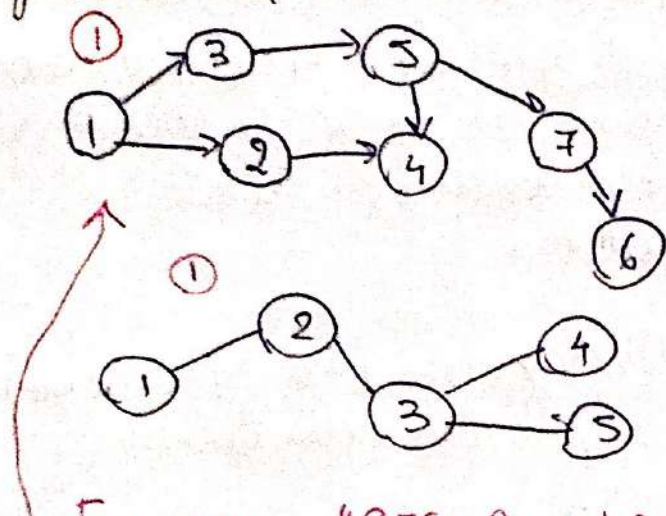
Δεδομένου ενός δέντρου κόκκινο-μαύρο μπορούμε να κατασκευάσουμε το αντίστοιχο (2,4) :

⇒ Συγχώνευση κόκκινου κόμβου : Κάθε κόκκινος κόμβος v συγχωνεύεται με τον γονιό του. Το κλειδί που περιέχεται στο v αποθηκεύεται στην δεσφονική κόμβο δημιουργώντας έτσι έναν κόμβο -3 ή κόμβο-4 ανάλογα με τον αριθμό των ενώσεων.

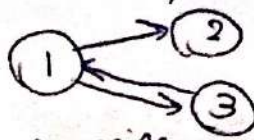
Ανακατανομή χρωμάτων : Τα δύο παιδιά του v γίνονται το αριστερό και το δεξί παιδί του v στον νέο κόμβο.

Γράφοι

Συλλογές από κόμβους στις οποίες διάφορα ζεύγη συνδέονται με ευθύγραμμα τμήματα. Οι κόμβοι ονομάζονται κορυφές και τα ευθύγραμμα τμήματα ακμές



κατευθυνόμενος (με $\rightarrow$)



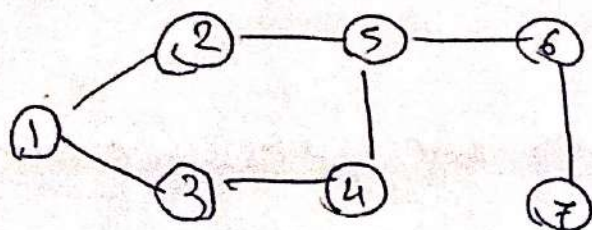
1: προέλευση
2: προορισμός

μη κατευθυνόμενος

Για τον κατευθυνόμενο γραφο 1.

→ Οι κορυφές 1 και 3 είναι γειτονικές

→ Η ακμή (1,3) είναι προσήγουσα από την κορυφή 1 και είναι προσήγουσα στην κορυφή 3. (1,2,5,6)



Η ακολουθία κορυφών είναι ένα **αλγό** μήκους 3.

Ένα μονοπάτι είναι μια ακολουθία γειτονικών κορυφών. Ένα μονοπάτι είναι **αλγό**, αν δεν επισκεψάμαστε 2 ή περισσότερες φορές - κυκλικό: αριθμολογία

Το (1,2,5,6,5) δεν είναι αλγό

Το (2,5,6) είναι υπομονοπάτι του (1,2,5,6,7) αλγό κυκλικό (1,2,3,1)

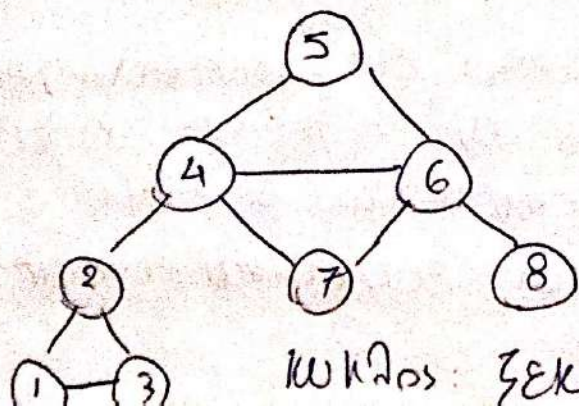
(4,5,6,7,4)

(4,5,6,4)

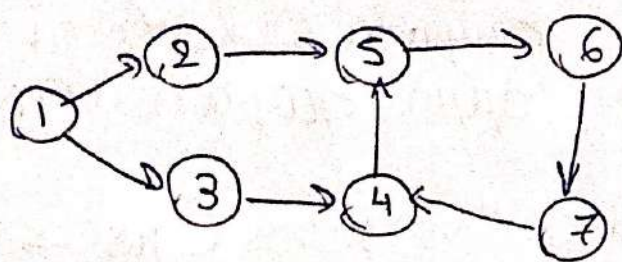
(4,6,7,4)

όχι αλγό κυκλικό (4,5,6,4,7,6,4)

κυκλός: ξεκινάμε από έναν κόμβο και καταλή-

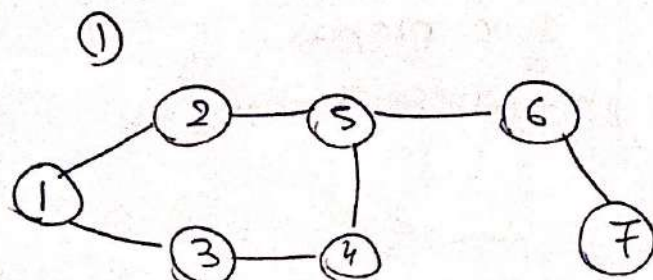


γύρισε σε αυτές διατηρώντας τον
αριθμό κόμβων: δεν απαγορεύονται κορυφές.



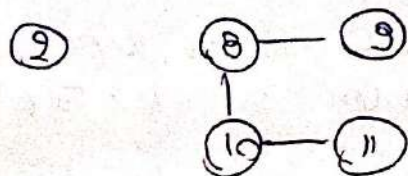
(1, 2, 5, 6) από κατευθυνόμενο μονοπάτι μήκους 3.
(2, 5, 4) δεν είναι μονοπάτι γιατί δεν υπάρχει κατευθυνόμενο ακμή (5, 4).

(5, 6, 7, 4, 5) → αριθμός κύκλος



①, ② κορυφές 1-6 προσβάσιμες ή μα από την αλλη

ή κορυφή 1 δεν είναι προσβάσιμη από την κορυφή 8 (ή 9 ή 10 ή 11)



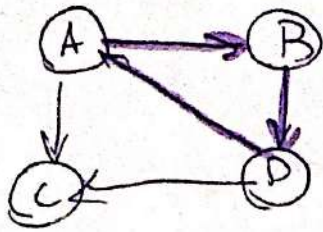
Συνεκτικός γραφός: κάθε κορυφή είναι προσβάσιμη από κάθε άλλη

Συνεκτική συνιστώσα: μέγιστη υποσυνολό κορυφών που είναι όλες προσβάσιμες ή μα από την άλλη

↳ Ένας undirected graph είναι συνεκτικός αν και μόνο αν έχει ακριώς μία συνεκτική συνιστώσα.

Ισχυρά συνεκτικός γραφός: ένας κατευθυνόμενος γραφός είναι ισχυρά συνεκτικός αν κάθε δύο κορυφές είναι προσβάσιμες ή μα από την άλλη (δηλαδή υπάρχει μονοπάτι από την 1η στην δεύτερη και αντίστροφα)

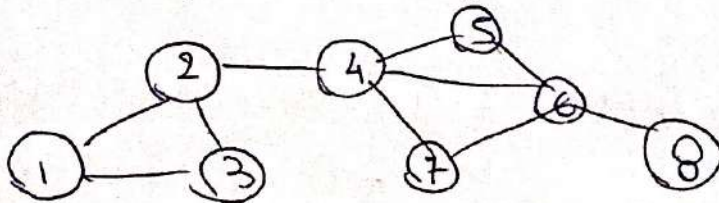
Ισχυρά συνεκτική συνιστώσα: ενός κατευθ. υφένου γραφού είναι ένα μέγιστο σύνολο κορυφών στο οποίο υπάρχει μονοπάτι από μια κορυφή σε μια άλλη



Ισχυρά συνεκτική συνιστώσα

Βαθμός σε μη κατευθυνόμενο γραφο: ο αριθμός των ακμών στις οποίες ο x είναι ένα από τα άκρα της ακμής. $\deg(x)$

πχ 3 ακμές $\rightarrow \deg(x) = 3$



$$\deg(1) = 2$$

$$\deg(4) = 5$$

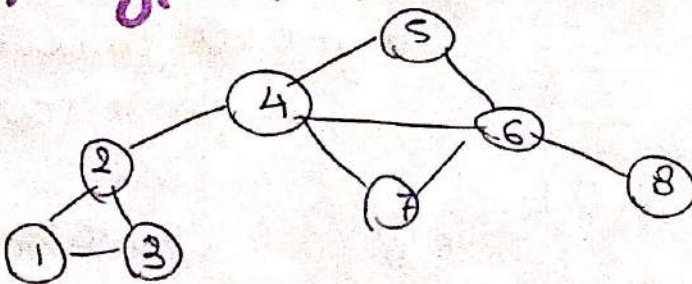
$$\deg(8) = 2$$

Γενικά: $\sum_{v \in G} \deg(v) = 2m \leftarrow \text{ακμές}$

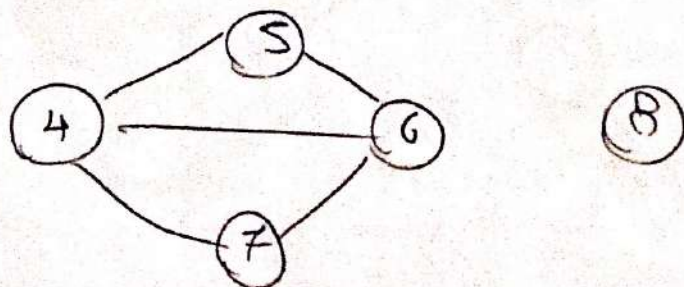
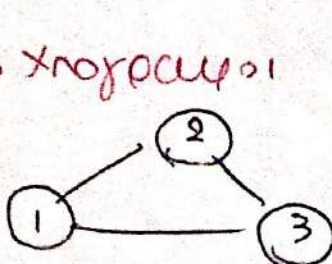
Προηγούμενοι και Επόμενοι κομβοί σε κατευθ. γραφο

$$\sum \text{indeg}(v) = \sum \text{outdeg}(v) = m$$

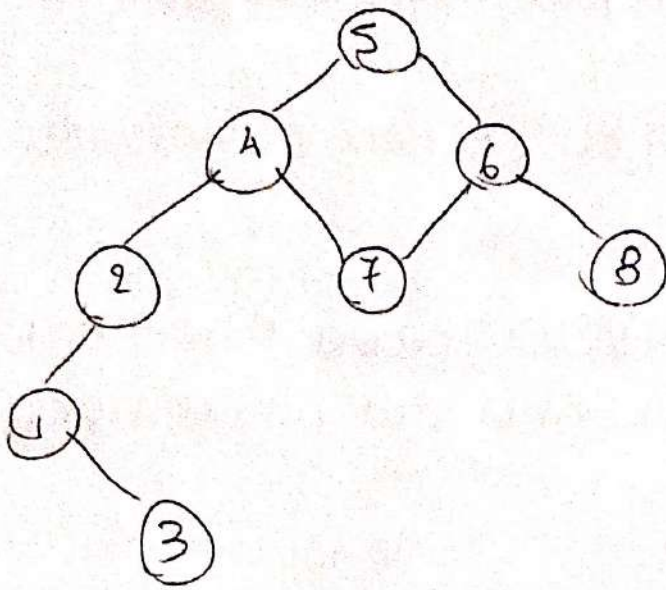
Υπογραφοί



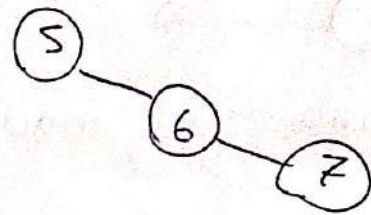
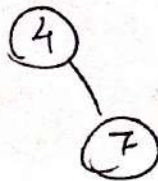
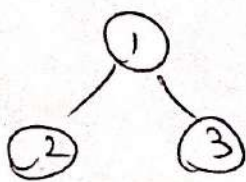
Ενας γραφος του οποίου οι κομβοί και οι ακμές είναι υποσύνολα του αρχικού κομβοί



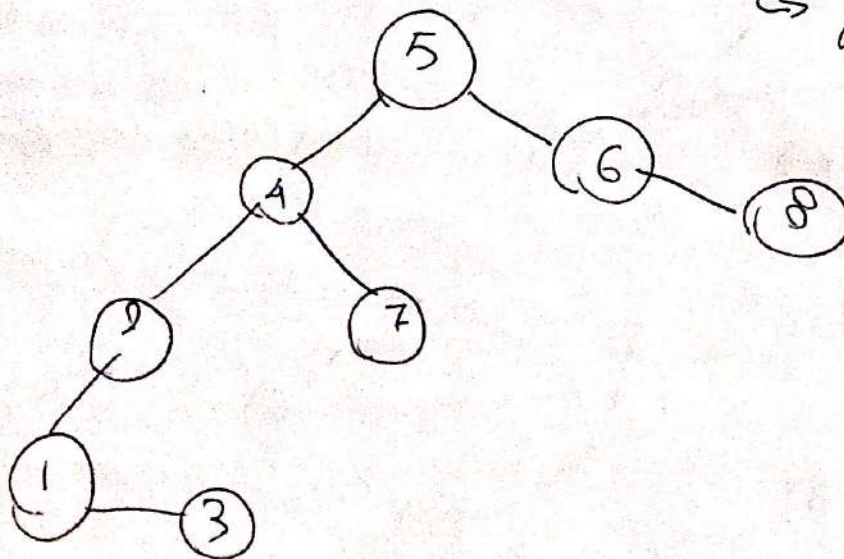
spanning subgraph γράφος που περιλαμβάνει όλες τις κορυφές



Δασος: μη κατευθυνόμενος γράφος που δεν έχει κύκλους



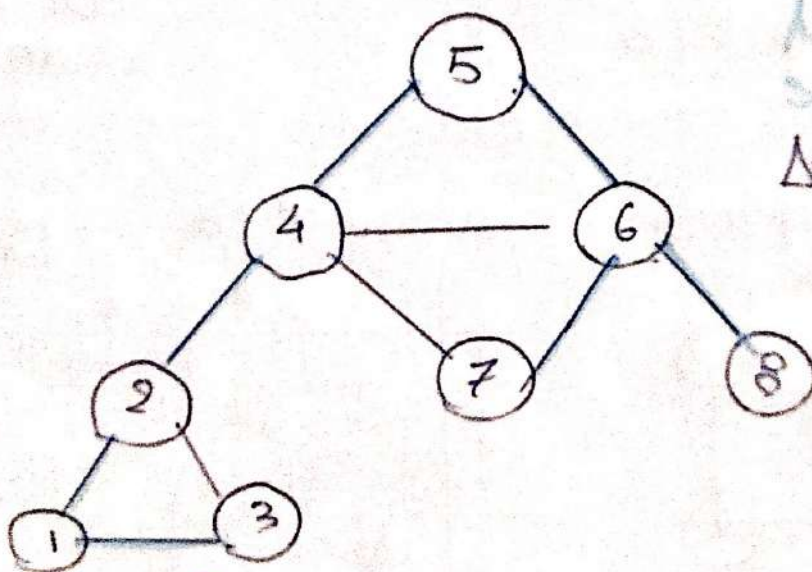
Ελεύθερο δέντρο: Συνεκτικό δασος (ένας μη κατ. γράφος όπου κάθε κορυφή είναι προσβάσιμη από μια αλληλ. και δεν έχει κύκλους)



→ για να ισχύει η ιδιότητα του δέντρου.

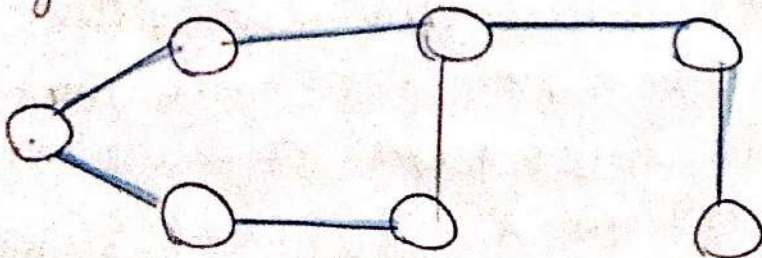
Spanning tree: Ένας μη κατευθυνόμενος γράφος είναι ένας spanning subgraph που είναι ελεύθερο δέντρο. Δηλαδή ένας υπογράφος που περιλαμβάνει όλες τις κορυφές του αρχικού γράφου, κάθε κορυφή είναι προσβάσιμη από μια αλλη και δεν σχηματίζονται κύκλοι.

Εστω ότι έχουμε τον γράφο της προηγούμενης σελίδας:

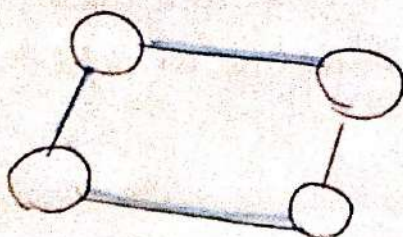


Με — το spanning tree
Δεν είναι μοναδικό

Spanning forest: Ένας μη κατευθυνόμενος γράφος που είναι η ένωση δέντρων συσχετισμένης, ένα για κάθε συνεκτική συνιστώσα του γράφου.

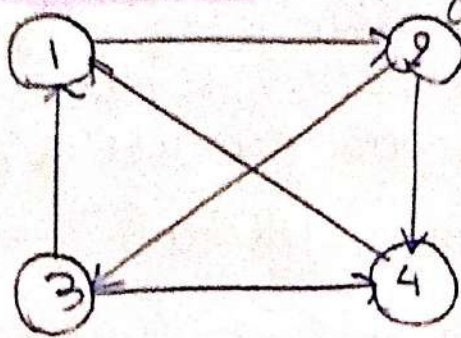


Οι δυο παραπάνω όροι μόνο για μη κατευθυνόμενους γράφους



ΠΙΝΑΚΕΣ ΓΕΙΤΝΙΑΣΗΣ

Αναπαράσταση γραφών
κατευθυνόμενος



2ος τρόπος
αποδοτικοί
όροι

	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	1	0	0	1
4	1	0	0	0

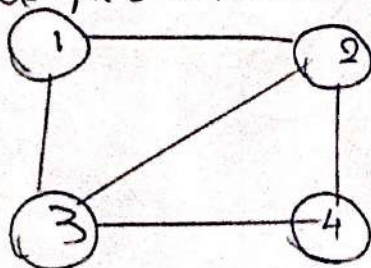
ή κ ή
όδοι οι
κορβές

1: Αν υπάρχει ακμή από το i στο j
0: Αν δεν υπάρχει

δεν είναι
απαραίτητα

0: σε όλα την κύρια διαγώνιο ($i=j$) συμμετρικός

μη κατευθυνόμενος
αμερόμετρος οαφεί



συμμετρικός

(0,1,2)

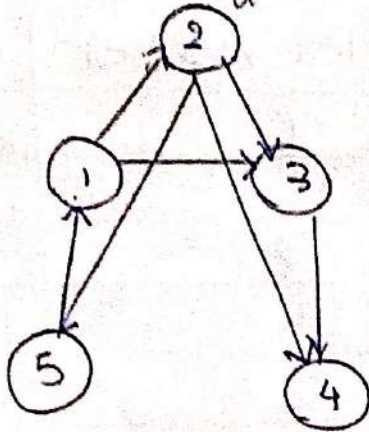
	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1
4	0	1	1	0

ΣΥΝΟΛΑ ΓΕΙΤΝΙΑΣΗΣ

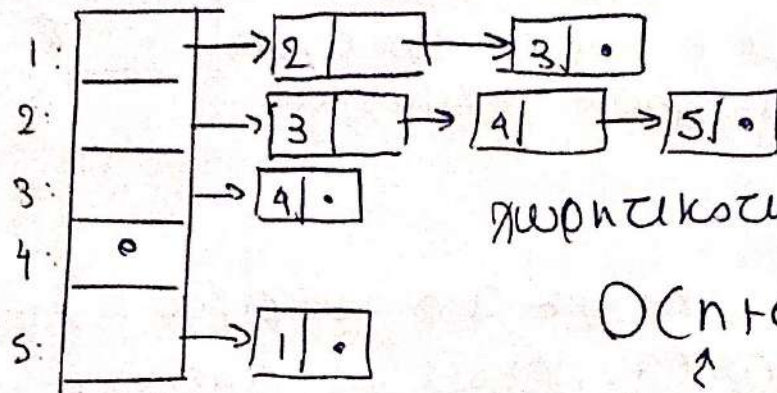
Έστω V_x το σύνολο όλων των κορυφών που είναι γειτονικές με τον κόμβο x σε έναν μη κατευθυνόμενο γραφο G . Αν παρείχαμε το σύνολο κορυφών V και τη συλλογή $A = \{V_x | x \in V\}$ των συνόλων γειτνιάσεως με κάθε κορυφή στο V , τότε έχουμε παρείχει επαρκείς πληροφορίες για να ορίσουμε τον γραφο G .

ΛΙΣΤΕΣ ΠΕΡΙΤΗΛΙΑΣΜΟΣ

κατευθυνόμενος γραφός



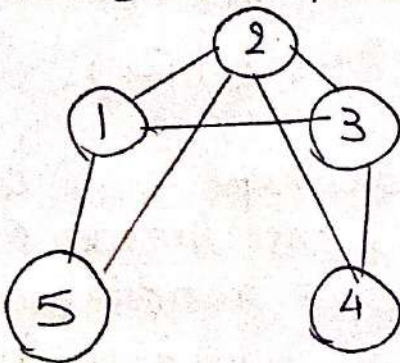
vertex num.	out degree	Adjacency list
1	2	2 3
2	3	3 4 5
3	1	4
4	0	
5	1	1



χωρητικότητα χώρου

$O(n \cdot e)$
 $\uparrow$ $\uparrow$
 κόμβοι ακμές

μη κατευθυνόμενος γραφός

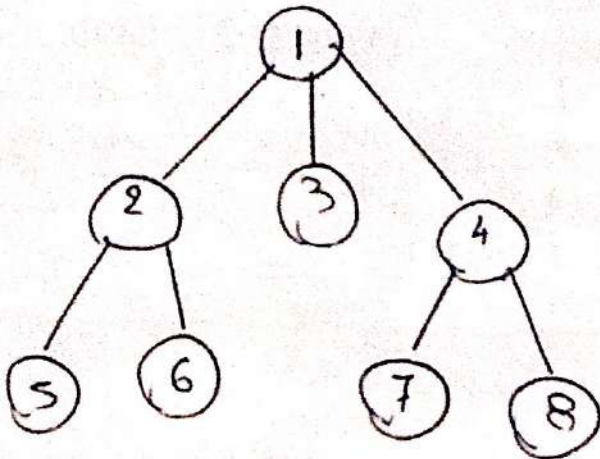


vertex num	out deg	Adj
1	3	2 3 5
2	4	1 3 4 5
3	3	1 2 4
4	2	2 3
5	2	1 2

DFS

κατα βάθος διασπορά

preorder



Ποια είναι η σειρά των κόμβων που επισκεπτόμαστε ξεκινώντας από το 1.

Με στοίβα:

Βήμα 1: Ξεκινάμε από την ρίζα 1. Αφού η υλομίση είναι με στοίβα πρέπει να πάρουμε τα στοιχεία από τον κόμβο 1 που αντιστοιχούν στην λίστα γειτνίασης και να τα βάλουμε ανανδα (για LIFO).

Οπότε έχουμε:

vertex number	Adj list
1	4 3 2
2	6 5 1
3	1
4	8 7 1
5	2
6	2
7	4
8	4

Βήμα 2 Ξεκινάμε από το 1ο στοιχείο της λίστας (4) και γράφουμε τα στοιχεία που αντιστοιχούν στη λίστα γειτνίασης του κόμβου 4 μέχρι να βρούμε στοιχεία που έχουμε επισκεφτεί ήδη να μην βρω (backtracking).

1, 4, 8, 7, 3, 2, 6, 5

χρονική πολυπλοκότητα: O(n²)

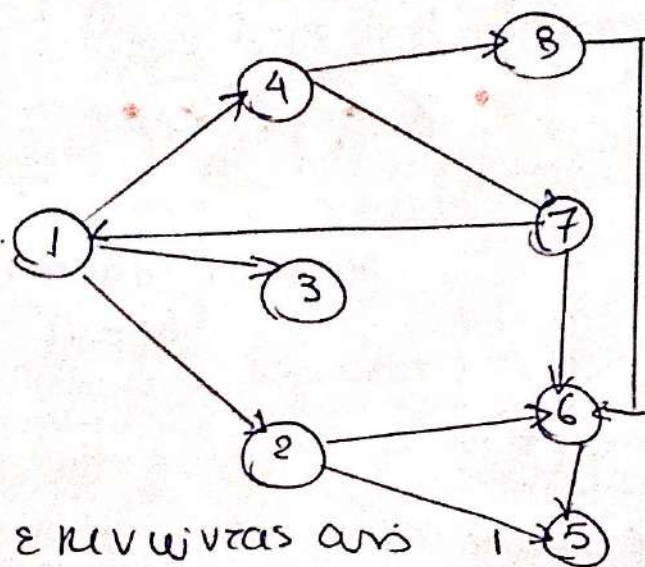
Αν ο πίνακας είναι απλός (adj) αλλιώς τότε η υλομίση με στοίβα είναι πιο αργή από λίστα γειτνίασης

Αναδρομικά

Ποια είναι η σειρά των κορυφών που επισκεπτόμαστε ξεκινώντας από 1

1, 2, 5, 6, 3, 4, 7, 8 (Διαφορετικά αν stack συνόδου ή χρονική ποσότητα O(n))

Άλλο παράδειγμα με στοίβα (backtracking)



= ξεκινώντας από 1

1 4 8 6 5 7 3 2

=> Εξαιρετική αλγόριθμος

=> 0 Θρόνος σαν λούβινος

vertexnum	Adjlist
1	2 3 4 → 4 3 2
2	5 6 → 6 5
3	.
4	7 8 → 8 7
5	.
6	5 → 5
7	1 3 6 → 6 3 1
8	6 → 6

ποσότητα

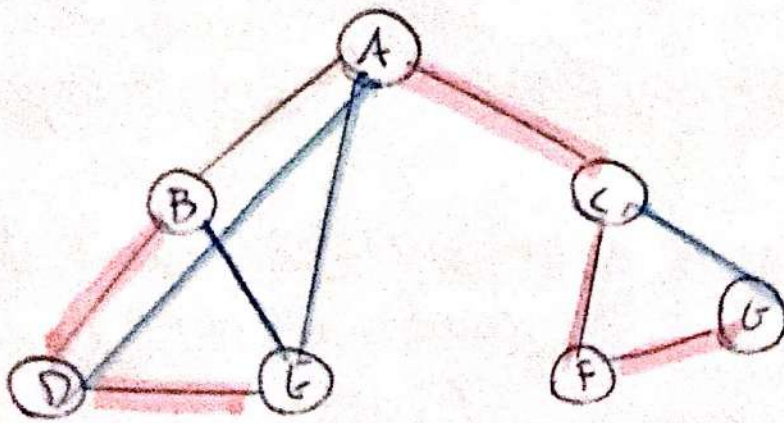
Με λίστα: O(n²)

Με πίνακα: O(n²)

χρονική: O(n)

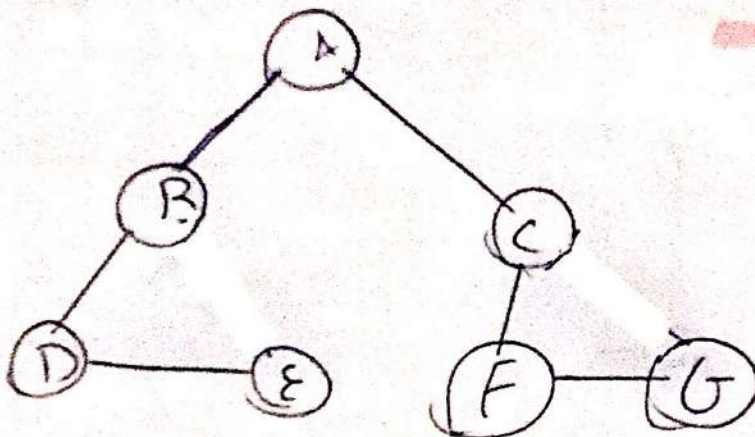
DFS σε μη κατ. γραφικό

- κατά την διαδρομή DFS traversal οι ακμές που οδηγούν σε νέες κορυφές που δεν έχουν επισκεφτεί ονομάζονται ακμές δέντρου ή ακμές ανακάλυψης
- Οι ακμές δέντρου σχηματίζουν ένα δάσος εξαρτήσεων
- Back edges: Οι γραμμές που δεν ανήκουν στο δέντρο εξαρτήσεων και τις έχουμε ήδη επισκεφτεί



vert	Adj list
A	B C D E
B	A D E
C	A F G
D	A B E
E	A B D
F	C G
G	F C

- back edges
- edge tree



DFS spanning forest

Μια διασχίση DFS μπορεί να:

1. ελέγξει αν ένας γραφός είναι συνεκτικός
2. εύρεση spanning tree, αν είναι συνεκτικός
3. Υπολογισμός μιας διαδρομής μεταξύ δυο κόμβων, αν υπάρχει.
4. Υπολογισμός ενός κύκλου στον G ή εύρεση αν ο G δεν έχει κύκλους

DFS σε κατευθυνόμενο γράφο
tree edges

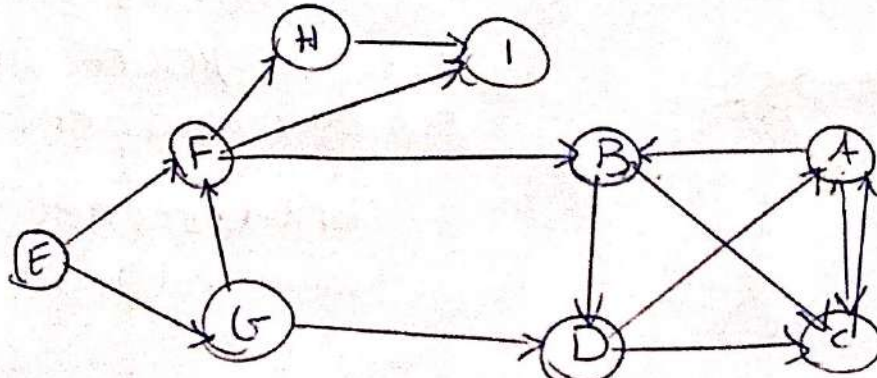
Οι ακμές που οδηγούν σε κόμβους που δεν έχουμε επισκεφτεί και ανήκουν στο spanning forest.

back edges: Ανηλθουν απμα κόμβου σε ένα από τους προηγούμενους της στο δεύτερο DFS

Cross edges: Όλες οι υπόλοιπες ακμές. Μπορούν να ανήκουν μεταξύ κόμβων που είναι στο δεύτερο DFS

αρκεί μια κορυφή να μην είναι πρόγονος της αϊτής ή μπορούν να ηγούνται μεταξύ κορυφών σε διαφορετικά δέντρα DFS.

Α 2ο παράδειγμα **αναδρομικά**

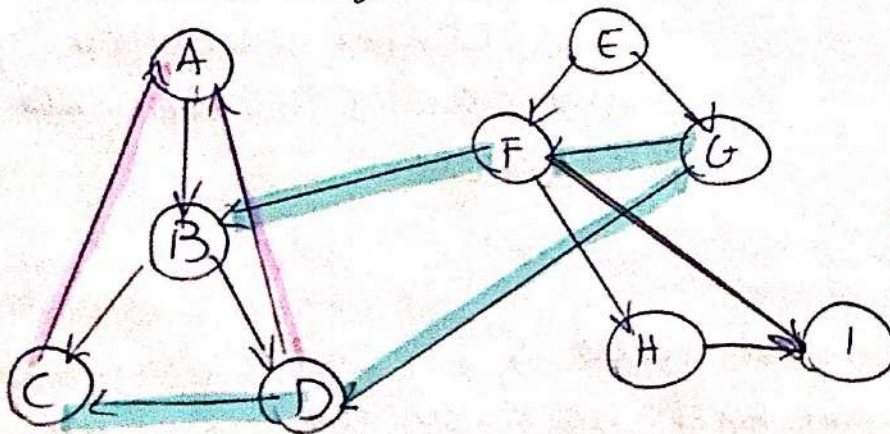


vertex	Adj list
A	B C
B	C D
C	A
D	A C
E	F G
F	H I
G	F
H	I
I	

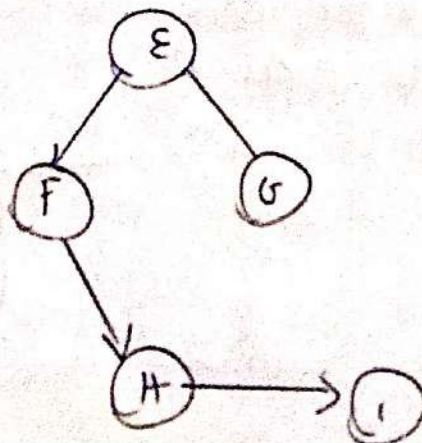
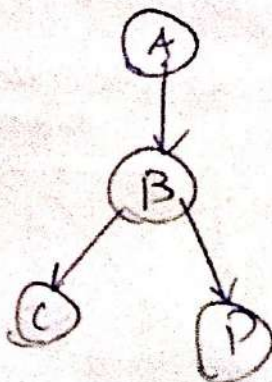
Ξεκινώντας από την κορυφή A

A B C D E F H I

Α 2ο παράδειγμα



cross edges
back edges
forward edges
tree edges

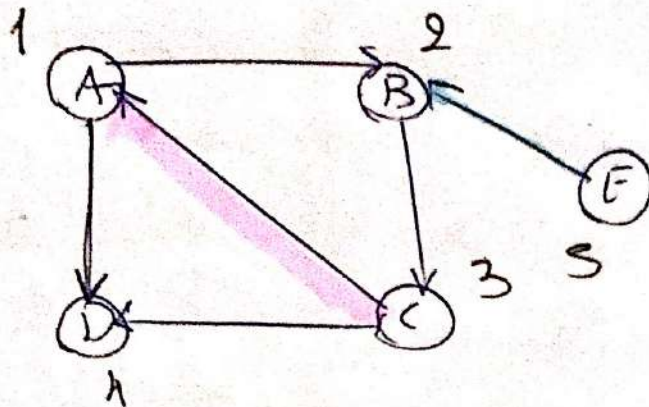


DFS forest
and των αναδρομικών γράφων

ΑΡΙΘΜΗΣΗ ΤΩΝ ΚΑΥΣΕΩΝ

=> preorder numbering
=> postorder numbering

Παράδειγμα Αναδρομής



Αριθμίζουμε τις κορυφές του γραφού με τη σειρά που τις επισκεπτόμαστε κατά τη διαδρομή του DFS

forward edges
tree edges

back edges
cross edges

v. n	A list
A	B D
B	C
C	A D
D	.
E	B

A B C D E

forward edges:

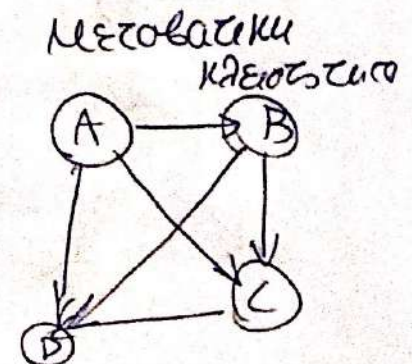
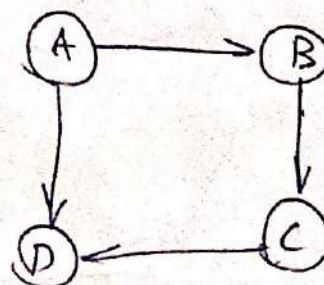
αν οδηγεί σε κορυφή με υψηλότερη προκαταρκτική αρίθμηση.

ΜΕΤΑΒΑΤΙΚΗ ΚΛΕΙΣΤΕΥΣΗ

Ένας κατευθυνόμενος γραφός ^{or} με τις ίδιες κορυφές με τον G που περιέχει μια ακμή ^(u,v) αν και μόνο αν στον γραφό G υπάρχει ένα κατευθυνόμενο μονοπάτι από την κορυφή u στη v .

→ Floyd Warshall

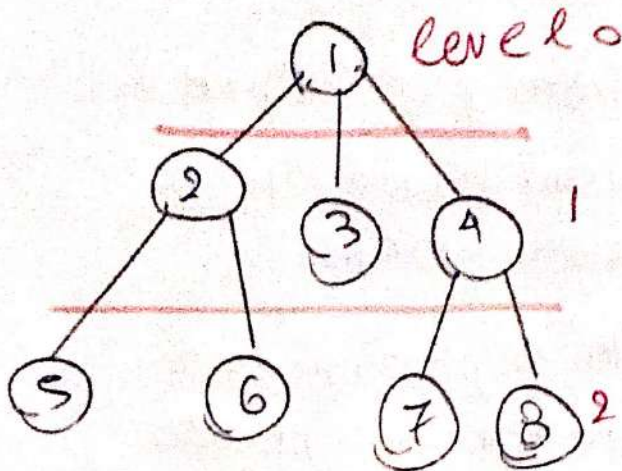
→ Αναδρομικός DFS



BFS

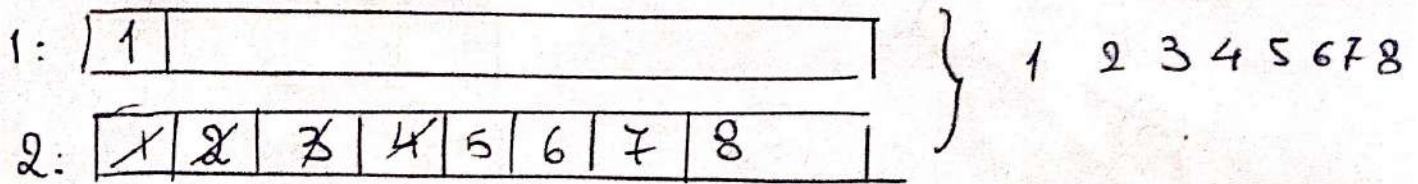
κατα μήκος διασχίση

level order

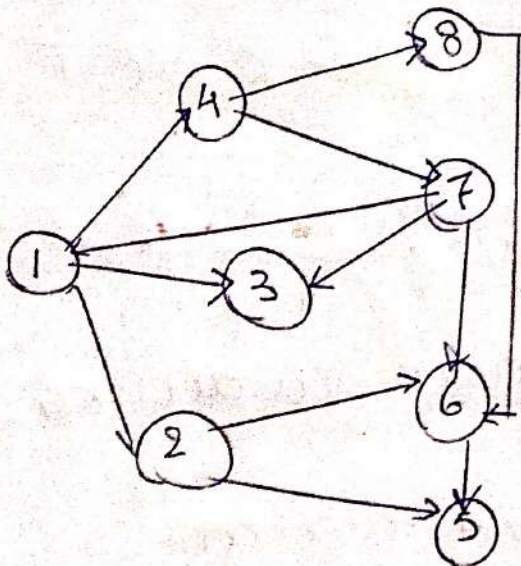


v.n	list
1	2 3 4
2	1 5 6
3	1
4	1 7 8
5	2
6	2
7	4
8	4

Να είναι η σειρά των κόμβων που επισκεψάσαστε ξεκινώντας από το 1
 Λεξογραμμή:



Βάζουμε το στοιχείο στην ουρά, το επισκεψάσαστε
 το αφαιρούμε από τη ουρά, και τοποθετούμε στην
 ουρά τα στοιχεία που αντιστοιχούν στη λίστα γει-
 αμίας του συγκεκριμένου κόμβου.



v.n	adj list
1	2 3 4 8
2	5 6
3	7
4	7 8
5	6
6	7
7	1 3 6 8
8	1 4

ΞΕΚΙΝΩΝΤΑΣ ΑΠΟ 1:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

1, 2, 3, 4, 5, 6, 7, 8

ΜΕΛΗΘΕΝΤΩΣΤΑ . ΜΕ ΔΙΟΤΑ ΓΙΑΤ $O(n^2)$

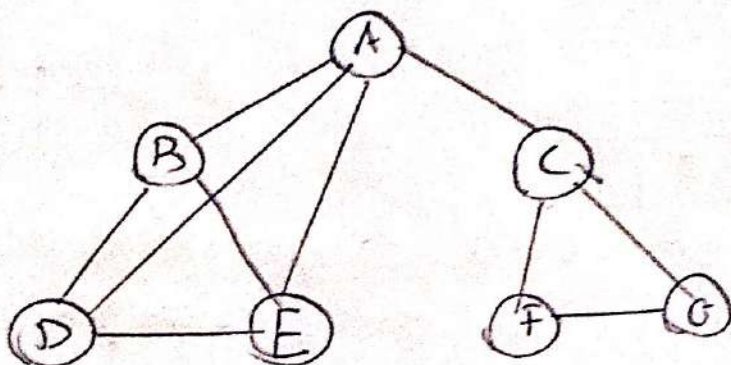
ΠΙΝΑΚΑ ΓΙΑΤ $O(n^2)$

ΧΩΡΙΣΤΗ $O(n)$

=> ΕΞΕΡΕΥΝΑ ΠΡΩΤΑ ΟΤΑ ΤΑ ΓΕΙΤΟΝΙΚΑ ΕΥΡΕΘΑ
ΠΡΩΤΟΥ «ΚΑΤΕΒΕΙ» ΒΑΘΥΤΕΡΑ ΠΕΝΤΡΑΒΟ

=> ο αλγόριθμος παρατίθεται στον κώδικα είναι ούτως

Άλλο Παράδειγμα με κώδικα



Vertex	Adj. list
A	B C D E
B	A D E
C	A F G
D	A B E
E	A B D
F	C G
G	F C

A	B	C	D	E	F	G		
---	---	---	---	---	---	---	--	--

A B C D E F G

BFS ΟΣΤΕ **ΚΩ**

ΚΑΤΕΥΘΥΝΩΣΗ
ΓΡΑΦΟ.

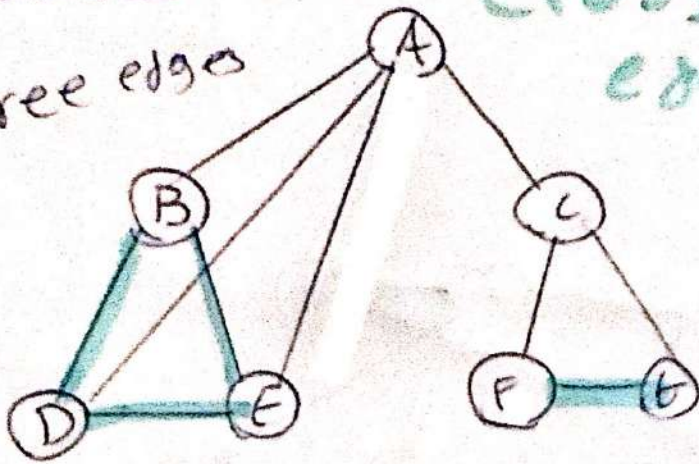
■ Ακριβή δέντρο: Επισκεφτόμαστε πρώτα όλους
τις κορυφές της

■ Εμφανισια ακμή: Δεν είναι ακμή του δέντρου

Μεσοπα

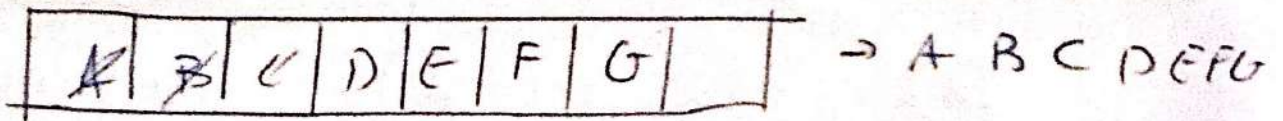
Cross edges

tree edges



Ξεκινώντας από τον κόμβο A

vert n	Adj list
A	B C D E
B	A D E
C	A F G
D	A B E
E	A B D
F	C G
G	F G



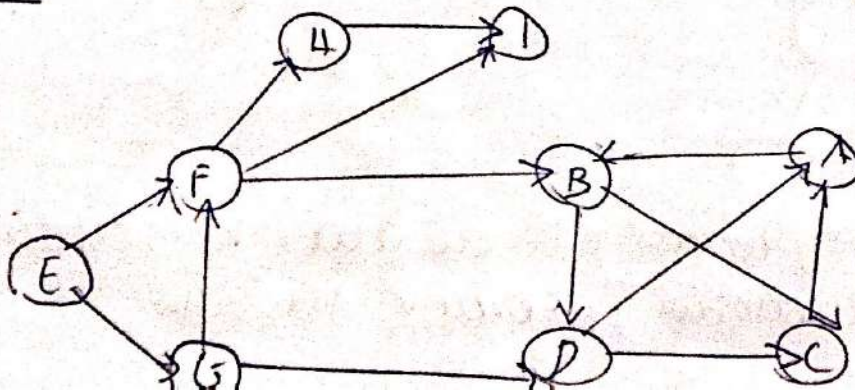
BFS σε κατευθυνόμενο γράφο

Ακμές δεύτερου : ορίζουν ένα δακτύλιο υπογράφο BFS

Back edges συνδέουν μια κορυφή με έναν από τους προγόνους της σε δακτύλιο υπογράφο BFS

Cross edges συνδέουν μια κορυφή με μια άλλη που δεν είναι ούτε πρόγονος ούτε απόγονος της σε δακτύλιο υπογράφο.

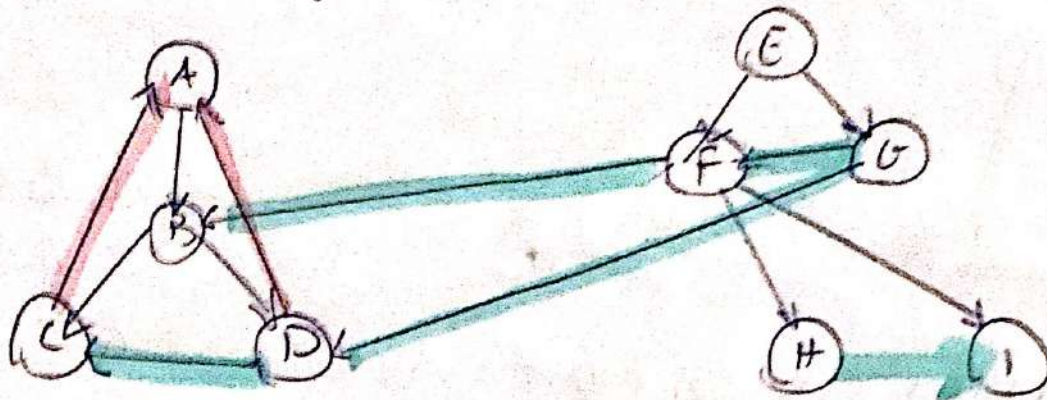
Δε υπάρχουν ακμές προωθησιμότητας



v. id	Adj list
A	B C
B	C D
C	A
D	A C
E	F G
F	H I
G	F
H	I
I	

Με ουσια: A, B, C, D, E, F, G, H, I

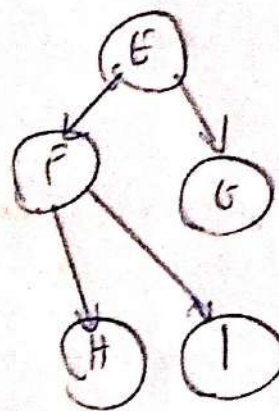
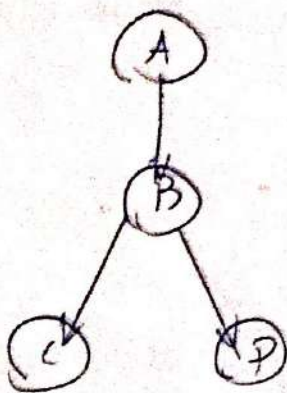
Από παραδειγμα



tree edges

cross edges

back edges



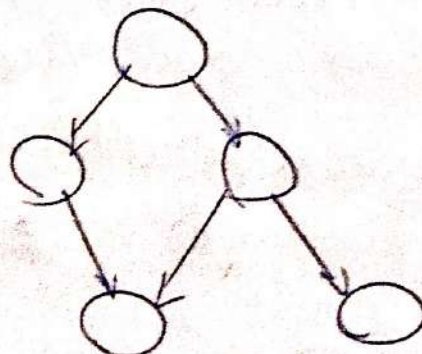
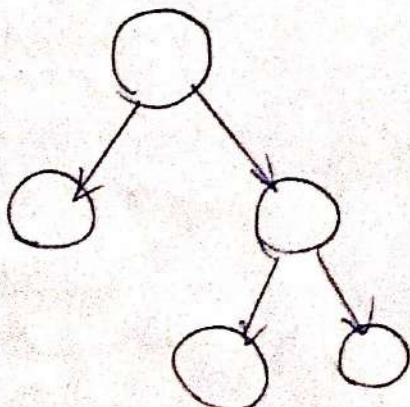
BFS forest

DAGS

κατευθυνόμενα ακυκλικά γραφήματα

↳ πιο γενικά από τα δέντρα

Ενας κατευθυνόμενος γραφός χωρίς κύκλους



Χρήσιμα στους μαθητές για την αναπαράσταση αριθμητικών επεξεργασιών, μεθόδων αξιών

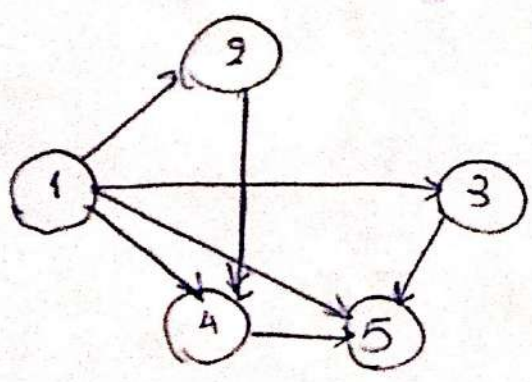
Αν βυθωντευθεί εστω μια back edge καρα DFS
 εστω ο ραυος δεν ειναι DAG (και αντιστρωσα)

! Σε κατατευθωντα γραφηματα ονιαδιμτα αλκυ
 προς μια κορυφη που εχει ηδη εξισκεφτει, υποδεικνυει
 ενα κωκτο

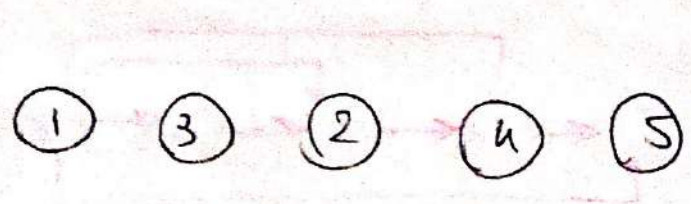
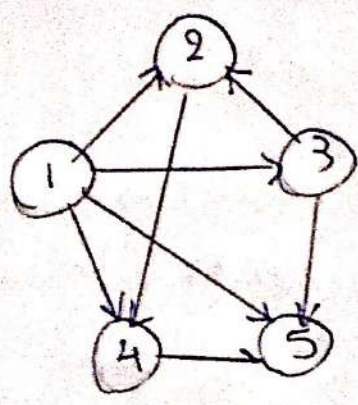
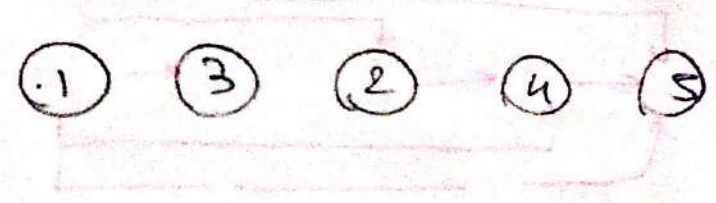
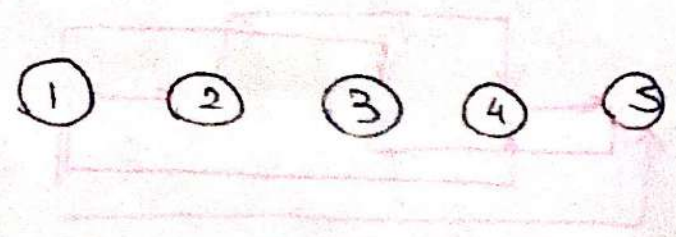
Σε κατατευθωνμενα γραφηματα αυτο ισχυει ην ο
 ηα back edges.

Τοπολογικη διαταξη

Δεν ειναι μοναδικη ενος dag

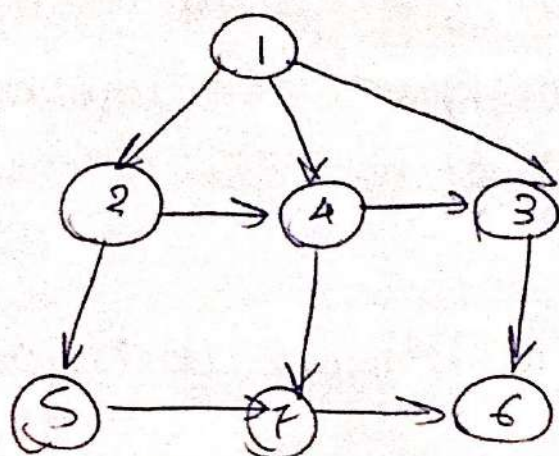


Τοπολογικη διαταξη



Μηνηδωκτα τοπολογικη
 ταξινομουσ . O (η τε)

№ 210 ΘΕΡΑ 2020



A. 1 → 2 → 5 → 4 → 7 → 3 → 6

1 2 3 4 5 6 7

1 3 6 4 7 2 5

1 4 7 2 3 5 6

1 2 4 3 5 7 6

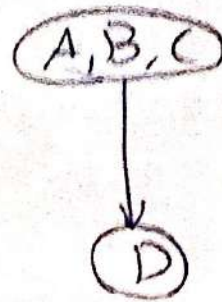
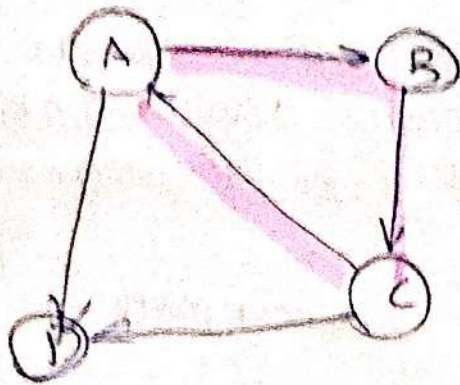
Εξαρτημένος γραφός

→ πάντα DAG

Σε ένα κατευθυνόμενο γραφικό κάποιες ακμές ανήκουν σε μια ισχυρή συνιστώσα και άλλες όχι. Οι ακμές που ανήκουν ονομάζονται, ακμές διασυνδέσεως (cross-component edges).

Μπορούμε να αναπαραστήσουμε τις διασυνδέσεις μεταξύ των συνιστωσών κατασκευάζοντας έναν εξαρτημένο γραφικό με ως γραφικό G

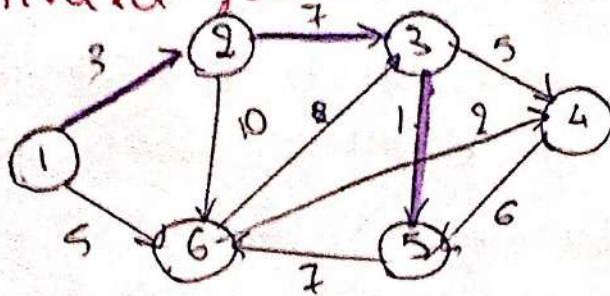
Οι κορυφές του ελεγχόμενου γραφού αντιστοιχούν στις ισχυρές συνιστώσες του G



Αλγόριθμος Kosaraju → υποαλγόριθμοι βήματα
 που απαιτούνται (CnT e) συνιστώσων

ΓΡΑΦΟΙ ΜΕ ΒΑΡΗ

Μεγεθύνεται γειτνίαση



	1	2	3	4	5	6
1	0	3	∞	∞	∞	5
2	∞	0	7	∞	∞	10
3	∞	∞	0	5	1	∞
4	∞	∞	∞	0	6	∞
5	∞	∞	∞	∞	0	7
6	∞	∞	8	2	∞	0

$T[i][j]$ = βάρος, αν υπάρχει
 ακμή $u_i \rightarrow u_j$

$T[i][i]$ = κύρια διαγώνιος

$T[i][j] = \infty$ αν δεν υπάρχει ακμή $u_i \rightarrow u_j$

Με διότες γειτνίαση

Κορυφές	Διότες γειτνίαση
1	(2,3) (6,5)
2	(2,7) (6,10)
3	(4,5) (5,1)
4	(5,6)
5	(6,7)

↓ κορυφές γειτνίαση
 (u, w)
 ↑ βάρος ακμής

Συντομότερο μονοπάτι

- Το **μήκος (ή βάρος)** ενός μονοπατιού είναι το άθροισμα των βαρών των ακμών του.
- Το συντομότερο μονοπάτι μεταξύ δύο κορυφών s και t είναι ένα κατευθυνόμενο από s στο t μονοπάτι από το s στο t τέτοιο ώστε κανένα άλλο μονοπάτι να μην έχει μικρότερο συνολικό βάρος.

π.χ. αν οι κορυφές 1 στην 5 shortest path:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Πρόβλημα συντομότερου μονοπατιού

Μια κορυφή ορίζεται ως **αφετηρία** και σελήας μας είναι να βρούμε το μήκος του συντομότερου μονοπατιού από την αφετηρία προς κάθε κορυφή του συνόλου V .

Απλοστοι αλγόριθμοι (Greedy)

Οι αλγόριθμοι για προβλήματα **βελτιστοποίησης** επεξεργάζονται σε διαδοχικά βήματα, όπου σε κάθε βήμα υπάρχουν μία ή περισσότερες πιθανές επιλογές.

Το πρόβλημα των συντομότερων μονοπατιών με κοινή αφετηρία, είναι πρόβλημα βελτιστοποίησης.

Ένας **απλοστοι αλγόριθμος** επιλέγει κάθε βήμα αυτό που του φαίνεται καλύτερο εκείνη τη στιγμή ελπίζοντας ότι αυτό τελικά θα οδηγήσει στην βέλτιστη λύση. Οι απλοστοι αλγόριθμοι δεν εγγυώνται πάντα τη βέλτιστη λύση.

Αλγόριθμος Dijkstra → απλοστοι για εύρεση συντομότερου μονοπατιού.
χρονική πολυπλοκότητα $O(n^2)$ → για αραιούς γραφούς
 $O(n \log n)$ για πυκνούς γραφούς
για εύρεση συντομότερου μονοπατιού από μία κορυφή σε όλες τις άλλες.

Αλγόριθμος Floyd $\rightarrow$ $O(n^3)$ ζευγαρωμένοι βρόχοι
Εύρεση συντομότερων μοναχιών.

Αλγόριθμος warshall $\rightarrow$ αν υπάρχει διαδρομή μεταξύ
δύο κόμβων ανεξαρτήτως του μήκους διαδρομής

Μητρα/πίνακας βάρων $\rightarrow$ υποθέτουμε ότι ο πίνακας C
είναι αρχικά ένας πίνακας γειτνίασης του γραφού.

- $C[i, j] = 1$ αν υπάρχει ακμή από την κορυφή i στη j
και ο αντίστροφος

Θέλουμε να υπολογίσουμε τον πίνακα A που

- $A[i, j] = 1$ αν υπάρχει διαδρομή από την κορυφή i στη j με
μήκος 1 ή μεγαλύτερο
- $A[i, j] = 0$ αν δεν υπάρχει τέτοια διαδρομή

Αυτός ο πίνακας A είναι η **μεταβατική κλειστότητα**
(transitive closure) του πίνακα γειτνίασης C .

Γενικά, η μεταβατική κλειστότητα ενός γραφού μας λέει
αν υπάρχει διαδρομή μεταξύ οποιωνδήποτε 2 κορυφών.
(Υπολογίζεται με διαδικασία παρόμοια του Floyd)

Γραφικό μάθημα $\rightarrow$ $O(n^3)$ όπως ο Floyd

Minimum Spanning Trees

spanning subgraph υπογράφοι που περιλαμβάνουν
όλες τις κορυφές του αρχικού γραφού

spanning tree spanning subgraph που είναι
και ελεύθερο δέντρο

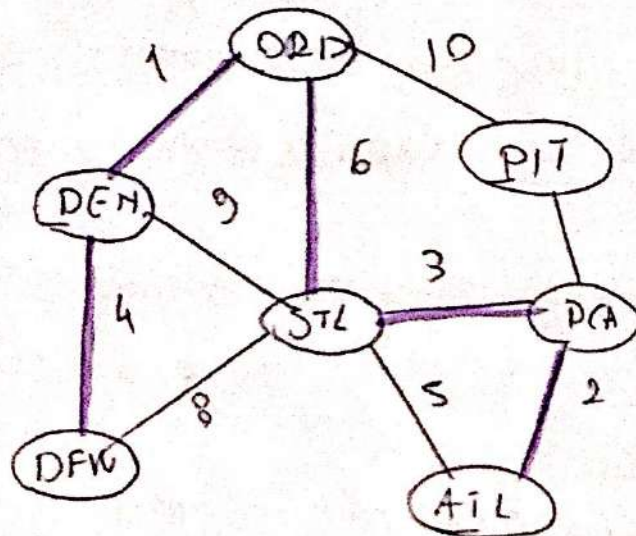
Minimum Spanning tree

η βαρύν του οποίου το σύνολο είναι εφικτό να δοθεί

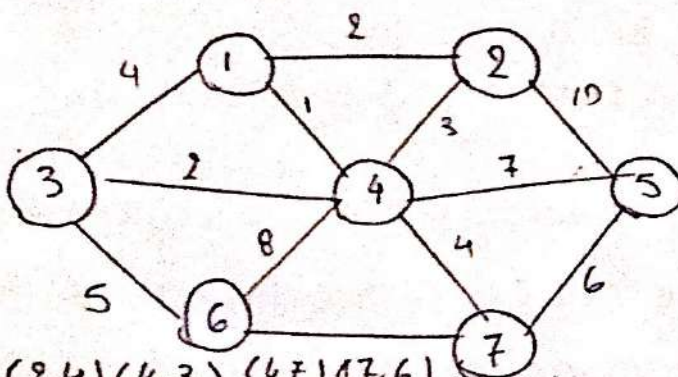
Spanning tree
βαρύν των αλφών

(Δίκτυα επικοινωνιών / μεταφορών)

Παραδείγματα



MST



Μιας αλφής οημα-
τιζούν ένα εφικτό
επικοινωνίας δέντρο

A) (1,2) (2,4) (4,3) (4,7) (7,6) (6,3)

B) (1,2) (1,3) (3,4) (4,7) (7,6) (7,5)

Γ) (3,1) (1,2) (2,5) (5,7) (7,6) (6,3) (5,4) (3,4)

Δ) (1,3) (3,4) (4,2) (4,7) (7,6) (7,5)

Ε) (1,2) (1,4) (3,4) (4,7) (6,7) (5,7)

$$A) 2 + 3 + 2 + 4 + 6 + 5 = 22 \text{ (δεν περιλαμβάνει το 5) } \times$$

$$B) 2 + 4 + 2 + 4 + 1 + 6 = 19 \times$$

$$Γ) 4 + 2 + 10 + 6 + 1 + 5 + 7 + 2 = - \text{ Σημειώνεται κήδος}^*$$

$$Δ) 4 + 2 + 3 + 4 + 1 + 6 = 20 \times$$

$$Ε) 2 + 1 + 2 + 4 + 1 + 6 = 16^V \text{ (δεν σημειώνονται κήδοι, περιλαμβάνονται όλες οι κορυφές)}$$

cycle property

- Έστω T ένα MST του G γραφού. Έστω e μια ακμή του G που δεν ανήκει στο T και έστω C ο κύκλος που σχηματίζεται προσθέτοντας την e στο T . Για κάθε ακμή f του C ισχύει $\text{βάρος}(f) \geq \text{βάρος}(e)$

partition property

Έστω ότι έχουμε μια διαμέριση των κορυφών του γραφήματος G σε δύο υποσύνολα U και V . Έστω e η ακμή ελάχιστου βάρους που συνδέει κορυφές από διαφορετικά σύνολα. (δηλαδή που διασχίζει τη διαμέριση). Υπάρχει ένα MST του G που περιέχει την ακμή e .

Αλγόριθμος Kruskal

- λίστα γειτνιασών
 - δαδός δεικτών
 - priority queue
- } (Antileogm) ^{βαθμολογία ακμών}

Βρίσκει το MST

Αλγόριθμος Prim Jarník

→ Παρόμοιος με Dijkstra

→ ο γραφός αναπαρίσταται με λίστα γειειασών

→ χρησιμοποιούμε ουρά προτεραιότητας

→ **χρονική πολυπλοκότητα**: $O(n^2 \log n)$

Βρίσκει MST. Διαφέρει από τον Kruskal ως προς τη στρατηγική επιλογής ακμών